

基于 CRITIC-TOPSIS 与场景化决策的主流大语言模型综合性能评价研究

摘要

本文面向公开资料条件下的大语言模型综合评价问题，建立“数据冻结-可比性清洗-指标筛选-客观赋权-场景评价-成本效益-稳健性检验”的完整流程。评价体系采用能力、工程效率、经济成本与能耗边界的分层结构。以 2026 年 8 月 17 日为数据截止日期，从 9 个初始候选模型中确定 6 个最终研究对象，并从高难推理、长文本、代码、专业任务、多模态与文档理解等维度冻结 9 个核心指标。原始缺失值保持为 NA，不进行估算、插值或零值填充。

问题一采用相关性分析、CRITIC 客观赋权与 TOPSIS 综合评价。Claude Fable 5、Kimi K3、GPT-5.6 Sol 分列通用能力综合排名前三；GLM-5.2 因仅有 4/9 项核心成绩而不进入主排名。问题二按照“50% CRITIC 权重 + 50% 题意驱动场景偏好”构造三类基准场景权重；在该等比例基准下，Kimi K3 在科研长文本场景升至第一，Claude Fable 5 在日常对话与代码开发场景排名第一。问题三以 100 万输入 token 与 20 万输出 token 为标准工作负载，得到 Kimi K3 与 Claude Fable 5 构成的性能-成本 Pareto 前沿；严格部署效率比较仅覆盖配置兼容的 4 个模型。

熵权法对照与 36 个单因素权重扰动均保持前三名次序，最低 Kendall 秩相关系数为 0.8。成本-性能回归仅基于 5 个可排名模型，故只解释为当前样本内的探索性关系。本文完整保留模型版本、评测设置、数据来源与时间截面，为不同场景和预算条件下的模型选型提供可复核依据。

关键词：大语言模型；CRITIC；TOPSIS；场景评价；Pareto 前沿；敏感性分析

目录

1	问题重述与分析	3
1.1	问题重述	3
1.2	问题分析	3
2	模型假设与数据边界	4
2.1	符号说明	4
3	数据来源、模型池与核心指标	4
3.1	候选模型筛选	4
3.2	潜在指标池与核心指标筛选	5
3.3	数据来源与缺失处理	7
4	问题一：CRITIC-TOPSIS 综合评价	8
4.1	相关性与指标保留	8
4.2	CRITIC 客观赋权	8
4.3	TOPSIS 综合评价	10
5	问题二：三类场景评价	12
5.1	场景权重构造	12
5.2	跨场景排名差异的内在机理	14
6	问题三：性能-成本与部署效率	15
6.1	货币成本和 Pareto 前沿	15
6.2	算力能耗数据边界	17
6.3	严格可比部署效率子分析	17
6.4	成本-性能探索性关系	18
7	稳健性分析	19
7.1	方法与参数稳健性	19
7.2	样本集合敏感性检验	19
7.3	场景组合系数敏感性	20
8	模型评价与局限	21
8.1	方法特点	21
8.2	局限性	21
8.3	改进方向	22
9	结论与建议	22
10	AI 工具使用声明	22
A	支撑材料文件清单	24
B	完整源程序代码	29

1 问题重述与分析

1.1 问题重述

大语言模型在科学推理、长文本理解、代码开发、多模态任务和专业工作等方面表现不同，调用价格与推理效率又会影响实际部署。单一 Benchmark 或单一价格指标无法同时反映能力、成本和应用适配性。本文需要完成三项任务：建立分层综合评价体系；针对科研长文本、日常对话和代码开发构造场景化评价；结合货币调用成本与严格可比效率数据提出选型建议。

1.2 问题分析

为避免不同版本、工具协议和测试配置被误合并，本文首先冻结候选模型、严格比较 cohort 与数据截止日，再进行指标筛选和建模。总体技术路线为：公开数据审计 → 固定可比 cohort → 缺失与单位检查 → 相关性分析 → CRITIC 客观赋权 → TOPSIS 排序 → 场景偏好组合 → 成本与效率分析 → 稳健性检验。

本文总体建模流程如图 1 所示。

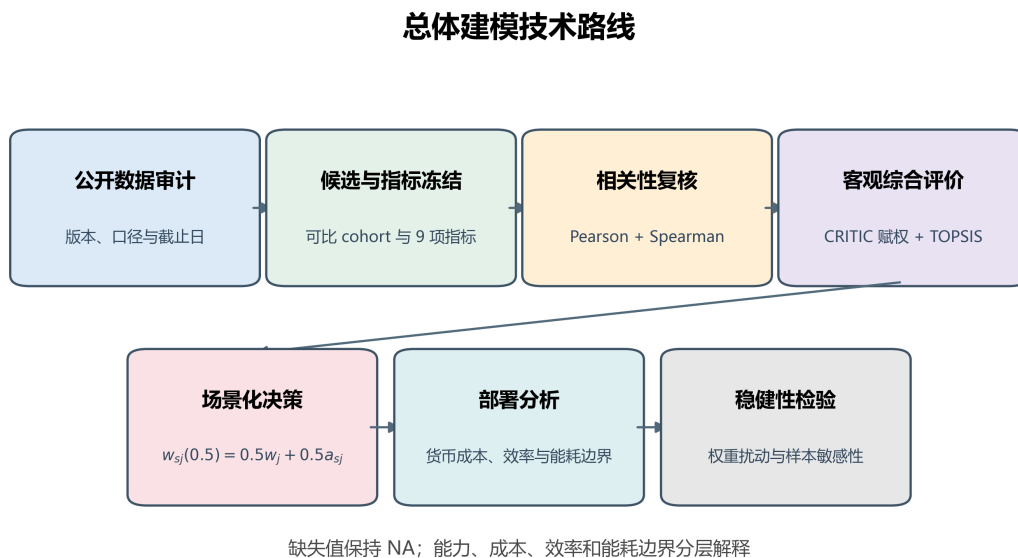


图 1 总体建模技术路线

1.2.1 问题一分析

先依据可比性、覆盖率、判别力与能力语义，从候选模型和潜在指标中冻结核心评价矩阵；再用 Pearson、Spearman 相关分析识别冗余风险，以 CRITIC 确定客观权重、TOPSIS 完成综合排序，并结合分项表现分析 Kimi K3 的优势与不足。

1.2.2 问题二分析

针对科研长文本、大众日常对话和代码开发三类应用需求，分别设置透明的场景偏好权重，与 CRITIC 权重组合后重新进行 TOPSIS 排序；通过能力画像、加权空间及正负理想解距离解释跨场景差异，并用 λ 敏感性检验场景结论的条件性。

1.2.3 问题三分析

以固定 token 工作负载下的 API 货币成本衡量直接使用成本，通过 Pareto 前沿和预算约束给出选型建议；另在严格兼容子样本中比较工程效率，审计能耗数据的可比性边界，并以小样本探索性回归描述成本与性能的样本内关系。

2 模型假设与数据边界

- (1) 同一冻结 cohort 内的成绩可横向比较，但不同厂商披露的最高 reasoning effort 不等价于完全相同的计算预算。
- (2) 所有数据只代表 2026 年 8 月 17 日截面之前公开的信息；滚动效率观测不是模型的永久属性。
- (3) 9 个核心能力指标均为效益型，数值越大表示相应能力越强。
- (4) 缺失值不等于零，不进行估算、插值或任何形式的插补。
- (5) 主 TOPSIS 采用 complete-case：仅对 9 项核心成绩完整的模型排序。
- (6) 场景偏好权重是依据题意与应用需求作出的透明设定，不代表专家调查结果。

2.1 符号说明

表 1 主要符号说明

符号	含义	单位	符号	含义	单位
z_{ij}	模型 i 的第 j 项标准化指标值	无量纲	w_j	第 j 项指标的 CRITIC 客观权重	无量纲
C_j	第 j 项指标的 CRITIC 信息量	无量纲	r_{jk}	指标 j 与 k 的 Pearson 相关系数	无量纲
D_i^+	模型 i 到正理想解的距离	无量纲	D_i^-	模型 i 到负理想解的距离	无量纲
S_i	模型 i 的 TOPSIS 相对贴近度	无量纲	a_{sj}	场景 s 中第 j 项指标的人工偏好权重	无量纲
λ	CRITIC 客观权重占比	无量纲	M_i	模型 i 在标准工作负载下的 API 货币成本	USD
N_{in}	输入 token 数量	token	N_{out}	输出 token 数量	token
$p_{i,in}$	模型 i 的输入单价	USD/10 ⁶ tokens	$p_{i,out}$	模型 i 的输出单价	USD/10 ⁶ tokens

3 数据来源、模型池与核心指标

3.1 候选模型筛选

综合考虑模型代表性、版本明确性、公开时间、数据截止日期以及严格可比指标覆盖情况，从初始 9 个候选模型中确定 6 个最终研究对象。Gemini 3.1 Pro Preview、DeepSeek V4 Pro 0813 与 Qwen3.8 2.4T A95B 因预览状态或距数据截止日期发布时间过短、严格

可比分项覆盖不足，因此不进入核心综合排名，但原始资料仍予保留。75% 覆盖率是核心指标约束之一，并非倒推缩小模型池的唯一理由。

表 2 候选模型及冻结状态

序号	模型	精确版本	状态	说明
1	Kimi K3	kimi-k3	final	题目指定重点对象,进入核心研究池
2	GPT-5.6 Sol	gpt-5.6-sol	final	分项公开数据充分
3	Claude Fable 5	claude-fable-5	final	能力数据充分,效率 fallback 另行排除
4	Claude Opus 4.8	claude-opus-4.8	final	成熟对照数据充分
5	GPT-5.5	gpt-5.5	final	稳定基线模型
6	GLM-5.2	GLM-5.2	final	保留国产开源代表,缺失项保持 NA
7	Gemini 3.1 Pro Preview	gemini-3.1-pro-preview	excluded	preview 且严格可比覆盖不足
8	DeepSeek V4 Pro 0813	DeepSeek V4 Pro 0813	excluded	距截止日 4 天, 分项覆盖不足
9	Qwen3.8 2.4T A95B	Qwen3.8 2.4T A95B	excluded	距截止日 5 天, 分项覆盖不足

3.2 潜在指标池与核心指标筛选

为避免直接从少量熟悉的 Benchmark 中选取指标，本文先按能力构念建立潜在指标池。候选池共含 17 个概念指标：GPQA Diamond、HLE-Full、FrontierMath、MathVision、SWE-Bench Pro、DeepSWE、Terminal-Bench、FrontierSWE、SciCode、GDPval-AA v2、Agents’ Last Exam、OfficeQA Pro、MMMU-Pro、OmniDocBench、CharXiv RQ、AA-LCR 与 GraphWalks BFS，覆盖高难推理、数学、代码、专业任务、多模态、文档理解和长文本等能力维度。由于 HLE-Full、MMMU-Pro、CharXiv RQ 与 MathVision 区分有无工具设置，FrontierMath 和 GraphWalks 又按难度或上下文档位拆分，17 个概念指标在覆盖率统计表中展开为 23 个 Benchmark 条目；Artificial Analysis Intelligence Index 另作模型筛选辅助，不计入这 23 项，也不进入能力综合评价。

指标筛选遵循“先可比、再覆盖、后判别与语义复核”的顺序，实际执行过程见表 3。其中，75% 仅是覆盖率约束；来源可追溯、版本与测试设置一致、指标方向明确、具有非零变异以及能力语义不重复同样是必要条件。

表 3 潜在指标到最终核心指标的实际筛选链

阶段	输入	判据	输出	处理结果
1	17 个概念指标	区分工具策略、难度层级与上下文档位	23 条	结构化展开；综合指数另作辅助
2	23 个 Benchmark 条目	来源、版本、测试设置与 cohort 可比，覆盖率不低于 75%	9 项	冻结核心候选；不插补、不跨口径拼接
3	9 项核心候选	正向化检查且样本标准差非零	9 项	九项均为正向能力指标且具有区分度
4	9 项核心候选	$ r $ 或 $ \rho \geq 0.85$ 时进入冗余复核	4 对	只识别候选相关对，不自动删除
5	4 对候选相关对	综合语义维度、来源、覆盖及场景用途	9 项	删除 0 项，最终保留全部九项

最终冻结 9 个核心指标。4 个指标达到 6/6 覆盖，5 个指标达到 5/6，即 83.3%；全部达到预设的 75% 模型覆盖要求。5 个缺失项均来自 GLM-5.2，论文不掩盖这一事实。层次上，综合评价还涉及工程效率、成本与能耗边界，但问题一的能力主排名只使用下表九项 Benchmark；工程效率与成本在问题三中另行评价，避免不同性质指标混为同一层。

表 4 最终核心指标及覆盖情况

指标	维度	覆盖	覆盖率	缺失模型
GPQA Diamond	高难科学推理	6/6	100.0%	无
HLE-Full (no tools)	高难开放推理	5/6	83.3%	GLM-5.2
AA-LCR	长上下文推理	6/6	100.0%	无
SciCode	科学编程	6/6	100.0%	无
GDPval-AA v2	专业任务	6/6	100.0%	无
MMMU-Pro (no tools)	多模态推理	5/6	83.3%	GLM-5.2
OmniDocBench	文档理解	5/6	83.3%	GLM-5.2
CharXiv RQ (no tools)	科研文档推理	5/6	83.3%	GLM-5.2
MathVision (no tools)	多模态数学	5/6	83.3%	GLM-5.2

本文据此建立表 5 所示的分层综合评价体系。问题一的 TOPSIS 得分严格定位为“通用能力核心得分”，而非混入价格、时延和能耗后的唯一终极总分。

表 5 大语言模型综合评价体系的分层结构

评价层	指标或信息	在本文中的作用
能力性能层	9 项核心 Benchmark	采用 CRITIC-TOPSIS 形成通用能力核心得分
工程效率层	TTFT、Output Speed、500-token E2E	仅在 4 个严格兼容模型的子样本中比较
经济成本层	API 输入/输出价格、固定工作负载成本	用于性能-成本 Pareto 与预算约束选型
算力能耗层	理论上关注 J/token、kWh/任务等	完成公开数据可得性审计；不伪造量化结果

四层不强行合成为单一总分：能力 Benchmark 覆盖 5 个完整排名模型，严格工程效率仅覆盖 4 个兼容模型，而能耗又缺少统一可比观测；若直接混合，将导致样本集合、测量口径和数据质量不一致。故价格与工程属性在问题三作为决策约束和落地属性分析，既覆盖综合评价要求，也避免重复计权。

3.3 数据来源与缺失处理

Benchmark 原始表含 148 条记录；模型元数据含 9 条模型记录及 79 条逐字段证据；效率数据含 9 条模型记录及 27 条逐指标证据。核心能力数据主要来自官方技术报告中的统一对照表，并辅以 Benchmark 榜单和独立第三方来源。每条记录保留版本、test setting、source name、URL 与检索日期；不同工具策略、agent scaffold 或快照不混入同一核心 cohort。

表 6 论文主要数据来源

数据类别	数据内容	主要来源	来源类型	检索日期
能力 Benchmark	9 项核心成绩	Kimi K3 Technical Report; OpenAI 与 Z.ai 官方报告作辅助证据 [1, 4, 9]	厂商官方报告	2026-08-16/17
模型身份	provider、exact version、发布日期	Moonshot、OpenAI、Anthropic 与 Z.ai 官方模型资料 [3, 5, 7, 9]	第一方官方文档	2026-08-16/17
API 价格	输入、输出标准价	模型厂商官方定价页面 [6, 8, 10]	第一方定价文档	2026-08-16/17
部署效率	TTFT、Output Speed、E2E	Artificial Analysis API Provider Performance Benchmarking[2]	独立评测平台	2026-08-16
能耗边界	可得性与可比性	对模型官方资料、公开定价资料及独立评测数据的可得性审计	公开数据审计	2026-08-17

冻结的 9 项核心成绩均主要采用 Kimi K3 Technical Report 的统一 cohort，以保持测试设置一致。OpenAI、Z.ai 官方报告、Benchmark 官方资料和 Artificial Analysis 等作为辅助来源证据，但不替换冻结 cohort，也不宣称已完成独立交叉验证。数据整理时逐条记录来源 URL、比较口径和检索日期，以保证核心数字可追溯。

AI 使用标注：本节的数据来源一致性检查与数据核验使用了 AI 工具辅助，最终数据取舍、缺失值保留和截止日期均由参赛队人工确认，详见支撑材料《人工智能工具使用详情》。

Raw 层缺失统一保持为 NA。描述统计与 Pearson、Spearman 相关性采用逐对完整观测；主 TOPSIS 采用 complete-case。GLM-5.2 只有 4/9 项核心成绩，因此不进入主排名。本文没有将缺失值写为零，也没有进行跨来源平均、擅自取最大值或任何形式的插补。

不同量纲指标采用效益型极差标准化：

$$z_{ij} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}. \quad (1)$$

标准化在每项指标内部独立完成，因此百分制、Elo 等不同原始量纲不直接相加，而是统一转换为当前候选集合内的相对位置。若某指标仅有 5 个满足同口径要求的有效观测，则该指标的最大值与最小值仅由这 5 个有效模型确定，缺失模型不参与极值计算。

4 问题一：CRITIC-TOPSIS 综合评价

4.1 相关性与指标保留

9 个指标的标准差均非零。本文同时计算逐对完整观测上的 Pearson 系数 r 与 Spearman 系数 ρ ，并将 $|r|$ 或 $|\rho| \geq 0.85$ 设为高相关候选的冗余审查阈值；该阈值仅用于触发后续语义复核，并非统计显著性检验阈值。由于共同观测模型仅为 5–6 个，本文不依据相关系数进行总体参数显著性推断。由此得到 4 组候选对：GPQA Diamond–MMM–Pro ($r = 0.9500$, $\rho = 0.9211$)、GPQA Diamond–MathVision ($r = 0.8611$, $\rho = 0.6669$)、SciCode–CharXiv RQ ($r = 0.9326$, $\rho = 0.9747$) 以及 MMM–Pro–MathVision ($r = 0.9349$, $\rho = 0.8208$)，共同观测量均为 $n = 5$ 。这些关系只表明当前小样本内存在较高线性或秩相关，是冗余判断的辅助证据，并不能证明指标语义等价。前置筛选已将 17 个概念指标展开为 23 个条目，再按可比性、覆盖率与判别力缩减至 9 项；相关性分析的作用是对最终候选进行冗余风险检查，而非机械继续压缩数量。综合覆盖率、来源口径、能力构念和后续场景用途复核后，高相关指标仍测量不同能力构念并承担不同场景解释作用，故 4 组候选对均保留；删除 0 项表示相关性经语义与场景信息复核后尚不足以支持删项，而非相关性分析无效。

图 2 直观展示 Pearson 相关结构；Spearman 结果仍按正文所述共同参与冗余复核。

4.2 CRITIC 客观赋权

本文选择 CRITIC，是因为九项指标均为客观公开 Benchmark 结果，且需要同时利用指标的样本变异性与指标间冲突性，减少主权重完全依赖主观判断。AHP 需要真实专家判断矩阵，本研究未实施专家调查，故不将其用作主权重；熵权法主要依赖离散程度，本文将其保留为稳健性对照且所得排名一致；PCA 更强调降维和线性组合，其主成分不如原始 Benchmark 权重便于解释。因此，CRITIC 与本题存在相关结构且强调可解释客观赋权的需求更匹配。

*AI 使用标注：*相关程序实现与调试使用了 AI 代码辅助工具；模型公式、参数、输入数据和最终计算结果均由参赛队复核。

CRITIC 方法同时利用标准化指标的变异性与指标间冲突性 [12]。令第 j 个指标的

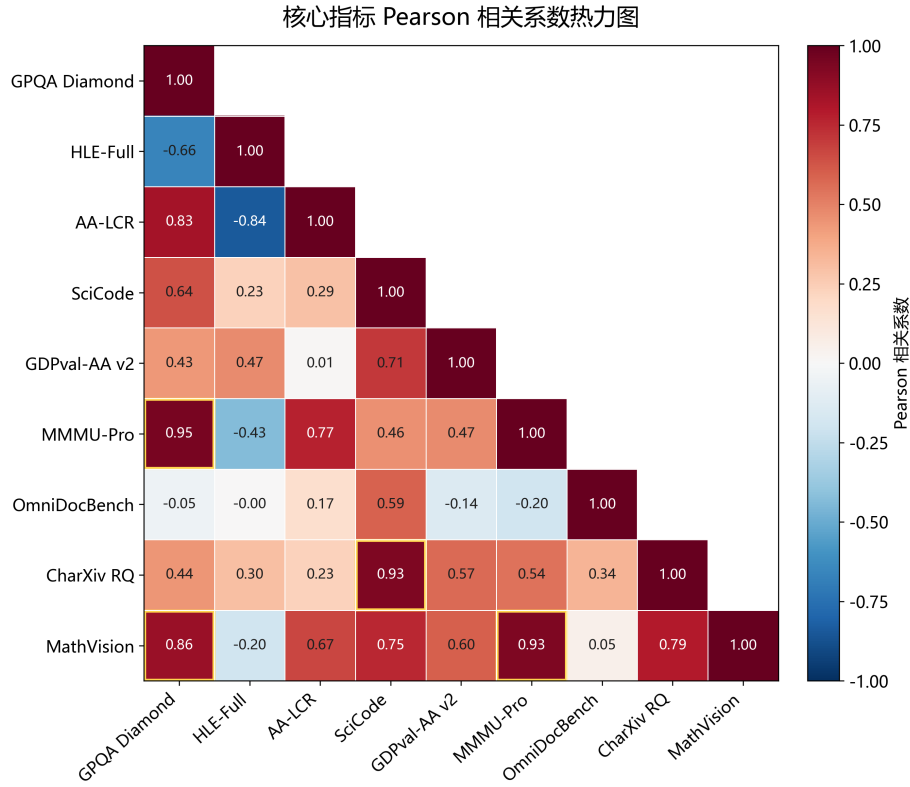


图 2 核心指标 Pearson 相关系数热力图

总体标准差为 σ_j ，逐对完整观测上的 Pearson 相关系数为 r_{jk} ，则

$$C_j = \sigma_j \sum_{k=1}^n (1 - r_{jk}), \quad w_j = \frac{C_j}{\sum_{j=1}^n C_j}. \quad (2)$$

表 7 CRITIC 最终权重

指标	权重	权重排序
GPQA Diamond	0.1039	5
HLE-Full	0.2013	1
AA-LCR	0.1270	3
SciCode	0.0674	9
GDPval-AA v2	0.1167	4
MMMU-Pro	0.0865	6
OmniDocBench	0.1483	2
CharXiv RQ	0.0734	8
MathVision	0.0756	7

HLE-Full、OmniDocBench 与 AA-LCR 权重依次居前三，SciCode 权重最低。权重反映当前样本中的变异性与冲突性，不等同于脱离样本和场景的绝对重要程度。

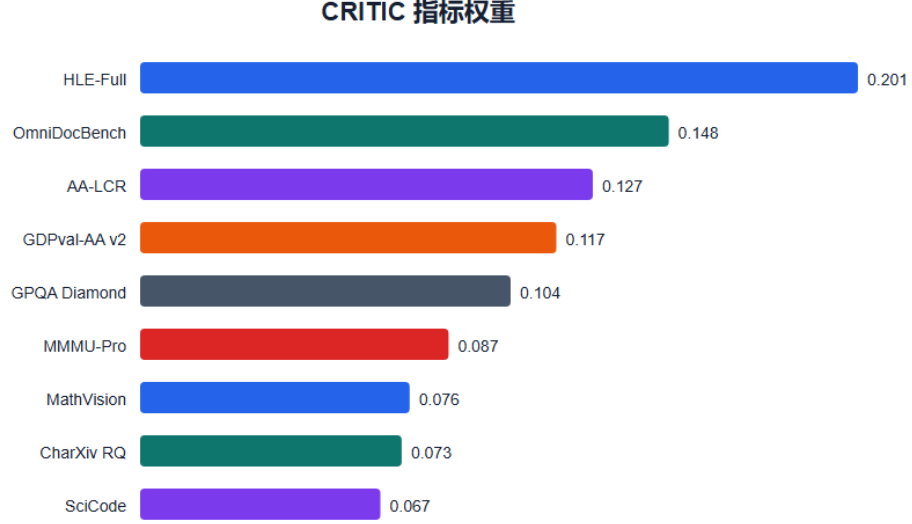


图 3 CRITIC 指标权重

4.3 TOPSIS 综合评价

TOPSIS 以模型到正、负理想解的距离刻画相对贴近程度 [11]。首先构造加权标准化矩阵：

$$v_{ij} = w_j z_{ij}. \quad (3)$$

正、负理想解为

$$v_j^+ = \max_i v_{ij}, \quad v_j^- = \min_i v_{ij}. \quad (4)$$

模型到两个理想解的欧氏距离为

$$D_i^+ = \sqrt{\sum_{j=1}^n (v_{ij} - v_j^+)^2}, \quad (5)$$

$$D_i^- = \sqrt{\sum_{j=1}^n (v_{ij} - v_j^-)^2}. \quad (6)$$

相对贴近度为

$$S_i = \frac{D_i^-}{D_i^+ + D_i^-}. \quad (7)$$

式 (3)–(7) 与程序实际使用的加权矩阵和距离完全一致。

表 8 通用能力 TOPSIS 核心得分与排名

排名	模型	TOPSIS 得分	状态
1	Claude Fable 5	0.7254	完整样本排名
2	Kimi K3	0.5933	完整样本排名
3	GPT-5.6 Sol	0.5178	完整样本排名
4	GPT-5.5	0.4410	完整样本排名
5	Claude Opus 4.8	0.3902	完整样本排名
—	GLM-5.2	NA	4/9，数据不足不排名

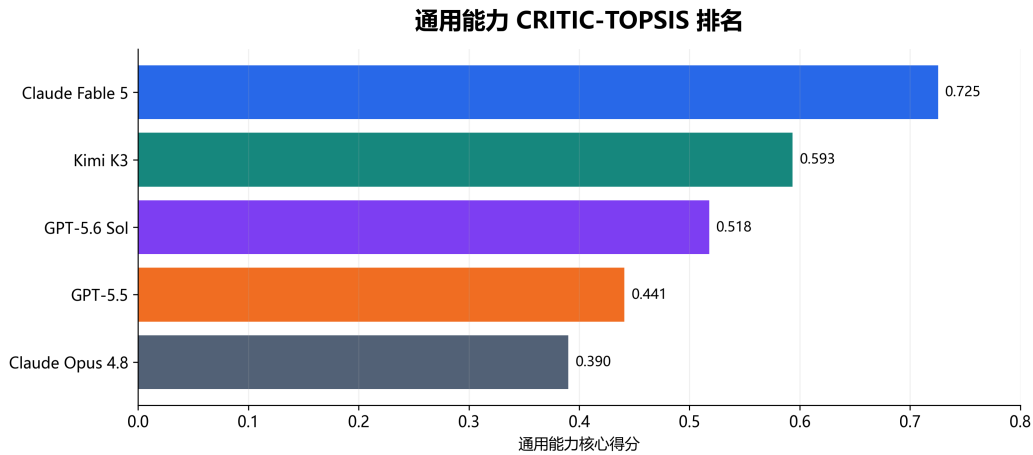


图 4 通用能力综合排名

Kimi K3 在 AA-LCR 与 OmniDocBench 上排名第一，在 GPQA Diamond、SciCode 和 MMMU-Pro 上排名第二，优势集中在长上下文、文档理解、高难科学推理与科学编程；GDPval-AA v2、CharXiv RQ 与 MathVision 分别处于第 3、第 2 和第 3，体现其专业任务、科研图表和多模态数学能力处于中上位置。其 HLE-Full 标准化得分为 0.1765，在 5 个完整模型中位列第 4，是最明确的能力短板。场景上，Kimi 在 $\lambda = 0.5$ 的科研长文本场景排名第一，在日常对话和代码开发场景均排名第二，且科研榜首具有 λ 条件性；成本上其标准工作负载为 6 美元并位于 Pareto 前沿。严格效率结论仅依据 4 模型兼容子样本，不使用 Fable 或 GLM 的不兼容观测作横向推断。

图 5 将 9 项核心指标按冻结能力维度映射为 8 个能力画像维度，其中高难度知识与逻辑推理由 GPQA Diamond 和 HLE-Full 按其 CRITIC 权重在维度内归一聚合，其余维度各对应一项核心指标。该图仅用于描述完整样本中的能力画像，不参与 CRITIC 赋权或 TOPSIS 排名。

GLM-5.2 局部观察。 基于四项共同可观测指标的局部比较结果显示，在六模型共同有观测的 GPQA Diamond、AA-LCR、SciCode 与 GDPval-AA v2 上，GLM-5.2 分别位列第 5、第 4、第 6 和第 5；其相对各指标领先模型的差距依次为 2.9 个百分点、3.4 个百分点、9.7 个百分点和 237 Elo。该比较仅用于描述共同观测维度，不构成九项完整综合得分，也不用于推断 GLM-5.2 的主排名；其余 5 项缺失仍保持 NA，不进行插补。

另据 Z.ai 官方资料，GLM-5.2 的独立 HLE 成绩为 40.5，但其测试来源与本文冻结 cohort 不一致，因此仅作为补充证据，不用于替换主矩阵中的 HLE 缺失值 [9]。

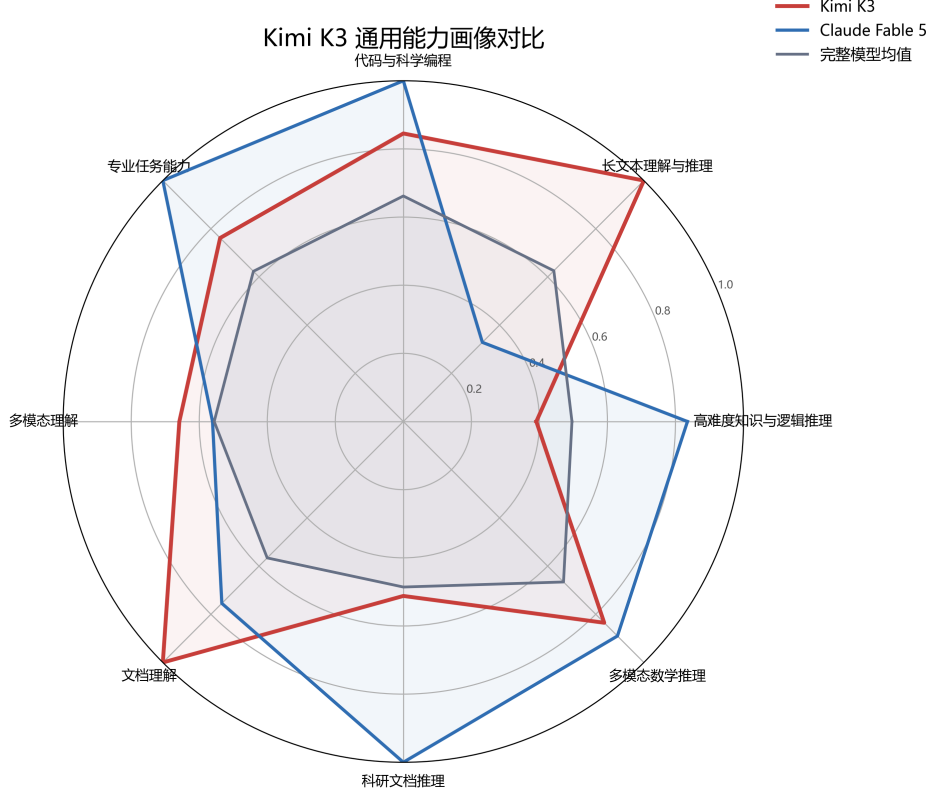


图 5 Kimi K3 通用能力画像对比

5 问题二：三类场景评价

5.1 场景权重构造

对场景 s ，令 a_{sj} 为依据题意与应用需求设定的人工场景偏好， λ 为 CRITIC 客观权重占比，则组合权重为

$$w_{sj}(\lambda) = \lambda w_j + (1 - \lambda) a_{sj}. \quad (8)$$

本文以 $\lambda = 0.5$ 作为正式基准，即对 CRITIC 客观信息与题意驱动的场景偏好给予等比例组合。该取值用于构造透明、易解释的基准场景，并非由优化程序或专家调查得到的唯一最优比例。三组 a_{sj} 与 $w_{sj}(0.5)$ 均非负且和为 1；本文没有实施专家调查，因此不将 a_{sj} 称为专家权重。使用 $w_{sj}(0.5)$ 替代通用 CRITIC 权重后，按照式 (3)–(7) 重新计算场景 TOPSIS 得分。

表 9 列出正式人工场景偏好 a_{sj} 。这些数值依据赛题定义的应用需求与各指标能力语义进行透明的规则化设定：科研长文本突出长上下文、科研文档与高难推理；日常对话更重视专业任务泛化、多模态理解与任务均衡；代码开发显著提高与科学编程直接相关的 SciCode。它们属于场景先验而非统计估计量，故本文进一步以 λ 敏感性分析检验其对结论的影响。

表 9 三类场景人工偏好权重设定

指标	科研长文本 a_{sj}	日常对话 a_{sj}	代码开发 a_{sj}
GPQA Diamond	0.12	0.12	0.12
HLE-Full	0.12	0.12	0.08
AA-LCR	0.25	0.10	0.08
SciCode	0.05	0.03	0.40
GDPval-AA v2	0.08	0.20	0.12
MMMU-Pro	0.08	0.15	0.05
OmniDocBench	0.12	0.10	0.05
CharXiv RQ	0.13	0.06	0.05
MathVision	0.05	0.12	0.05
合计	1.00	1.00	1.00

正式最终场景权重不另行人工设定，而是由 $w_{sj}(0.5) = 0.5w_j + 0.5a_{sj}$ 计算得到。三类场景的最终组合权重如图 6 所示，其中展示的是 $w_{sj}(0.5)$ ，而非人工偏好 a_{sj} 。

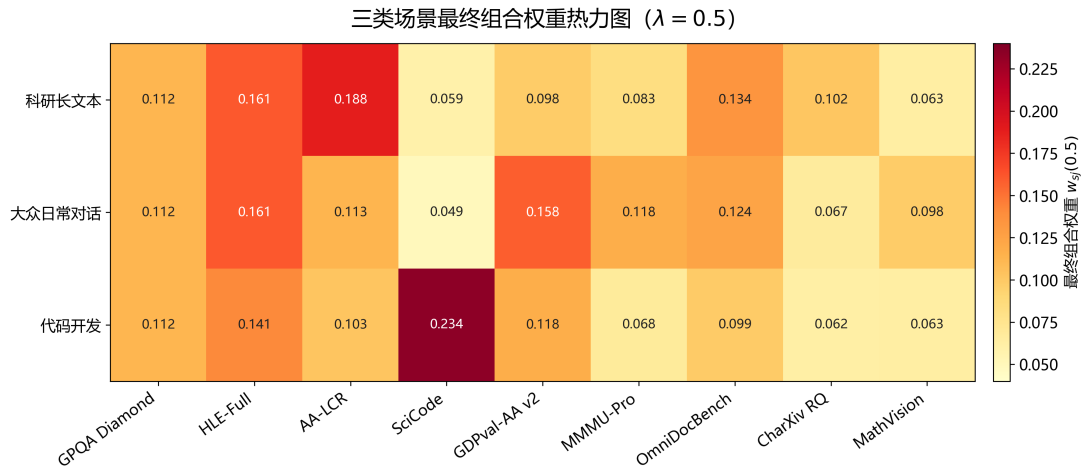
图 6 三类场景最终组合权重热力图 ($\lambda = 0.5$)

表 10 三类场景完整排名与得分

场景	排名	模型	场景得分	相对通用排名
科研长文本	1	Kimi K3	0.6547	上升 1 位
	2	Claude Fable 5	0.6471	下降 1 位
	3	GPT-5.6 Sol	0.5703	不变
	4	GPT-5.5	0.5276	不变
	5	Claude Opus 4.8	0.3147	不变
大众日常对话	1	Claude Fable 5	0.7208	不变
	2	Kimi K3	0.6343	不变
	3	GPT-5.6 Sol	0.6024	不变
	4	GPT-5.5	0.4392	不变
	5	Claude Opus 4.8	0.3431	不变
代码开发	1	Claude Fable 5	0.7722	不变
	2	Kimi K3	0.6840	不变
	3	GPT-5.6 Sol	0.5838	不变
	4	GPT-5.5	0.4896	不变
	5	Claude Opus 4.8	0.3384	不变

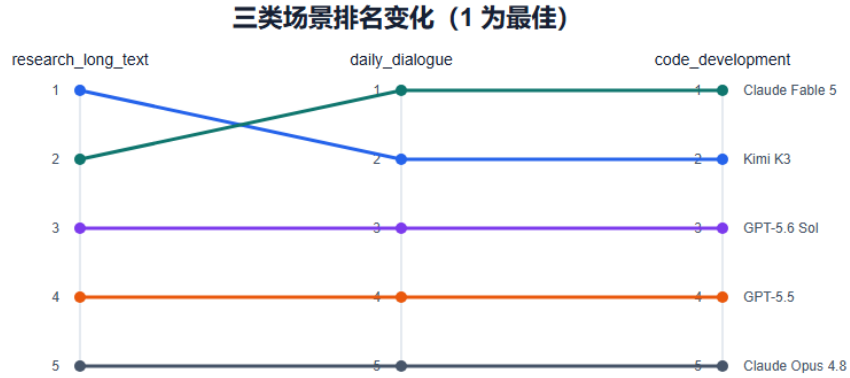


图 7 三类场景排名变化

在 $\lambda = 0.5$ 的基准组合下，科研长文本场景提高了 AA-LCR、CharXiv RQ 和 GPQA Diamond 等指标的权重作用，使 Kimi K3 从通用第 2 升至第 1。日常对话与代码开发中 Claude Fable 5 保持第一，Kimi K3 保持第二。组合系数敏感性分析进一步表明，科研长文本榜首在离散测试网格的 $\lambda = 0.5$ 与 0.6 之间发生 Kimi K3 与 Claude Fable 5 的切换，故该榜首结论应理解为基准设定下的条件性结果。GLM-5.2 仍因核心矩阵不完整而不进入场景 TOPSIS 排名。

5.2 跨场景排名差异的内在机理

模型 i 的九维标准化能力画像 $\mathbf{z}_i = (z_{i1}, \dots, z_{in})$ 不随场景改变；变化的是场景权重向量 $\mathbf{w}_s$ 。对同一能力画像进行 $v_{isj} = w_{sj}z_{ij}$ 加权后，模型在加权标准化空间中的位置随场景需求而改变，进而改变其到正、负理想解的距离 D_i^+ 、 D_i^- 以及 TOPSIS 贴近度。因此，跨场景排名差异不是模型基础能力突然变化，而是同一能力画像在不同需求权重下被重

新评价。下文提及的指标作用只描述能力差异与权重结构的方向性匹配，不将其称为单项 TOPSIS 得分贡献。

科研长文本场景的核心是两种能力结构的强对冲。相对 Claude Fable 5, Kimi K3 在 AA-LCR、GPQA Diamond 和 OmniDocBench 上的标准化优势分别为 0.6714、0.2903 和 0.2453; Fable 则在 HLE-Full、CharXiv RQ 和 GDPval-AA v2 上分别领先 0.8235、0.4881 和 0.2383。基准场景相对通用 CRITIC 权重重点提高 AA-LCR，并降低 HLE-Full 与 GDPval-AA v2 的权重作用，使 Kimi 的长文本和科学推理优势得到更充分体现，但未消除 Fable 在极高难推理和科研图文理解上的优势。在 $\lambda = 0.5$ 时，Kimi 的 $D^+ = 0.148272$ 略大于 Fable 的 0.146343，即 Kimi 并非更接近正理想解；但其 $D^- = 0.281092$ 大于 Fable 的 0.268320，整体更远离负理想解。两种距离共同作用后，Kimi 以 0.654670 对 0.647080 小幅领先，差距仅为 0.007590。

当科研场景的 λ 由 0.5 增至 0.6 时，CRITIC 客观权重作用增强而定向场景偏好减弱。Kimi 最主要的 AA-LCR 优势权重下降，同时 HLE-Full 权重回升，进一步体现 Fable 的相对优势；CharXiv RQ 权重下降与 OmniDocBench 权重上升对 Kimi 形成一定反向补偿，但不足以抵消前两项变化。对应的距离结构中，Kimi 的 D^+ 由 0.148272 增至 0.153590， D^- 由 0.281092 降至 0.275137；Fable 的 D^+ 由 0.146343 降至 0.139215， D^- 由 0.268320 增至 0.273219。于是 Kimi 的贴适度降至 0.641754，Fable 升至 0.662454，与离散网格上 [0.5, 0.6] 内出现的榜首切换一致。

大众日常对话场景主要提高 GDPval-AA v2、MMM-U-Pro 和 MathVision 的权重。Fable 在 GDPval-AA v2、HLE-Full 和 CharXiv RQ 上的优势，与 Kimi 在 AA-LCR、GPQA Diamond 和 OmniDocBench 上的优势相对抗；基准权重下 Fable 的 $D^+ = 0.111706$ 小于 Kimi 的 0.149683，且 $D^- = 0.288426$ 大于 Kimi 的 0.259591，故其同时更接近正理想解并更远离负理想解，以 0.720827 对 0.634273 保持明显领先。Kimi 的长文本、推理和文档能力仍支持其稳定处于第 2，而不能据此概括为日常能力较弱。

代码开发场景最显著的调整是 SciCode 权重相对通用权重提高 0.166317。Fable 的 SciCode 标准化值为 1，Kimi 为 0.8454，这一增权强化了 Fable 在科学编程上的相对优势；Fable 在 HLE-Full 和 GDPval-AA v2 上也占优。Kimi 虽在 AA-LCR、GPQA Diamond 和 OmniDocBench 上保持优势，但这些指标在代码场景中的权重作用不足以抵消前述差距。基准权重下，Fable 的 $D^+ = 0.096432$ 小于 Kimi 的 0.132460， $D^- = 0.326939$ 大于 Kimi 的 0.286748，贴适度因而为 0.772229，高于 Kimi 的 0.684023。该结果说明当前指标体系中 SciCode 增权与 Fable 稳定排名第一相一致，而不能解释为 Kimi 不适合代码开发。

6 问题三：性能-成本与部署效率

6.1 货币成本和 Pareto 前沿

定义标准工作负载为 100 万输入 token 和 20 万输出 token。模型 i 的货币调用成本为

$$M_i = \frac{N_{\text{in}}}{10^6} p_{i,\text{in}} + \frac{N_{\text{out}}}{10^6} p_{i,\text{out}}. \quad (9)$$

预算约束直接比较美元成本 M_i ，不与无量纲成本指数混用。根据当前截面官方标准价 [8]，Claude Fable 5 的输入与输出单价分别为 10 美元/百万 token 和 50 美元/百万 token，对应标准工作负载成本为 20 美元。

表 11 标准工作负载下的性能–成本结果

模型	成本/美元	通用能力核心 得分	Pareto	状态
Kimi K3	6.0000	0.5933	是	纳入
GPT-5.6 Sol	11.0000	0.5178	否	纳入
Claude Fable 5	20.0000	0.7254	是	纳入
Claude Opus 4.8	10.0000	0.3902	否	纳入
GPT-5.5	6.0000	0.4410	否	纳入
GLM-5.2	1.9180	NA	—	无通用能力核心 得分

Kimi K3 与 Claude Fable 5 构成 Pareto 前沿。预算为 6、10、12、16 美元时，在可行且具有完整通用能力排名的模型中均推荐 Kimi K3；预算达到 20 美元时推荐 Claude Fable 5。GLM-5.2 虽有较低的调用成本，但没有完整通用能力核心得分，不能进入 Pareto 或预算最优比较。

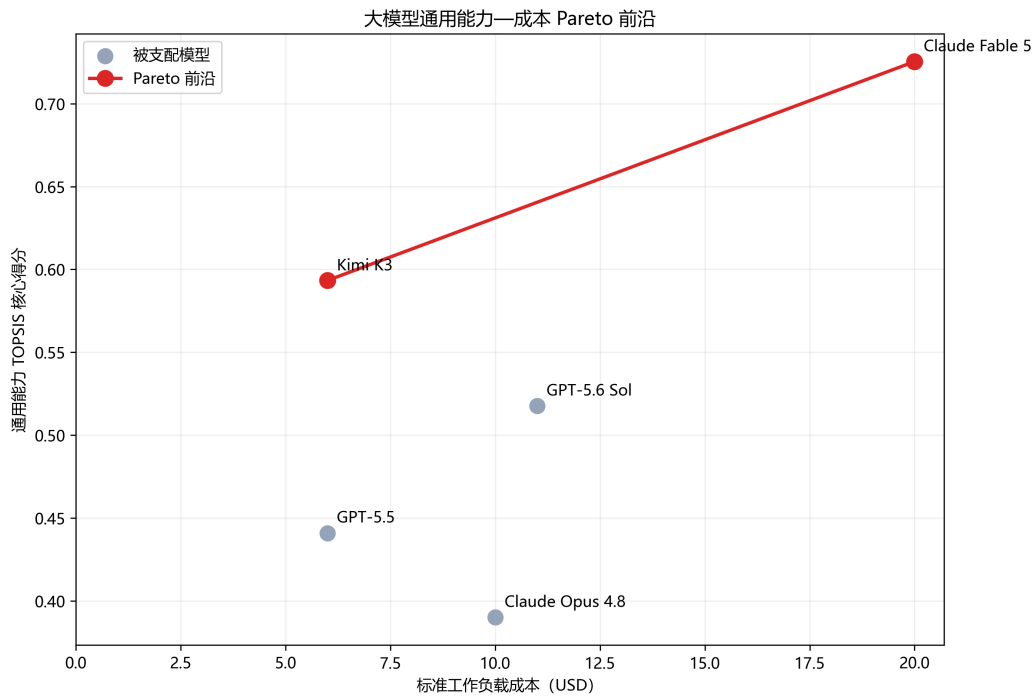


图 8 性能–成本 Pareto 前沿

表 12 预算约束下的模型推荐

预算/美元	可行排名模型数	推荐模型	推荐模型成本/美元
6	2	Kimi K3	6
10	3	Kimi K3	6
12	4	Kimi K3	6
16	4	Kimi K3	6
20	5	Claude Fable 5	20

6.2 算力能耗数据边界

赛题要求考察算力消耗，因此本文对公开能耗数据进行了专门可得性审计。严格横向比较需要统一硬件平台、模型精度、batch size、输入与输出长度、reasoning configuration 及 provider 部署条件；但当前公开资料不含满足这些条件的统一模型级 J/token 或 kWh/任务观测，部分闭源 API 亦未披露底层 GPU 与量化精度。因此，本文不用参数量、GPU TDP、推理时延或 API 价格人为推算单次能耗，而将公开 API 货币成本与严格可比的工程效率指标分层分析。算力能耗很重要，但当前只作为无法统一量化的数据边界，不参与正式排名。

为明确能耗在落地决策中的理论位置，定义部署成本框架

$$C_i^{\text{deploy}} = M_i + c_e E_i, \quad (10)$$

其中， M_i 为标准 token 工作负载下的 API 货币成本， E_i 为同一硬件、精度、batch、输入输出长度、reasoning configuration 和 provider 条件下的推理能耗， c_e 为单位能耗折算成本。当前 M_i 有公开可比观测，而 E_i 对候选模型缺乏统一公开观测，故正式实证仅对 M_i 定量， E_i 保留在理论框架中但不代入伪造数值。本文也不以 GPU TDP×时延、参数量×时间、FLOPs、TTFT、E2E 或 API 价格充当 E_i 的代理，因为这些量不等价于实际电力能耗，且闭源 API 的 GPU 型号、数量、量化精度、batch 与并行策略无法统一确认。因此，该框架属于受公开数据约束的分层部署成本模型。

6.3 严格可比部署效率子分析

Artificial Analysis 效率数据 [2] 的检索日期为 2026 年 8 月 16 日，测量窗口为过去 72 小时滚动窗口，统计量为 P50，故没有唯一的单日测量日期，measurement_date=NA。TTFT 表示 Time to First Token；Output Speed 单位为 tokens/s；E2E 表示 Artificial Analysis 标准化 500-token 端到端响应时间，不是任意长度回答的总响应时间。

最终 6 个模型中，只有 Kimi K3、GPT-5.6 Sol、Claude Opus 4.8 和 GPT-5.5 同时满足版本、配置、provider 与 workload 一致性要求。Claude Fable 5 的页面配置包含 Opus fallback；GLM-5.2 为 Together AI 第三方部署且量化/精度披露不足，二者均为 compatible=false，原始观测只作补充记录，不参与严格部署效率排名。

表 13 严格可比效率数据及部署得分

模型	Output Speed			工程效率	部署得分
	TTFT/s	tokens/s	E2E/s		
Kimi K3	3.4631	39.4660	66.8087	0.5695	0.5862
GPT-5.6 Sol	142.4287	73.7378	149.2095	0.2283	0.4309
Claude Opus 4.8	24.7382	61.0633	32.9264	0.7595	0.5010
GPT-5.5	48.4270	89.5060	54.0132	0.8317	0.5582

工程效率由 TTFT 成本型正向化、输出速度效益型正向化和 E2E 成本型正向化后等权平均；部署得分由 70% 通用能力核心得分与 30% 工程效率组成。该结论只覆盖表 13 中的 4 个模型，不能扩展为 6 模型效率排名。

图 9 展示严格可比的 4 个模型的三项效率构成、工程效率与部署得分。

工程效率与部署得分（严格可比的 4 个模型）

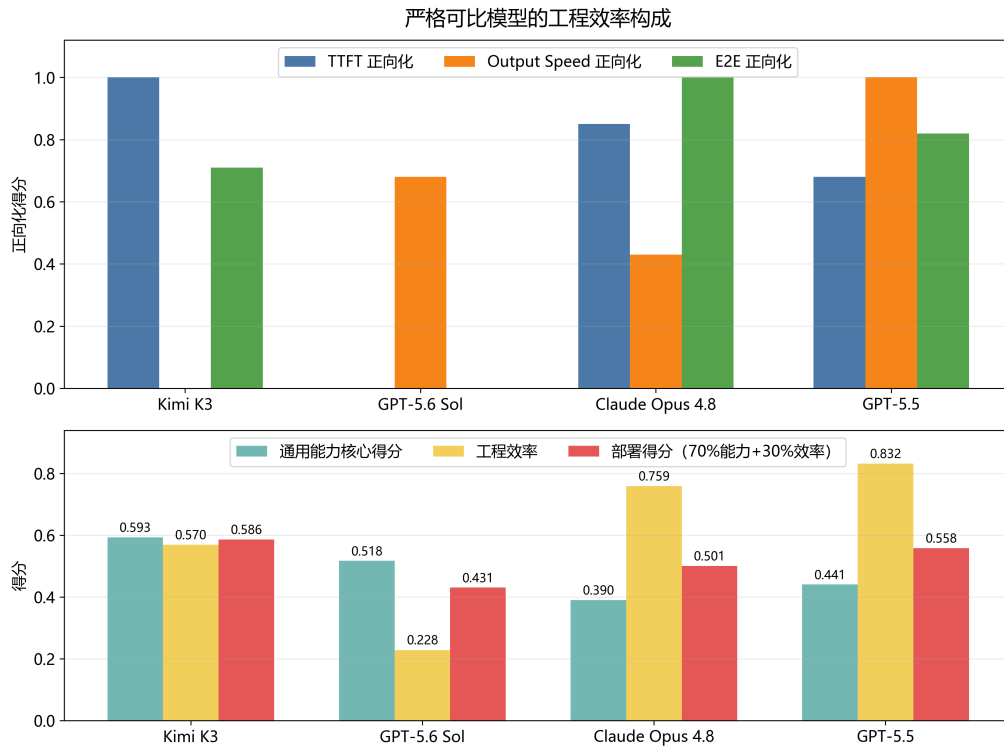


图 9 严格可比模型的工程效率与部署得分

6.4 成本-性能探索性关系

对 5 个具有完整通用能力排名的模型拟合，其中 Performance 表示问题一的通用能力核心得分：

$$\text{Performance} = 0.199408 + 0.148104 \ln(\text{Cost}), \quad (11)$$

得到 $R^2 = 0.314672$ 、调整 $R^2 = 0.086230$ 、 $\text{RMSE} = 0.097773$ 。由于 $n = 5$ 且解释力有限，式 (11) 只描述当前样本内的探索性弱正相关，不构成成本与性能之间的普遍定律或因果证据。

五个完整模型的观测点与对数拟合曲线如图 10 所示。

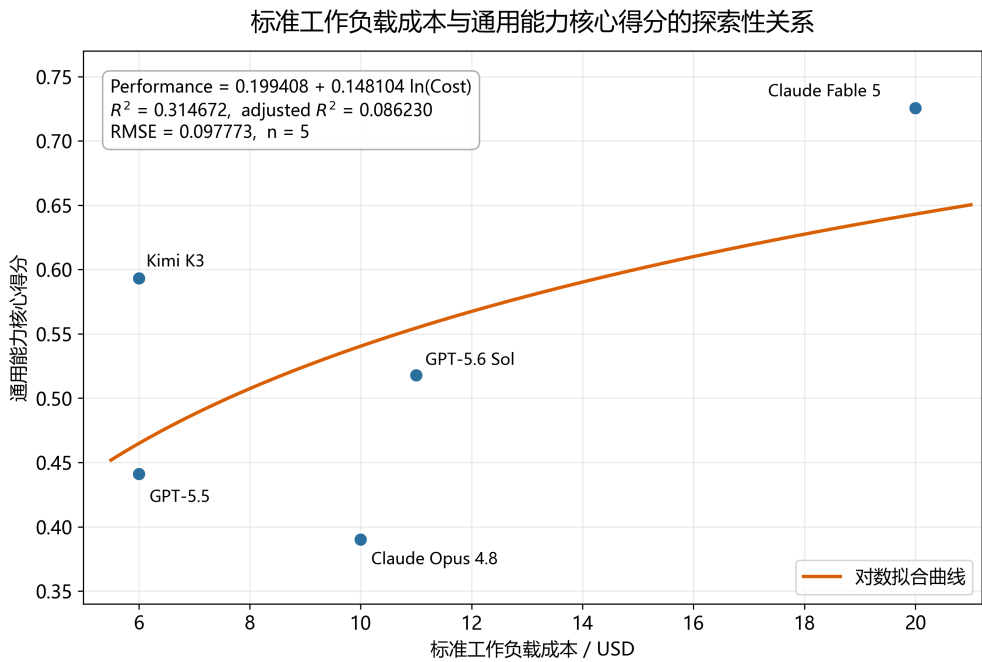


图 10 标准工作负载成本与通用能力核心得分的探索性关系

7 稳健性分析

7.1 方法与参数稳健性

方法稳健性方面，以熵权法替代 CRITIC 计算权重，5 个完整模型的排名与 CRITIC-TOPSIS 完全一致。参数稳健性方面，对 9 个 CRITIC 权重分别乘以 0.8、0.9、1.1 和 1.2，每次重新归一化并重算 TOPSIS，共形成 36 个确定性单因素扰动场景。

所有扰动中 Claude Fable 5、Kimi K3 与 GPT-5.6 Sol 始终保持第 1、2、3 名。只有 HLE-Full 权重乘以 1.2 时，Claude Opus 4.8 与 GPT-5.5 交换第 4、5 名；扰动排名相对基准的最低 Kendall τ 为 0.8。因此，前三结论在当前局部扰动范围内稳定，但这 36 次检验不能替代全局不确定性分析。

36 组单因素权重扰动下的 Kendall τ 如图 11 所示。

表 14 权重扰动下的排名稳定性

模型	基准排名	最低排名	最高排名	平均排名	第一频率
Claude Fable 5	1	1	1	1.0000	1.0000
Kimi K3	2	2	2	2.0000	0
GPT-5.6 Sol	3	3	3	3.0000	0
GPT-5.5	4	4	5	4.0278	0
Claude Opus 4.8	5	4	5	4.9722	0

7.2 样本集合敏感性检验

正式 CRITIC 对各指标使用其可得的 5 或 6 个模型计算总体标准差，并以逐对完整观测计算 Pearson 相关；主 TOPSIS 则只对 9 项指标均完整的 5 个模型排序。为检验缺失模式及样本集合对客观权重的影响，进一步排除 GLM-5.2 的部分观测，以与主 TOPSIS 完

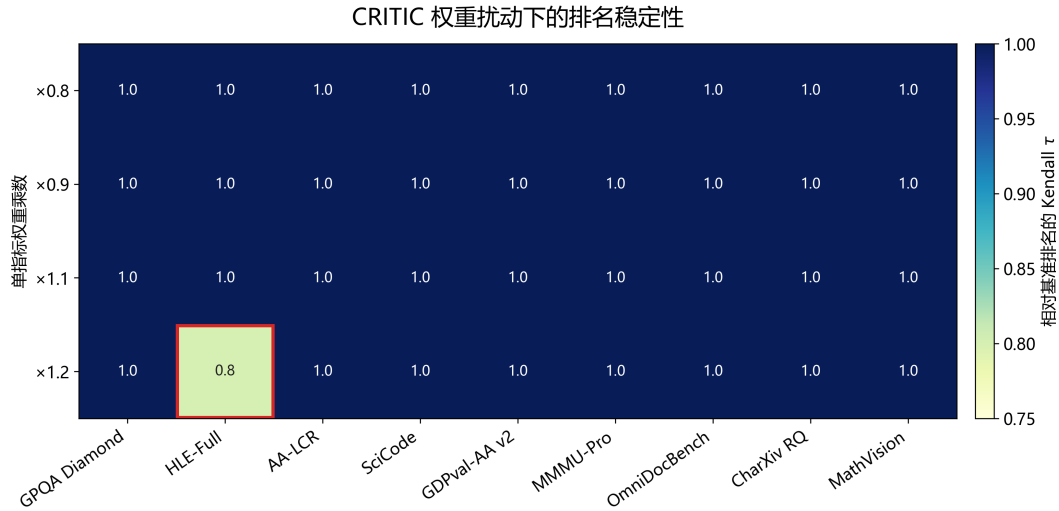


图 11 CRITIC 权重扰动下的排名稳定性

全相同的 Claude Fable 5、Kimi K3、GPT-5.6 Sol、GPT-5.5 和 Claude Opus 4.8 构成 5×9 完整矩阵。在标准化方法、总体标准差、Pearson 相关、CRITIC 冲突度和 TOPSIS 算法均保持不变的条件下，仅将 CRITIC 的计算样本固定为这 5 个模型并重新进行对照计算。

对照权重相对正式权重的最大绝对变化为 0.00905，平均绝对变化为 0.00364；HLE-Full、OmniDocBench 和 AA-LCR 仍依次为权重前三，只有低权重指标的内部次序发生小幅变化。5 个模型的完整综合排名保持不变，正式排名与对照排名的 Spearman 系数和 Kendall τ 均为 1。Kimi K3 在两种口径下均排名第 2，与第 1 名的 TOPSIS 分差由 0.1320 缩小至 0.1210。该结果表明，主排名对当前样本缺失处理具有较强的局部稳健性，但该对照只作为样本敏感性证据，不替换正式 CRITIC 权重或正式 TOPSIS 结果。

7.3 场景组合系数敏感性

为检验客观信息与场景偏好组合比例对问题二结论的影响，在不改变 CRITIC 权重、人工偏好向量及 TOPSIS 算法的条件下，取 $\lambda = 0.3, 0.4, 0.5, 0.6, 0.7$ 进行合理邻域敏感性分析。这里 λ 越大表示 CRITIC 客观权重影响增强、场景人工偏好影响减弱；该离散网格不代表全部可能偏好，也不用于证明 $\lambda = 0.5$ 最优。

科研长文本场景中，Kimi K3 相对 Claude Fable 5 的得分差随 λ 增大由 0.063106 持续缩小，在 $\lambda = 0.5$ 时为 0.007590，在 $\lambda = 0.6$ 时变为 -0.020700 。因此，在所检验的离散网格上，榜首切换区间只能确定为 $[0.5, 0.6]$ ，不能据此给出精确临界值。Kimi K3 的基准第一具有条件性，其优势部分来自科研场景对长文本与科研文档能力的定向加权；但前三模型集合始终不变，GPT-5.6 Sol、GPT-5.5 与 Claude Opus 4.8 分别始终处于第 3、4、5 位，最低 Spearman 系数为 0.9、最低 Kendall τ 为 0.8，说明整体模型梯队仍具有较高稳定性。

大众日常对话和代码开发场景在 $\lambda = 0.3-0.7$ 内完整排名均保持不变，最低 Spearman 与 Kendall τ 均为 1。Claude Fable 5 对 Kimi K3 的领先差在日常对话中由 0.068175 扩大至 0.105107，在代码开发中由 0.071038 扩大至 0.106507。三类场景的汇总结论见表 15。

表 15 场景组合系数的局部敏感性汇总

场景	λ 范围	榜首稳定性	前三集合	最低 Spearman	最低 Kendall	结论
科研长文本	0.3–0.7	Kimi/Fable 切换	稳定	0.9	0.8	对 λ 中度敏感
大众日常对话	0.3–0.7	Fable 始终第一	稳定	1.0	1.0	高度稳定
代码开发	0.3–0.7	Fable 始终第一	稳定	1.0	1.0	高度稳定

8 模型评价与局限

8.1 方法特点

本文从原始来源、精确版本和评测设置开始建立可追溯链路，缺失值处理保守；CRITIC 权重、TOPSIS 排名、场景权重、货币成本和稳健性结果均可由冻结脚本复算。能力、货币成本和效率采用分层分析，避免把价格或时延在通用能力核心得分中重复计权。

8.2 局限性

- (1) 核心 Benchmark cohort 高度集中于厂商技术报告的统一对照数据，尽管设置统一且可追溯，仍可能存在厂商报告偏差。
- (2) 部分新模型接近截止日才发布，导致严格可比分项覆盖不足；最终池筛选不能解释为单纯追求覆盖率。
- (3) 相关性、CRITIC 和回归只基于 5–6 个模型，统计量容易随样本池改变，不宜外推。0.85 相关阈值仅是冗余复核规则，相关分析不进行总体显著性推断；尽管完整样本对照未改变排名，这仍只是当前小样本模型池内的稳健性证据，不能外推至任意模型集合。
- (4) 严格部署效率只覆盖 6 个最终模型中的 4 个；Fable 与 GLM 的不兼容观测不能用于横向效率高低判断。
- (5) Artificial Analysis 指标是过去 72 小时滚动 P50，只代表 2026 年 8 月 16 日检索时段附近的 provider 表现。
- (6) API 成本只对应 100 万输入加 20 万输出 token 的固定 workload 与截面标准价，未完整建模缓存、Batch、长上下文阶梯价、区域加价和峰谷套餐。
- (7) 算力能耗缺乏统一硬件、精度、批量和推理设置下的公开可比观测，故未进行人工估算或能耗排名。
- (8) 三类场景的人工偏好为题意驱动设定，尚未经过真实专家问卷或团队成对比较校准。
- (9) 当前核心能力层未将知识事实准确性单列为独立评价指标；受版本、测试设置及公开数据同口径可比性约束，本文未将异源事实性 Benchmark 强行并入冻结核心

矩阵。后续若获得同版本、同设置且覆盖充分的公开事实性基准，可进一步补充该维度。

- (10) 36 次扰动是局部单因素敏感性分析，不能代替全局不确定性传播或大规模随机模拟。

8.3 改进方向

未来可建立滚动数据更新与版本、价格时间戳管理机制，并在覆盖充分后扩展模型池及知识事实准确性指标；使用真实专家调查、Delphi 法或 AHP 校准场景偏好；在统一 GPU、精度、batch 和推理设置下实测 J/token 等能耗；进一步将静态单模型选型扩展为预算、时延与质量多约束下的动态模型路由。

9 结论与建议

在不插补缺失数据的前提下，Claude Fable 5、Kimi K3 与 GPT-5.6 Sol 获得通用能力综合评价前三。本文的综合评价采用分层框架：通用能力由九项 Benchmark 形成核心得分，工程效率、货币成本和能耗边界在落地决策层进一步分析。Kimi K3 的竞争力集中在长文本、文档理解和科研材料处理，HLE-Full 所代表的极高难开放推理是其主要短板。在等比例组合的基准设定下，科研长文本场景 Kimi K3 排名第一；但组合系数敏感性表明，其与 Claude Fable 5 的榜首位置具有条件性，科研场景选型应结合决策者对客观通用能力与场景专门需求的侧重程度。日常对话与代码开发场景下 Claude Fable 5 保持第一。跨场景差异本质上来自模型能力画像与场景权重结构的匹配程度不同，并通过加权空间中的正、负理想解距离反映在 TOPSIS 排名中。

在当前标准 API 工作负载下，Kimi K3 与 Claude Fable 5 构成性能–成本 Pareto 前沿。在本文考察的 6、10、12 和 16 美元预算点下均推荐 Kimi K3；预算达到 20 美元且追求最高通用能力时推荐 Claude Fable 5。部署效率结论只适用于 4 个严格 compatible 模型。理论部署成本模型包含能耗项，但由于统一能耗观测不可得，本文不对其进行伪精确估算。熵权对照和权重扰动支持前三名的局部稳定性，但所有结论均应限定在数据截面、固定 cohort 和小样本范围内。

10 AI 工具使用声明

本参赛队在竞赛过程中使用了 ChatGPT 与 Codex 辅助代码调试、数据核验、论文结构检查、格式规范核验、可视化检查及语言表述优化 [13, 14]。核心建模方案、指标筛选原则、正式数据选择、数学计算、结果解释和最终结论均由参赛队员审阅并确认。具体 AI 使用环节、关键交互及人工修改情况见支撑材料《人工智能工具使用详情》；论文结构检查和部分语言表述使用了 AI 辅助优化。

参考文献

- [1] Moonshot AI. Kimi K3 Technical Report[EB/OL]. <https://github.com/MoonshotAI/Kimi-K3>, 2026-07-16.
- [2] Artificial Analysis. AI Model and API Provider Performance Benchmarking[EB/OL]. <https://artificialanalysis.ai/>, accessed 2026-08-16.

- [3] Moonshot AI. Kimi K3 Official Blog[EB/OL]. <https://www.kimi.com/blog/kimi-k3>, 2026-07-16, accessed 2026-08-16.
- [4] OpenAI. GPT-5.6 Launch Report[EB/OL]. <https://openai.com/index/gpt-5-6/>, accessed 2026-08-16.
- [5] OpenAI. API Model Comparison[EB/OL]. <https://developers.openai.com/api/docs/models/compare/>, accessed 2026-08-16.
- [6] OpenAI. API Pricing[EB/OL]. <https://platform.openai.com/docs/pricing/>, accessed 2026-08-16.
- [7] Anthropic. Claude Models Overview[EB/OL]. <https://platform.claude.com/docs/zh-CN/about-claude/models>, accessed 2026-08-16.
- [8] Anthropic. Claude API Pricing[EB/OL]. <https://platform.claude.com/docs/en/about-claude/pricing>, accessed 2026-08-17.
- [9] Z.ai. GLM-5.2 Official Launch Report[EB/OL]. <https://z.ai/blog/glm-5.2>, 2026-06-16, accessed 2026-08-17.
- [10] Zhipu AI. GLM API Pricing[EB/OL]. <https://open.bigmodel.cn/pricing>, accessed 2026-08-16.
- [11] Hwang C L, Yoon K. Multiple Attribute Decision Making: Methods and Applications[M]. Berlin: Springer-Verlag, 1981.
- [12] Diakoulaki D, Mavrotas G, Papayannakis L. Determining objective weights in multiple criteria problems: The CRITIC method[J]. Computers & Operations Research, 1995, 22(7): 763–770. DOI: 10.1016/0305-0548(94)00059-H.
- [13] ChatGPT, GPT-5.6 Sol, OpenAI, 2026-08-16–2026-08-19.
- [14] Codex, GPT-5.6 Sol, OpenAI, 2026-08-16–2026-08-19.

A 支撑材料文件清单

序号	相对路径	文件说明
1	AI_CHATGPT_RECORDS_NEED_USER.txt	待用户补充的真实 ChatGPT 交互说明
2	data/merged/.gitkeep	模型合并数据
3	data/merged/model_dataset.csv	模型合并数据
4	data/model_candidates.csv	模型池或数据配置
5	data/processed/.gitkeep	模型使用的处理后数据
6	data/processed/core_benchmark_long.csv	模型使用的处理后数据
7	data/processed/core_benchmark_matrix.csv	模型使用的处理后数据
8	data/processed/indicator_quality.csv	模型使用的处理后数据
9	data/processed/model_attributes.csv	模型使用的处理后数据
10	data/processed/phase2_quality_report.csv	模型使用的处理后数据
11	data/raw/benchmark_scores.csv	冻结原始数据
12	data/raw/cost_efficiency.csv	冻结原始数据
13	data/raw/model_metadata.csv	冻结原始数据
14	data/sources/benchmark_sources.csv	公开数据来源索引
15	data/sources/efficiency_sources.csv	公开数据来源索引
16	data/sources/metadata_sources.csv	公开数据来源索引
17	README.txt	支撑材料结构、运行环境与复现说明
18	results/.gitkeep	论文模型结果或图表
19	results/benchmark_coverage.csv	论文模型结果或图表
20	results/core_indicator_selection.csv	论文模型结果或图表
21	results/efficiency_configuration_review.csv	论文模型结果或图表
22	results/efficiency_staging.csv	论文模型结果或图表
23	results/final_model_summary.csv	论文模型结果或图表
24	results/phase3/descriptive_statistics.csv	论文模型结果或图表
25	results/phase3/indicator_screening.csv	论文模型结果或图表
26	results/phase3/pearson_correlation.csv	论文模型结果或图表
27	results/phase3/redundancy_flags.csv	论文模型结果或图表
28	results/phase3/spearman_correlation.csv	论文模型结果或图表
29	results/phase4/critic_weights.csv	论文模型结果或图表
30	results/phase4/general_ranking.csv	论文模型结果或图表
31	results/phase4/kimi_strengths_weaknesses.csv	论文模型结果或图表
32	results/phase4/normalized_scores.csv	论文模型结果或图表
33	results/phase4b/glm_official_hle_note.csv	论文模型结果或图表
34	results/phase4b/glm_partial_comparison.csv	论文模型结果或图表
35	results/phase4b/glm_partial_comparison.png	论文模型结果或图表
36	results/phase5/scenario_rankings.csv	论文模型结果或图表
37	results/phase5/scenario_weights.csv	论文模型结果或图表
38	results/phase6/budget_recommendations.csv	论文模型结果或图表
39	results/phase6/engineering_efficiency.csv	论文模型结果或图表
40	results/phase6/performance_cost.csv	论文模型结果或图表

序号	相对路径	文件说明
41	results/phase7/cost_performance_regression.csv	论文模型结果或图表
42	results/phase7/method_comparison.csv	论文模型结果或图表
43	results/phase7/rank_correlation.csv	论文模型结果或图表
44	results/phase7/rank_stability.csv	论文模型结果或图表
45	results/phase7/weight_method_comparison.csv	论文模型结果或图表
46	results/phase7/weight_perturbation_rankings.csv	论文模型结果或图表
47	results/q1/critic_weights.csv	论文模型结果或图表
48	results/q1/critic_weights_bar.png	论文模型结果或图表
49	results/q1/data_quality_report.csv	论文模型结果或图表
50	results/q1/final_indicator_system.csv	论文模型结果或图表
51	results/q1/high_correlation_pairs.csv	论文模型结果或图表
52	results/q1/kimi_k3_gap.png	论文模型结果或图表
53	results/q1/kimi_k3_metric_analysis.csv	论文模型结果或图表
54	results/q1/kimi_k3_radar.png	论文模型结果或图表
55	results/q1/normalized_data.csv	论文模型结果或图表
56	results/q1/pearson_correlation.csv	论文模型结果或图表
57	results/q1/pearson_heatmap.png	论文模型结果或图表
58	results/q1/rank_stability.csv	论文模型结果或图表
59	results/q1/removed_metrics.csv	论文模型结果或图表
60	results/q1/selected_metrics.csv	论文模型结果或图表
61	results/q1/sensitivity_analysis.csv	论文模型结果或图表
62	results/q1/spearman_correlation.csv	论文模型结果或图表
63	results/q1/spearman_heatmap.png	论文模型结果或图表
64	results/q1/topsis_ranking.csv	论文模型结果或图表
65	results/q1/topsis_ranking_bar.png	论文模型结果或图表
66	results/q2/coding_ranking.csv	论文模型结果或图表
67	results/q2/dialogue_ranking.csv	论文模型结果或图表
68	results/q2/kimi_k3_scenario_analysis.csv	论文模型结果或图表
69	results/q2/kimi_k3_scenario_radar.png	论文模型结果或图表
70	results/q2/rank_change.png	论文模型结果或图表
71	results/q2/rank_change_analysis.csv	论文模型结果或图表
72	results/q2/research_ranking.csv	论文模型结果或图表
73	results/q2/scenario_rank_comparison.png	论文模型结果或图表
74	results/q2/scenario_rankings.csv	论文模型结果或图表
75	results/q2/scenario_score_heatmap.png	论文模型结果或图表
76	results/q2/scenario_sensitivity.csv	论文模型结果或图表
77	results/q2/scenario_stability.csv	论文模型结果或图表
78	results/q2/scenario_weights.csv	论文模型结果或图表
79	results/q2/scenario_weights_heatmap.png	论文模型结果或图表
80	results/q3/budget_selection.csv	论文模型结果或图表
81	results/q3/budget_selection.png	论文模型结果或图表
82	results/q3/cost_data_audit.csv	论文模型结果或图表
83	results/q3/deploy_scores.csv	论文模型结果或图表
84	results/q3/engineering_efficiency.png	论文模型结果或图表

序号	相对路径	文件说明
85	results/q3/kimi_k3_cost_analysis.csv	论文模型结果或图表
86	results/q3/pareto_analysis.csv	论文模型结果或图表
87	results/q3/performance_cost.csv	论文模型结果或图表
88	results/q3/performance_cost_fit.png	论文模型结果或图表
89	results/q3/performance_cost_pareto.png	论文模型结果或图表
90	results/q3/performance_cost_regression.csv	论文模型结果或图表
91	results/q3/sensitivity_analysis.csv	论文模型结果或图表
92	source/figures/critic_weights.svg	模型计算、验证或可视化源文件
93	source/figures/general_ranking.svg	模型计算、验证或可视化源文件
94	source/figures/performance_cost_pareto.svg	模型计算、验证或可视化源文件
95	source/figures/scenario_rank_changes.svg	模型计算、验证或可视化源文件
96	source/requirements.txt	模型计算、验证或可视化源文件
97	source/run_all.ps1	模型计算、验证或可视化源文件
98	source/scripts/analyze_phase3_indicators.py	模型计算、验证或可视化源文件
99	source/scripts/analyze_phase7_robustness.py	模型计算、验证或可视化源文件
100	source/scripts/artificial_analysis_targets.json	模型计算、验证或可视化源文件
101	source/scripts/build_member_b_data.py	模型计算、验证或可视化源文件
102	source/scripts/build_paper.py	模型计算、验证或可视化源文件
103	source/scripts/check_coverage.py	模型计算、验证或可视化源文件
104	source/scripts/fetch_artificial_analysis_efficiency.py	模型计算、验证或可视化源文件
105	source/scripts/generate_figures.py	模型计算、验证或可视化源文件
106	source/scripts/merge_data.py	模型计算、验证或可视化源文件
107	source/scripts/model_phase4_critic_topsis.py	模型计算、验证或可视化源文件
108	source/scripts/model_phase5_scenarios.py	模型计算、验证或可视化源文件
109	source/scripts/model_phase6_cost_benefit.py	模型计算、验证或可视化源文件
110	source/scripts/phase2_core_cohorts.csv	模型计算、验证或可视化源文件

序号	相对路径	文件说明
111	source/scripts/phase5_scenario_priorities.csv	模型计算、验证或可视化源文件
112	source/scripts/process_phase2_data.py	模型计算、验证或可视化源文件
113	source/scripts/run_glm_supplement.py	模型计算、验证或可视化源文件
114	source/scripts/run_pipeline.py	模型计算、验证或可视化源文件
115	source/scripts/run_q1.py	模型计算、验证或可视化源文件
116	source/scripts/run_q2.py	模型计算、验证或可视化源文件
117	source/scripts/run_q3.py	模型计算、验证或可视化源文件
118	source/scripts/validate_data.py	模型计算、验证或可视化源文件
119	source/scripts/validate_modeling.py	模型计算、验证或可视化源文件
120	source/scripts/validate_paper.py	模型计算、验证或可视化源文件
121	source/scripts/validate_q1.py	模型计算、验证或可视化源文件
122	source/scripts/validate_q2_q3.py	模型计算、验证或可视化源文件
123	source/src/q1/__init__.py	模型计算、验证或可视化源文件
124	source/src/q1/analysis.py	模型计算、验证或可视化源文件
125	source/src/q1/glm_supplement.py	模型计算、验证或可视化源文件
126	source/src/q1/reporting.py	模型计算、验证或可视化源文件
127	source/src/q1/visualization.py	模型计算、验证或可视化源文件
128	source/src/q2/__init__.py	模型计算、验证或可视化源文件
129	source/src/q2/analysis.py	模型计算、验证或可视化源文件
130	source/src/q2/reporting.py	模型计算、验证或可视化源文件
131	source/src/q2/visualization.py	模型计算、验证或可视化源文件
132	source/src/q3/__init__.py	模型计算、验证或可视化源文件

序号	相对路径	文件说明
133	source/src/q3/analysis.py	模型计算、验证或可视化源文件
134	source/src/q3/reporting.py	模型计算、验证或可视化源文件
135	source/src/q3/visualization.py	模型计算、验证或可视化源文件
136	人工智能工具使用详情.pdf	AI 工具使用详情正式 PDF
137	人工智能工具使用详情.tex	AI 工具使用详情可编辑源文件

B 完整源程序代码

本研究使用了程序代码，以下按支撑材料中的相对路径列出完整、可复制的源程序。运行入口与环境说明见附录 A 所列 README.txt。

source/run_all.ps1

```

1 $ErrorActionPreference = 'Stop'
2 $sourceRoot = (Resolve-Path -LiteralPath $PSScriptRoot).Path
3 $supportRoot = (Resolve-Path -LiteralPath (Join-Path $sourceRoot '..')).Path
4 $stagedData = Join-Path $sourceRoot 'data'
5 $stagedResults = Join-Path $sourceRoot 'results'
6 $finalResults = Join-Path $supportRoot 'results'
7
8 try {
9     if (Test-Path -LiteralPath $stagedData) { Remove-Item -LiteralPath $stagedData -Recurse -Force }
10    if (Test-Path -LiteralPath $stagedResults) { Remove-Item -LiteralPath $stagedResults -Recurse -Force }
11    Copy-Item -LiteralPath (Join-Path $supportRoot 'data') -Destination $stagedData -Recurse
12    Copy-Item -LiteralPath $finalResults -Destination $stagedResults -Recurse
13
14    Push-Location -LiteralPath $sourceRoot
15    python scripts/run_pipeline.py
16    if ($LASTEXITCODE -ne 0) { throw "run_pipeline.py failed with exit code $LASTEXITCODE" }
17    python scripts/validate_modeling.py
18    if ($LASTEXITCODE -ne 0) { throw "validate_modeling.py failed with exit code $LASTEXITCODE" }
19    Pop-Location
20
21    Copy-Item -LiteralPath (Join-Path $stagedResults '*') -Destination $finalResults -Recurse -Force
22    Write-Host 'Reproducibility pipeline completed successfully.' -ForegroundColor Green
23 }
24 finally {
25     if ((Get-Location).Path -eq $sourceRoot) { Pop-Location }
26     if (Test-Path -LiteralPath $stagedData) { Remove-Item -LiteralPath $stagedData -Recurse -Force }
27     if (Test-Path -LiteralPath $stagedResults) { Remove-Item -LiteralPath $stagedResults -Recurse -Force }
28 }

```

source/scripts/analyze_phase3_indicators.py

```

1 """Analyze Phase 3 indicator distributions, correlations, and redundancy."""
2
3 from __future__ import annotations
4
5 import argparse
6 import csv
7 import math

```

```

8 from pathlib import Path
9
10 ROOT = Path(__file__).resolve().parents[1]
11 INPUT = ROOT / "data/processed/core_benchmark_long.csv"
12 DEFAULT_OUTPUT = ROOT / "results/phase3"
13 NA = "NA"
14
15
16 def read_rows(path: Path) -> list[dict[str, str]]:
17     with path.open(encoding="utf-8-sig", newline="") as handle:
18         return list(csv.DictReader(handle))
19
20
21 def write_rows(path: Path, fields: list[str], rows: list[dict[str, str]]) -> None:
22     path.parent.mkdir(parents=True, exist_ok=True)
23     with path.open("w", encoding="utf-8", newline="") as handle:
24         writer = csv.DictWriter(handle, fieldnames=fields)
25         writer.writeheader()
26         writer.writerows(rows)
27
28
29 def mean(values: list[float]) -> float:
30     return sum(values) / len(values)
31
32
33 def sample_std(values: list[float]) -> float:
34     if len(values) < 2:
35         return 0.0
36     center = mean(values)
37     return math.sqrt(sum((value - center) ** 2 for value in values) / (len(values) - 1))
38
39
40 def pearson(left: list[float], right: list[float]) -> float | None:
41     if len(left) < 3 or len(left) != len(right):
42         return None
43     left_mean, right_mean = mean(left), mean(right)
44     numerator = sum((x - left_mean) * (y - right_mean) for x, y in zip(left, right))
45     denominator = math.sqrt(
46         sum((x - left_mean) ** 2 for x in left) * sum((y - right_mean) ** 2 for y in right)
47     )
48     return numerator / denominator if denominator else None
49
50
51 def ranks(values: list[float]) -> list[float]:
52     ordered = sorted(enumerate(values), key=lambda item: item[1])
53     result = [0.0] * len(values)
54     index = 0
55     while index < len(ordered):
56         end = index + 1
57         while end < len(ordered) and ordered[end][1] == ordered[index][1]:
58             end += 1
59         average_rank = (index + 1 + end) / 2
60         for position in range(index, end):

```

```

61         result[ordered[position][0]] = average_rank
62     index = end
63     return result
64
65
66 def paired(values: dict[str, dict[str, float]], left: str, right: str) -> tuple[list[float], list[float]]:
67     model_ids = sorted(set(values[left]) & set(values[right]))
68     return [values[left][key] for key in model_ids], [values[right][key] for key in model_ids]
69
70
71 def fmt(value: float | None) -> str:
72     return NA if value is None else f"{value:.6f}"
73
74
75 def analyze(output_dir: Path) -> dict[str, int]:
76     rows = read_rows(INPUT)
77     indicators = list(dict.fromkeys(row["indicator_key"] for row in rows))
78     dimensions = {row["indicator_key"]: row["dimension"] for row in rows}
79     names = {row["indicator_key"]: row["indicator"] for row in rows}
80     values: dict[str, dict[str, float]] = {indicator: {} for indicator in indicators}
81     total_models = len({row["model_id"] for row in rows})
82     for row in rows:
83         if row["score"] != NA:
84             values[row["indicator_key"]][row["model_id"]] = float(row["score"])
85
86     descriptive = []
87     for indicator in indicators:
88         current = list(values[indicator].values())
89         current_mean = mean(current)
90         std = sample_std(current)
91         descriptive.append({
92             "indicator_key": indicator,
93             "dimension": dimensions[indicator],
94             "indicator": names[indicator],
95             "available_models": str(len(current)),
96             "missing_models": str(total_models - len(current)),
97             "mean": f"{current_mean:.6f}",
98             "sample_std": f"{std:.6f}",
99             "coefficient_of_variation": fmt(std / abs(current_mean) if current_mean else
None),
100             "minimum": f"{min(current):.6f}",
101             "maximum": f"{max(current):.6f}",
102             "range": f"{max(current) - min(current):.6f}",
103         })
104     write_rows(
105         output_dir / "descriptive_statistics.csv",
106         list(descriptive[0]),
107         descriptive,
108     )
109
110     correlation_fields = ["indicator_key"] + indicators
111     pearson_rows, spearman_rows = [], []

```

```

112     redundancy = []
113     flagged: set[str] = set()
114     for left in indicators:
115         pearson_row = {"indicator_key": left}
116         spearman_row = {"indicator_key": left}
117         for right in indicators:
118             x, y = paired(values, left, right)
119             p = pearson(x, y)
120             s = pearson(ranks(x), ranks(y)) if len(x) >= 3 else None
121             pearson_row[right] = fmt(p)
122             spearman_row[right] = fmt(s)
123             if indicators.index(left) < indicators.index(right) and s is not None and abs(s
124 ) >= 0.9:
125                 same_dimension = dimensions[left] == dimensions[right]
126                 redundancy.append({
127                     "indicator_a": left,
128                     "indicator_b": right,
129                     "pairwise_models": str(len(x)),
130                     "pearson": fmt(p),
131                     "spearman": fmt(s),
132                     "same_dimension": str(same_dimension).lower(),
133                     "decision": "review_not_auto_drop",
134                     "reason": "Only 5-6 models; correlation is unstable and indicators
represent distinct constructs" if not same_dimension else "Only 5-6 models; retain
pending robustness analysis",
135                 })
136                 flagged.update((left, right))
137                 pearson_rows.append(pearson_row)
138                 spearman_rows.append(spearman_row)
139             write_rows(output_dir / "pearson_correlation.csv", correlation_fields, pearson_rows)
140             write_rows(output_dir / "spearman_correlation.csv", correlation_fields, spearman_rows)
141             redundancy_fields = [
142                 "indicator_a", "indicator_b", "pairwise_models", "pearson", "spearman",
143                 "same_dimension", "decision", "reason",
144             ]
145             write_rows(output_dir / "redundancy_flags.csv", redundancy_fields, redundancy)
146
147     screening = []
148     for item in descriptive:
149         indicator = item["indicator_key"]
150         coverage = int(item["available_models"]) / total_models * 100
151         screening.append({
152             "indicator_key": indicator,
153             "dimension": dimensions[indicator],
154             "indicator": names[indicator],
155             "coverage_percent": f"{coverage:.1f}",
156             "variation_nonzero": str(float(item["sample_std"]) > 0).lower(),
157             "high_correlation_flag": str(indicator in flagged).lower(),
158             "phase3_decision": "retain",
159             "reason": "Coverage >=75%, nonzero variation, and distinct substantive
dimension; high correlations are not used for automatic deletion with n=6",
160         })
161     write_rows(
        output_dir / "indicator_screening.csv",

```

```

162     [
163         "indicator_key", "dimension", "indicator", "coverage_percent", "
variation_nonzero",
164         "high_correlation_flag", "phase3_decision", "reason",
165     ],
166     screening,
167 )
168 return {"indicators": len(indicators), "redundancy_flags": len(redundancy), "retained":
len(screening)}
169
170
171 def main() -> int:
172     parser = argparse.ArgumentParser(description=__doc__)
173     parser.add_argument("--output-dir", type=Path, default=DEFAULT_OUTPUT)
174     args = parser.parse_args()
175     try:
176         result = analyze(args.output_dir)
177     except (KeyError, ValueError) as exc:
178         print(f"ERROR: {exc}")
179         return 1
180     print(
181         f"Phase 3 analysis complete: {result['indicators']} indicators, "
182         f"{result['redundancy_flags']} high-correlation pair(s), {result['retained']}
retained."
183     )
184     return 0
185
186
187 if __name__ == "__main__":
188     raise SystemExit(main())

```

source/scripts/analyze_phase7_robustness.py

```

1  """Run entropy-weight comparison, weight sensitivity, and exploratory regression."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  import math
8  from collections import defaultdict
9  from pathlib import Path
10
11  try:
12      from scripts.model_phase5_scenarios import topsis
13  except ModuleNotFoundError:
14      from model_phase5_scenarios import topsis
15
16  ROOT = Path(__file__).resolve().parents[1]
17  DEFAULT_OUTPUT = ROOT / "results/phase7"
18  FACTORS = (0.8, 0.9, 1.1, 1.2)
19
20

```

```

21 def read_rows(path: Path) -> list[dict[str, str]]:
22     with path.open(encoding="utf-8-sig", newline="") as handle:
23         return list(csv.DictReader(handle))
24
25
26 def write_rows(path: Path, fields: list[str], rows: list[dict[str, str]]) -> None:
27     path.parent.mkdir(parents=True, exist_ok=True)
28     with path.open("w", encoding="utf-8", newline="") as handle:
29         writer = csv.DictWriter(handle, fieldnames=fields)
30         writer.writeheader()
31         writer.writerows(rows)
32
33
34 def ordered_ranks(scores: dict[str, float]) -> tuple[list[str], dict[str, int]]:
35     order = sorted(scores, key=lambda model: (-scores[model], model))
36     return order, {model: rank for rank, model in enumerate(order, start=1)}
37
38
39 def kendall_tau(rank_a: dict[str, int], rank_b: dict[str, int]) -> float:
40     models = sorted(rank_a)
41     concordant = discordant = 0
42     for i, left in enumerate(models):
43         for right in models[i + 1:]:
44             product = (rank_a[left] - rank_a[right]) * (rank_b[left] - rank_b[right])
45             concordant += product > 0
46             discordant += product < 0
47     return (concordant - discordant) / (concordant + discordant)
48
49
50 def entropy_weights(models: list[str], indicators: list[str], normalized: dict[tuple[str,
51     str], float]) -> dict[str, float]:
52     diversification = {}
53     scale = 1 / math.log(len(models))
54     for indicator in indicators:
55         values = [normalized[model, indicator] for model in models]
56         total = sum(values)
57         probabilities = [value / total for value in values] if total else [1 / len(models)]
58         * len(models)
59         entropy = -scale * sum(probability * math.log(probability) for probability in
60     probabilities if probability > 0)
61     diversification[indicator] = 1 - entropy
62     total = sum(diversification.values())
63     return {indicator: diversification[indicator] / total for indicator in indicators}
64
65
66 def linear_regression(x: list[float], y: list[float]) -> tuple[float, float, float]:
67     x_mean, y_mean = sum(x) / len(x), sum(y) / len(y)
68     denominator = sum((value - x_mean) ** 2 for value in x)
69     slope = sum((a - x_mean) * (b - y_mean) for a, b in zip(x, y)) / denominator
70     intercept = y_mean - slope * x_mean
71     fitted = [intercept + slope * value for value in x]
72     residual = sum((actual - predicted) ** 2 for actual, predicted in zip(y, fitted))
73     total = sum((actual - y_mean) ** 2 for actual in y)
74     return intercept, slope, 1 - residual / total if total else 0.0

```

```

72
73
74 def analyze(output_dir: Path) -> dict[str, object]:
75     normalized_rows = read_rows(ROOT / "results/phase4/normalized_scores.csv")
76     critic_rows = read_rows(ROOT / "results/phase4/critic_weights.csv")
77     ranking_rows = read_rows(ROOT / "results/phase4/general_ranking.csv")
78     cost_rows = read_rows(ROOT / "results/phase6/performance_cost.csv")
79     indicators = [row["indicator_key"] for row in critic_rows]
80     critic = {row["indicator_key"]: float(row["critic_weight"]) for row in critic_rows}
81     models = [row["model_id"] for row in ranking_rows if row["ranking_status"] == "
ranked_complete_case"]
82     names = {row["model_id"]: row["model_name"] for row in ranking_rows}
83     normalized = {
84         (row["model_id"], row["indicator_key"]): float(row["minmax_score"])
85         for row in normalized_rows if row["model_id"] in models and row["minmax_score"] !=
"NA"
86     }
87     entropy = entropy_weights(models, indicators, normalized)
88     critic_scores = topsis(models, indicators, normalized, critic)
89     entropy_scores = topsis(models, indicators, normalized, entropy)
90     critic_order, critic_rank = ordered_ranks(critic_scores)
91     entropy_order, entropy_rank = ordered_ranks(entropy_scores)
92     method_rows = []
93     for model in critic_order:
94         method_rows.append({
95             "model_id": model, "model_name": names[model],
96             "critic_score": f"{critic_scores[model]:.10f}", "critic_rank": str(critic_rank[
model]),
97             "entropy_score": f"{entropy_scores[model]:.10f}", "entropy_rank": str(
entropy_rank[model]),
98             "rank_difference": str(entropy_rank[model] - critic_rank[model]),
99         })
100     write_rows(
101         output_dir / "method_comparison.csv",
102         ["model_id", "model_name", "critic_score", "critic_rank", "entropy_score", "
entropy_rank", "rank_difference"],
103         method_rows,
104     )
105     weight_rows = [
106         {"indicator_key": key, "critic_weight": f"{critic[key]:.10f}", "entropy_weight": f
"{entropy[key]:.10f}", "difference": f"{entropy[key] - critic[key]:.10f}"}
107         for key in indicators
108     ]
109     write_rows(output_dir / "weight_method_comparison.csv", ["indicator_key", "
critic_weight", "entropy_weight", "difference"], weight_rows)
110
111     perturb_rows = []
112     rank_history: dict[str, list[int]] = defaultdict(list)
113     top_counts: dict[str, int] = defaultdict(int)
114     tau_rows = []
115     scenario_count = 0
116     for indicator in indicators:
117         for factor in FACTORS:
118             adjusted = dict(critic)

```

```

119         adjusted[indicator] *= factor
120         total = sum(adjusted.values())
121         adjusted = {key: value / total for key, value in adjusted.items()}
122         scores = topsis(models, indicators, normalized, adjusted)
123         order, ranks = ordered_ranks(scores)
124         scenario = f"{indicator}_x{factor:.1f}"
125         scenario_count += 1
126         top_counts[order[0]] += 1
127         tau_rows.append({"scenario": scenario, "kendall_tau_vs_baseline": f"{
kendall_tau(critic_rank, ranks):.10f}", "winner": order[0]})
128         for model in order:
129             rank_history[model].append(ranks[model])
130             perturb_rows.append({
131                 "scenario": scenario, "perturbed_indicator": indicator, "factor": f"{
factor:.1f}",
132                 "model_id": model, "score": f"{scores[model]:.10f}", "rank": str(ranks[
model]),
133                 "rank_change_vs_baseline": str(critic_rank[model] - ranks[model]),
134             })
135         write_rows(
136             output_dir / "weight_perturbation_rankings.csv",
137             ["scenario", "perturbed_indicator", "factor", "model_id", "score", "rank", "
rank_change_vs_baseline"],
138             perturb_rows,
139         )
140         write_rows(output_dir / "rank_correlation.csv", ["scenario", "kendall_tau_vs_baseline",
"winner"], tau_rows)
141         stability_rows = []
142         for model in critic_order:
143             history = rank_history[model]
144             stability_rows.append({
145                 "model_id": model, "model_name": names[model], "baseline_rank": str(critic_rank
[model]),
146                 "minimum_rank": str(min(history)), "maximum_rank": str(max(history)),
147                 "mean_rank": f"{sum(history) / len(history):.4f}",
148                 "top1_frequency": f"{top_counts[model] / scenario_count:.4f}",
149             })
150         write_rows(
151             output_dir / "rank_stability.csv",
152             ["model_id", "model_name", "baseline_rank", "minimum_rank", "maximum_rank", "
mean_rank", "top1_frequency"],
153             stability_rows,
154         )
155
156         included_cost = [row for row in cost_rows if row["analysis_status"] == "included"]
157         x = [math.log(float(row["standardized_api_cost_usd"])) for row in included_cost]
158         y = [float(row["general_performance_score"]) for row in included_cost]
159         intercept, slope, r_squared = linear_regression(x, y)
160         regression_rows = [{
161             "model": "performance = intercept + slope * ln(cost)",
162             "n": str(len(x)), "intercept": f"{intercept:.10f}", "slope_log_cost": f"{slope:.10f
}]",
163             "r_squared": f"{r_squared:.10f}", "interpretation": "exploratory only; n=5 is too
small for causal inference",

```

```

164     }]
165     write_rows(output_dir / "cost_performance_regression.csv", list(regression_rows[0]),
166                   regression_rows)
167     return {
168         "entropy_winner": entropy_order[0],
169         "scenarios": scenario_count,
170         "minimum_tau": min(float(row["kendall_tau_vs_baseline"]) for row in tau_rows),
171         "regression_r_squared": r_squared,
172     }
173
174 def main() -> int:
175     parser = argparse.ArgumentParser(description=__doc__)
176     parser.add_argument("--output-dir", type=Path, default=DEFAULT_OUTPUT)
177     args = parser.parse_args()
178     try:
179         result = analyze(args.output_dir)
180     except (KeyError, ValueError) as exc:
181         print(f"ERROR: {exc}")
182         return 1
183     print(
184         f"Phase 7 complete: {result['scenarios']} perturbations, entropy winner={result['entropy_winner']}, "
185         f"minimum Kendall tau={result['minimum_tau']:.3f}."
186     )
187     return 0
188
189 if __name__ == "__main__":
190     raise SystemExit(main())
191

```

source/scripts/build_member_b_data.py

```

1  """Build Member B raw metadata and source-evidence tables.
2
3  This script encodes *verified, source-backed* data gathered from official
4  model/pricing pages and reputable launch coverage, retrieved on 2026-08-16
5  (DATA_CUTOFF_DATE = 2026-08-17). It writes:
6
7      data/raw/model_metadata.csv      -- one row per candidate model
8      data/sources/metadata_sources.csv -- per-field evidence (one row per field)
9
10 Every numeric value is either a real number or the literal "NA" (never
11 imputed). Each metadata field has a corresponding evidence row in
12 metadata_sources.csv. The script is the single source of truth so the CSVs
13 stay reproducible and auditable.
14
15 ALIGNED WITH MEMBER A CANDIDATE POOL (9 models):
16     The model_id, model_name, provider, exact_version and release_date in this
17     table now match Member A's data-benchmark branch exactly, so that all three
18     raw tables (benchmark_scores, model_metadata, cost_efficiency) can be merged
19     by model_id without ID conflicts.
20

```

References (retrieved 2026-08-16):

Kimi K3 <https://www.kimi.com/blog/kimi-k3>
<https://www.kimi.com/help/kimi-api/api-troubleshooting>
 GPT-5.6 Sol <https://developers.openai.com/api/docs/models/compare/>
<https://developers.openai.com/api/docs/models/gpt-5.6-sol>
 Claude Fable 5 <https://platform.claude.com/docs/en/about-claude/pricing>
 Claude Opus 4.8 <https://platform.claude.com/docs/zh-CN/about-claude/models>
 GPT-5.5 <https://developers.openai.com/api/docs/models/compare/>
<https://platform.openai.com/docs/pricing/>
 GLM-5.2 <https://open.bigmodel.cn/pricing>
<https://docs.bigmodel.cn/cn/guide/models/text/glm-5.2>
<https://huggingface.co/zai-org/GLM-5.2>
 Gemini 3.1 Pro Preview <https://ai.google.dev/gemini-api/docs/pricing>
 DeepSeek V4 Pro 0813 https://api-docs.deepseek.com/quick_start/pricing
 Qwen3.8 2.4T A95B <https://qwen.ai/blog?id=qwen3.8>
 (overseas price via <https://www.globaltimes.cn/page/202608/1367420.shtml>)

"""

from __future__ import annotations

import csv

from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]

RETRIEVAL = "2026-08-16"

CNY -> USD conversion used only for Zhipu (official list price is in CNY).

CNY_USD = 7.10 # approx. spot rate on retrieval date; documented in notes.

Raw metadata: one row per candidate model (Member B scope).

All prices are USD / 1M tokens. NA = not disclosed / not verified.

model_id / model_name / provider / exact_version / release_date are aligned

with Member A's data-benchmark branch (9 candidates).

METADATA = [

{

 "model_id": "kimi-k3",

 "model_name": "Kimi K3",

 "provider": "Moonshot AI",

 "exact_version": "kimi-k3",

 "release_date": "2026-07-16",

 "context_window": 1048576,

 "max_output_tokens": 1048576,

 "vision_support": "yes",

 "reasoning_support": "yes",

 "api_available": "yes",

 "input_price_usd_per_million": 3.00,

 "output_price_usd_per_million": 15.00,

 "cached_input_price_usd_per_million": 0.30,

 "batch_input_price": "NA",

 "batch_output_price": "NA",

 "long_context_price": "NA",

```

74     "peak_price": "NA",
75     "off_peak_price": "NA",
76     "pricing_effective_date": "2026-07-16",
77     "source_url": "https://www.kimi.com/blog/kimi-k3",
78     "retrieval_date": "2026-08-17",
79     "notes": "Required by problem statement. 2.8T MoE (104B active); native vision (
text/image/video); always-on thinking (effort low/high/max, max default). Official Kimi
API docs allow max_completion_tokens up to 1,048,576; actual output is bounded by
1,048,576 minus prompt_tokens, and the default is 131,072. No batch discount found. 1M
context at standard rate (no separate long-context tier found).",
80 },
81 {
82     "model_id": "gpt-5.6-sol",
83     "model_name": "GPT-5.6 Sol",
84     "provider": "OpenAI",
85     "exact_version": "gpt-5.6-sol",
86     "release_date": "2026-07-09",
87     "context_window": 1050000,
88     "max_output_tokens": 128000,
89     "vision_support": "yes",
90     "reasoning_support": "yes",
91     "api_available": "yes",
92     "input_price_usd_per_million": 5.00,
93     "output_price_usd_per_million": 30.00,
94     "cached_input_price_usd_per_million": 0.50,
95     "batch_input_price": 2.50,
96     "batch_output_price": 15.00,
97     "long_context_price": 10.00,
98     "peak_price": "NA",
99     "off_peak_price": "NA",
100    "pricing_effective_date": "2026-07-09",
101    "source_url": "https://developers.openai.com/api/docs/models/gpt-5.6-sol",
102    "retrieval_date": "2026-08-17",
103    "notes": "Flagship frontier model. Standard rates are input $5 / cached input $0.50
/ output $30 per 1M. For prompts with more than 272K input tokens, OpenAI charges 2x
input and 1.5x output for the full request, i.e. $10 input and $45 output;
long_context_price records the input rate. Batch = 50% off standard. Regional (data-
residency) endpoints +10%. Release date 2026-07-09 aligned with Member A.",
104 },
105 {
106     "model_id": "claude-fable-5",
107     "model_name": "Claude Fable 5",
108     "provider": "Anthropic",
109     "exact_version": "claude-fable-5",
110     "release_date": "2026-06-09",
111     "context_window": 1000000,
112     "max_output_tokens": 128000,
113     "vision_support": "yes",
114     "reasoning_support": "yes",
115     "api_available": "yes",
116     "input_price_usd_per_million": 10.00,
117     "output_price_usd_per_million": 50.00,
118     "cached_input_price_usd_per_million": 1.00,
119     "batch_input_price": 5.00,

```

```

120     "batch_output_price": 25.00,
121     "long_context_price": "NA",
122     "peak_price": "NA",
123     "off_peak_price": "NA",
124     "pricing_effective_date": "2026-06-09",
125     "source_url": "https://platform.claude.com/docs/en/about-claude/pricing",
126     "retrieval_date": "2026-08-17",
127     "notes": "Highest Anthropic tier (above Opus 4.8). Adaptive thinking with effort
controls. AA labels Opus 4.8 Fallback safeguards that affect some evaluations; C audit
flags HOLD because fallback routing frequency is not disclosed and model identity may
be mixed. 1M context at standard pricing (no surcharge). Official current rates are
input $10 / cached input $1 / output $50 per 1M; Batch API rates are $5 / $25.",
128 },
129 {
130     "model_id": "claude-opus-4.8",
131     "model_name": "Claude Opus 4.8",
132     "provider": "Anthropic",
133     "exact_version": "claude-opus-4.8",
134     "release_date": "2026-05-28",
135     "context_window": 1000000,
136     "max_output_tokens": 128000,
137     "vision_support": "yes",
138     "reasoning_support": "yes",
139     "api_available": "yes",
140     "input_price_usd_per_million": 5.00,
141     "output_price_usd_per_million": 25.00,
142     "cached_input_price_usd_per_million": 0.50,
143     "batch_input_price": 2.50,
144     "batch_output_price": 12.50,
145     "long_context_price": "NA",
146     "peak_price": "NA",
147     "off_peak_price": "NA",
148     "pricing_effective_date": "2026-05-28",
149     "source_url": "https://platform.claude.com/docs/zh-CN/about-claude/models",
150     "retrieval_date": RETRIEVAL,
151     "notes": "Recommended default flagship. 1M context at standard pricing (no long-
context surcharge). Cached input = prompt-cache read = 10% of input per Anthropic docs
($0.50). Batch API = 50% off. Adaptive thinking with effort controls (Adaptive
Reasoning, Max Effort per AA).Release date 2026-05-28.",
152 },
153 {
154     "model_id": "gpt-5.5",
155     "model_name": "GPT-5.5",
156     "provider": "OpenAI",
157     "exact_version": "gpt-5.5",
158     "release_date": "2026-04-23",
159     "context_window": 256000,
160     "max_output_tokens": 128000,
161     "vision_support": "yes",
162     "reasoning_support": "yes",
163     "api_available": "yes",
164     "input_price_usd_per_million": 3.00,
165     "output_price_usd_per_million": 15.00,
166     "cached_input_price_usd_per_million": 0.30,

```

```

167     "batch_input_price": 1.50,
168     "batch_output_price": 7.50,
169     "long_context_price": "NA",
170     "peak_price": "NA",
171     "off_peak_price": "NA",
172     "pricing_effective_date": "2026-04-23",
173     "source_url": "https://developers.openai.com/api/docs/models/compare/",
174     "retrieval_date": RETRIEVAL,
175     "notes": "Mature OpenAI baseline model; AA marks deprecated but data was publicly
verifiable before cutoff (DATA_RULES: deprecated lifecycle does not invalidate pre-
cutoff evidence). Reasoning model with xhigh effort (per AA). Standard rates for
context <270K; no separate long-context tier documented for this tier. Batch = 50% off
standard. Regional endpoints +10%.",
176 },
177 {
178     "model_id": "glm-5.2",
179     "model_name": "GLM-5.2",
180     "provider": "Z.ai",
181     "exact_version": "GLM-5.2",
182     "release_date": "2026-06-16",
183     "context_window": 1048576,
184     "max_output_tokens": 131072,
185     "vision_support": "no",
186     "reasoning_support": "yes",
187     "api_available": "yes",
188     "input_price_usd_per_million": 1.13,
189     "output_price_usd_per_million": 3.94,
190     "cached_input_price_usd_per_million": 0.28,
191     "batch_input_price": "NA",
192     "batch_output_price": "NA",
193     "long_context_price": "NA",
194     "peak_price": "NA",
195     "off_peak_price": "NA",
196     "pricing_effective_date": "2026-06-16",
197     "source_url": "https://open.bigmodel.cn/pricing",
198     "retrieval_date": "2026-08-17",
199     "notes": "Provider aligned to Z.ai per Member A candidate pool (official platform:
bigmodel.cn / z.ai). Official China list price is CNY: input Y8 / output Y28 / cache-
hit Y2 per 1M. Converted to USD at 7.10 CNY/USD on 2026-08-16 -> $1.13 / $3.94 / $0.28
(conversion reproducible; raw CNY in notes). Official GLM-5.2 docs specify 128K maximum
output, stored exactly as 131,072 tokens; the third-party 262K claim is rejected. The
official Z.ai model card classifies GLM-5.2 as text generation, so vision_support=no.
Release date 2026-06-16 (aligned with Member A). 1M context at standard rate.",
200 },
201 {
202     "model_id": "gemini-3.1-pro-preview",
203     "model_name": "Gemini 3.1 Pro Preview",
204     "provider": "Google",
205     "exact_version": "gemini-3.1-pro-preview",
206     "release_date": "2026-02-19",
207     "context_window": 2000000,
208     "max_output_tokens": "NA",
209     "vision_support": "yes",
210     "reasoning_support": "yes",

```

```

211     "api_available": "yes",
212     "input_price_usd_per_million": 2.00,
213     "output_price_usd_per_million": 12.00,
214     "cached_input_price_usd_per_million": 0.20,
215     "batch_input_price": 1.00,
216     "batch_output_price": 6.00,
217     "long_context_price": 4.00,
218     "peak_price": "NA",
219     "off_peak_price": "NA",
220     "pricing_effective_date": "2026-02-19",
221     "source_url": "https://ai.google.dev/gemini-api/docs/pricing",
222     "retrieval_date": RETRIEVAL,
223     "notes": "Tiered by prompt length: <=200K input $2.00 / output $12.00 / cached $0
.20; >200K input $4.00 / output $18.00 / cached $0.40. long_context_price records the
>200K input price; output is $18.00 (see notes). Batch = 50% off. Context window 2M per
model card / secondary sources; official pricing doc lists >200K tier but does not
print the window number. Max output not stated. Release date 2026-02-19 (aligned with
Member A). AA separately reports AI Studio and Vertex; C audit recommends freezing AI
Studio as the provider.",
224 },
225 {
226     "model_id": "deepseek-v4-pro-0813",
227     "model_name": "DeepSeek V4 Pro 0813",
228     "provider": "DeepSeek",
229     "exact_version": "DeepSeek V4 Pro 0813",
230     "release_date": "2026-08-13",
231     "context_window": 1000000,
232     "max_output_tokens": 384000,
233     "vision_support": "no",
234     "reasoning_support": "yes",
235     "api_available": "yes",
236     "input_price_usd_per_million": 0.435,
237     "output_price_usd_per_million": 0.87,
238     "cached_input_price_usd_per_million": 0.003625,
239     "batch_input_price": "NA",
240     "batch_output_price": "NA",
241     "long_context_price": "NA",
242     "peak_price": 0.44,
243     "off_peak_price": 0.22,
244     "pricing_effective_date": "2026-08-13",
245     "source_url": "https://api-docs.deepseek.com/quick_start/pricing",
246     "retrieval_date": RETRIEVAL,
247     "notes": "Thinking mode default. No multimodal (text-only). Prices are cache-miss (
standard) rates; cached input (cache hit) = $0.003625. Peak/off-peak billing took
effect 2026-08-16 16:00 UTC: PEAK input $0.44 / output $1.32; OFF-PEAK input $0.22 /
output $0.66. peak_price/off_peak_price record the cache-miss INPUT prices; outputs
noted here. No explicit batch discount found in retrieved docs. Release inferred from
version stamp 0813 -> 2026-08-13 (aligned with Member A).",
248 },
249 {
250     "model_id": "qwen3.8-2.4t-a95b",
251     "model_name": "Qwen3.8 2.4T A95B",
252     "provider": "Alibaba",
253     "exact_version": "Qwen3.8 2.4T A95B",

```

```

254     "release_date": "2026-08-12",
255     "context_window": 1000000,
256     "max_output_tokens": 65536,
257     "vision_support": "yes",
258     "reasoning_support": "yes",
259     "api_available": "yes",
260     "input_price_usd_per_million": 2.00,
261     "output_price_usd_per_million": 6.00,
262     "cached_input_price_usd_per_million": 0.25,
263     "batch_input_price": "NA",
264     "batch_output_price": "NA",
265     "long_context_price": "NA",
266     "peak_price": "NA",
267     "off_peak_price": "NA",
268     "pricing_effective_date": "2026-08-12",
269     "source_url": "https://qwen.ai/blog?id=qwen3.8",
270     "retrieval_date": RETRIEVAL,
271     "notes": "2.4T MoE (95B active); 1M context; vision (text/image) and thinking mode
(reasoning_effort xhigh/medium/low). Overseas API price $2/$6, implicit cache-hit $0.25
(domestic CNY price differs: 12/36 yuan, cache-hit 1.5 yuan). Official blog does not
list price; overseas price sourced from launch coverage(Global Times). No batch
discount found. 1M context at standard rate. Release date 2026-08-12 (aligned with
Member A).",
272 },
273 ]
274
275 # -----
276 # Per-field evidence. Each entry: (model_id, field, value, source_name,
277 # source_url, publication_date, retrieval_date, notes)
278 # -----
279 S = RETRIEVAL
280 SOURCES = [
281     # ---- kimi-k3 ----
282     ("kimi-k3", "provider, exact_version, release_date", "Moonshot AI; kimi-k3;
2026-07-16",
283     "Kimi K3 official blog", "https://www.kimi.com/blog/kimi-k3", "2026-07-16", S,
284     "Launch date 2026-07-16; weights released 2026-07-27. exact_version slug kimi-k3
aligned with Member A."),
285     ("kimi-k3", "context_window", "1048576",
286     "Kimi K3 official blog", "https://www.kimi.com/blog/kimi-k3", "2026-07-16", S,
287     "1,048,576-token context window per Hugging Face model card cited in blog."),
288     ("kimi-k3", "max_output_tokens", "1048576 (context-bound; default 131072)",
289     "Kimi API troubleshooting", "https://www.kimi.com/help/kimi-api/api-troubleshooting",
"2026-08-17", "2026-08-17",
290     "Official docs: max_completion_tokens defaults to 131072 and can be set up to 1048576;
actual output maximum equals 1048576 minus prompt_tokens."),
291     ("kimi-k3", "vision_support", "yes",
292     "Kimi K3 official blog", "https://www.kimi.com/blog/kimi-k3", "2026-07-16", S,
293     "Native multimodal: text, image, video input."),
294     ("kimi-k3", "reasoning_support", "yes",
295     "Kimi K3 official blog", "https://www.kimi.com/blog/kimi-k3", "2026-07-16", S,
296     "Always-on thinking; effort low/high/max (max default)."),
297     ("kimi-k3", "api_available", "yes",
298     "Kimi API Platform", "https://platform.kimi.ai/", "2026-07-16", S,

```

```

299     "Available via Kimi API, OpenRouter, GitHub Copilot (Fireworks), Perplexity."),
300 ("kimi-k3", "input/output/cached input price", "3.00 / 15.00 / 0.30 USD per 1M",
301 "Kimi K3 official blog", "https://www.kimi.com/blog/kimi-k3", "2026-07-16", S,
302 "Standard API: input $3, cached input $0.30, output $15 per 1M tokens."),
303 ("kimi-k3", "batch / long-context / peak-offpeak price", "NA",
304 "Kimi K3 official blog", "https://www.kimi.com/blog/kimi-k3", "2026-07-16", S,
305 "No batch discount, no separate long-context tier, no peak/off-peak found at retrieval
    ."),
306
307 # ---- gpt-5.6-sol ----
308 ("gpt-5.6-sol", "provider, exact_version, release_date", "OpenAI; gpt-5.6-sol;
    2026-07-09",
309 "OpenAI model compare", "https://developers.openai.com/api/docs/models/compare/",
    "2026-07-09", S,
310 "exact_version slug gpt-5.6-sol and release date 2026-07-09 aligned with Member A."),
311 ("gpt-5.6-sol", "context_window", "1050000",
312 "OpenAI model compare", "https://developers.openai.com/api/docs/models/compare/",
    "2026-08-16", S,
313 "1,050,000-token context window."),
314 ("gpt-5.6-sol", "max_output_tokens", "128000",
315 "OpenAI model compare", "https://developers.openai.com/api/docs/models/compare/",
    "2026-08-16", S,
316 "128,000 max output tokens."),
317 ("gpt-5.6-sol", "vision_support", "yes",
318 "OpenAI model compare", "https://developers.openai.com/api/docs/models/compare/",
    "2026-08-16", S,
319 "Image input supported."),
320 ("gpt-5.6-sol", "reasoning_support", "yes",
321 "OpenAI model compare", "https://developers.openai.com/api/docs/models/compare/",
    "2026-08-16", S,
322 "Reasoning model; AA labels max effort."),
323 ("gpt-5.6-sol", "api_available", "yes",
324 "OpenAI API", "https://platform.openai.com/docs/pricing/", "2026-08-16", S,
325 "Generally available via OpenAI API."),
326 ("gpt-5.6-sol", "input/output/cached input price", "5.00 / 30.00 / 0.50 USD per 1M",
327 "OpenAI API pricing", "https://platform.openai.com/docs/pricing/", "2026-08-16", S,
328 "Standard rates up to 272K input tokens: input $5, cached $0.50, output $30."),
329 ("gpt-5.6-sol", "batch price", "input 2.50 / output 15.00 USD per 1M",
330 "OpenAI API pricing", "https://platform.openai.com/docs/pricing/", "2026-08-16", S,
331 "Batch API = 50% off standard."),
332 ("gpt-5.6-sol", "long-context / peak-offpeak price", "input 10.00 / output 45.00 USD
    per 1M; peak/offpeak NA",
333 "OpenAI GPT-5.6 Sol model page", "https://developers.openai.com/api/docs/models/gpt
    -5.6-sol", "2026-08-17", "2026-08-17",
334 "Prompts with >272K input tokens are billed at 2x input and 1.5x output for the full
    request; no peak/off-peak price. long_context_price stores the $10 input rate."),
335
336 # ---- claude-fable-5 ----
337 ("claude-fable-5", "provider, exact_version, release_date", "Anthropic; claude-fable-5;
    2026-06-09",
338 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
    models", "2026-06-09", S,
339 "Highest Anthropic tier; released 2026-06-09. exact_version slug claude-fable-5
    aligned with Member A."),

```

```

340 ("claude-fable-5", "context_window", "1000000",
341 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
342 "1,000,000-token context window (same as Opus 4.8, Anthropic frontier standard)."),
343 ("claude-fable-5", "max_output_tokens", "128000",
344 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
345 "128K max output tokens."),
346 ("claude-fable-5", "vision_support", "yes",
347 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
348 "Vision capable."),
349 ("claude-fable-5", "reasoning_support", "yes",
350 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
351 "Adaptive thinking with effort controls; AA labels Adaptive Reasoning, Max Effort."),
352 ("claude-fable-5", "api_available", "yes",
353 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
354 "GA on Claude API, Bedrock, Vertex, Foundry."),
355 ("claude-fable-5", "input/output/cached input price", "10.00 / 50.00 / 1.00 USD per 1M
",
356 "Anthropic pricing docs", "https://platform.claude.com/docs/en/about-claude/pricing",
"2026-08-17", "2026-08-17",
357 "Official current pricing: input $10, output $50, cache hits and refreshes $1 per 1M
."),
358 ("claude-fable-5", "batch price", "input 5.00 / output 25.00 USD per 1M",
359 "Anthropic pricing docs", "https://platform.claude.com/docs/en/about-claude/pricing",
"2026-08-17", "2026-08-17",
360 "Batch API = 50% off standard."),
361 ("claude-fable-5", "long-context / peak-offpeak price", "NA",
362 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
363 "1M context at standard rate (no surcharge); no peak/off-peak. Note: AA labels Opus
4.8 Fallback; C audit HOLD."),
364
365 # ---- claude-opus-4.8 ----
366 ("claude-opus-4.8", "provider, exact_version, release_date", "Anthropic; claude-opus
-4.8; 2026-05-28",
367 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
368 "API ID claude-opus-4-8; exact_version slug claude-opus-4.8 aligned with Member A;
released 2026-05-28."),
369 ("claude-opus-4.8", "context_window", "1000000",
370 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
371 "1,000,000-token context window."),
372 ("claude-opus-4.8", "max_output_tokens", "128000",
373 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
374 "128K max output tokens."),
375 ("claude-opus-4.8", "vision_support", "yes",
376 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,

```

```

377     "Vision capable."),
378 ("claude-opus-4.8", "reasoning_support", "yes",
379 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
380 "Adaptive thinking with effort controls; AA labels Adaptive Reasoning, Max Effort."),
381 ("claude-opus-4.8", "api_available", "yes",
382 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
383 "GA on Claude API, Bedrock, Vertex, Foundry."),
384 ("claude-opus-4.8", "input/output/cached input price", "5.00 / 25.00 / 0.50 USD per 1M
",
385 "Anthropic models overview + pricing docs", "https://platform.claude.com/docs/zh-CN/
about-claude/models", "2026-08-16", S,
386 "Input $5, output $25; prompt-cache read = 10% of input = $0.50."),
387 ("claude-opus-4.8", "batch price", "input 2.50 / output 12.50 USD per 1M",
388 "Anthropic pricing docs", "https://platform.claude.com/docs/zh-CN/about-claude/models
", "2026-08-16", S,
389 "Batch API = 50% off."),
390 ("claude-opus-4.8", "long-context / peak-offpeak price", "NA",
391 "Anthropic models overview", "https://platform.claude.com/docs/zh-CN/about-claude/
models", "2026-08-16", S,
392 "1M context at standard rate (no surcharge); no peak/off-peak."),
393
394 # ---- gpt-5.5 ----
395 ("gpt-5.5", "provider, exact_version, release_date", "OpenAI; gpt-5.5; 2026-04-23",
396 "OpenAI model compare", "https://developers.openai.com/api/docs/models/compare/",
"2026-04-23", S,
397 "exact_version slug gpt-5.5 and release date 2026-04-23 aligned with Member A."),
398 ("gpt-5.5", "context_window", "256000",
399 "OpenAI model compare", "https://developers.openai.com/api/docs/models/compare/",
"2026-08-16", S,
400 "256,000-token context window (mature tier, smaller than GPT-5.6 Sol's 1.05M)."),
401 ("gpt-5.5", "max_output_tokens", "128000",
402 "OpenAI model compare", "https://developers.openai.com/api/docs/models/compare/",
"2026-08-16", S,
403 "128,000 max output tokens."),
404 ("gpt-5.5", "vision_support", "yes",
405 "OpenAI model compare", "https://developers.openai.com/api/docs/models/compare/",
"2026-08-16", S,
406 "Image input supported."),
407 ("gpt-5.5", "reasoning_support", "yes",
408 "OpenAI model compare", "https://developers.openai.com/api/docs/models/compare/",
"2026-08-16", S,
409 "Reasoning model; AA labels xhigh effort. AA marks deprecated but evidence publicly
verifiable before cutoff."),
410 ("gpt-5.5", "api_available", "yes",
411 "OpenAI API", "https://platform.openai.com/docs/pricing/", "2026-08-16", S,
412 "Available via OpenAI API (deprecated lifecycle but accessible before cutoff)."),
413 ("gpt-5.5", "input/output/cached input price", "3.00 / 15.00 / 0.30 USD per 1M",
414 "OpenAI API pricing", "https://platform.openai.com/docs/pricing/", "2026-08-16", S,
415 "Standard rates: input $3, cached $0.30, output $15. Mature baseline pricing, below
GPT-5.6 Sol ($5/$30)."),
416 ("gpt-5.5", "batch price", "input 1.50 / output 7.50 USD per 1M",
417 "OpenAI API pricing", "https://platform.openai.com/docs/pricing/", "2026-08-16", S,

```

```

418     "Batch API = 50% off standard."),
419 ("gpt-5.5", "long-context / peak-offpeak price", "NA",
420 "OpenAI API pricing", "https://platform.openai.com/docs/pricing/", "2026-08-16", S,
421 "No separate long-context tier documented for this tier; no peak/off-peak."),
422
423 # ---- glm-5.2 ----
424 ("glm-5.2", "provider, exact_version, release_date", "Z.ai; GLM-5.2; 2026-06-16",
425 "Zhipu official pricing", "https://open.bigmodel.cn/pricing", "2026-06-16", S,
426 "Provider aligned to Z.ai per Member A; official platform bigmodel.cn / z.ai. Release
date 2026-06-16 aligned with Member A."),
427 ("glm-5.2", "context_window", "1048576",
428 "Zhipu official pricing / OpenRouter", "https://open.bigmodel.cn/pricing",
"2026-08-09", S,
429 "1,048,576-token context window."),
430 ("glm-5.2", "max_output_tokens", "131072",
431 "Zhipu GLM-5.2 official model docs", "https://docs.bigmodel.cn/cn/guide/models/text/
glm-5.2", "2026-06-16", "2026-08-17",
432 "Official model page specifies 128K maximum output; stored as 128*1024=131072 tokens.
Third-party 262K value is rejected."),
433 ("glm-5.2", "vision_support", "no",
434 "Z.ai official model card", "https://huggingface.co/zai-org/GLM-5.2", "2026-08-17",
"2026-08-17",
435 "Official Z.ai repository is classified as Text Generation and documents a text-
generation pipeline; no image-input modality is listed."),
436 ("glm-5.2", "reasoning_support", "yes",
437 "Zhipu official pricing", "https://open.bigmodel.cn/pricing", "2026-08-09", S,
438 "GLM supports thinking/reasoning mode."),
439 ("glm-5.2", "api_available", "yes",
440 "Zhipu official pricing", "https://open.bigmodel.cn/pricing", "2026-08-09", S,
441 "Available via bigmodel.cn / Z.ai platform."),
442 ("glm-5.2", "input/output/cached input price (CNY list, USD converted)", "CNY 8 / 28 /
2 -> USD 1.13 / 3.94 / 0.28 per 1M",
443 "Zhipu official pricing", "https://open.bigmodel.cn/pricing", "2026-08-09", S,
444 "Official list price CNY: input Y8, output Y28, cache-hit Y2. Converted at 7.10 CNY/
USD on 2026-08-16."),
445 ("glm-5.2", "batch / long-context / peak-offpeak price", "NA",
446 "Zhipu official pricing", "https://open.bigmodel.cn/pricing", "2026-08-09", S,
447 "No batch discount, no separate long-context tier, no peak/off-peak found in retrieved
docs."),
448
449 # ---- gemini-3.1-pro-preview ----
450 ("gemini-3.1-pro-preview", "provider, exact_version, release_date", "Google; gemini
-3.1-pro-preview; 2026-02-19",
451 "Gemini API pricing", "https://ai.google.dev/gemini-api/docs/pricing", "2026-02-19", S
,
452 "exact_version slug gemini-3.1-pro-preview and release date 2026-02-19 aligned with
Member A."),
453 ("gemini-3.1-pro-preview", "context_window", "2000000",
454 "Gemini model card / secondary", "https://ai.google.dev/gemini-api/docs/pricing",
"2026-08-16", S,
455 "2M context per model card and secondary sources; official pricing doc prints >200K
tier but not the window number."),
456 ("gemini-3.1-pro-preview", "max_output_tokens", "NA",
457 "Gemini API pricing", "https://ai.google.dev/gemini-api/docs/pricing", "2026-08-16", S

```

```

,
"Max output not stated in retrieved sources."),
("gemini-3.1-pro-preview", "vision_support", "yes",
"Gemini API pricing", "https://ai.google.dev/gemini-api/docs/pricing", "2026-08-16", S
,
"Multimodal (text/image/audio)."),
("gemini-3.1-pro-preview", "reasoning_support", "yes",
"Gemini API pricing", "https://ai.google.dev/gemini-api/docs/pricing", "2026-08-16", S
,
"Thinking/reasoning supported."),
("gemini-3.1-pro-preview", "api_available", "yes",
"Gemini API pricing", "https://ai.google.dev/gemini-api/docs/pricing", "2026-08-16", S
,
"Available via Gemini Developer API (AI Studio) and Vertex AI. AA separately reports
both; C audit recommends freezing AI Studio."),
("gemini-3.1-pro-preview", "input/output/cached input price (<=200K)", "2.00 / 12.00 /
0.20 USD per 1M",
"Gemini API pricing", "https://ai.google.dev/gemini-api/docs/pricing", "2026-08-16", S
,
"Standard tier, prompt <=200K: input $2, output $12, cached $0.20."),
("gemini-3.1-pro-preview", "batch price (<=200K)", "input 1.00 / output 6.00 USD per 1M
",
"Gemini API pricing", "https://ai.google.dev/gemini-api/docs/pricing", "2026-08-16", S
,
"Batch = 50% off standard."),
("gemini-3.1-pro-preview", "long-context price (>200K)", "input 4.00 / output 18.00 /
cached 0.40 USD per 1M",
"Gemini API pricing", "https://ai.google.dev/gemini-api/docs/pricing", "2026-08-16", S
,
"Prompt >200K: input $4, output $18, cached $0.40. long_context_price records input $4
.00."),
("gemini-3.1-pro-preview", "peak-offpeak price", "NA",
"Gemini API pricing", "https://ai.google.dev/gemini-api/docs/pricing", "2026-08-16", S
,
"No peak/off-peak; service tiers Standard/Batch/Flex/Priority instead."),
# ---- deepseek-v4-pro-0813 ----
("deepseek-v4-pro-0813", "provider, exact_version, release_date", "DeepSeek; DeepSeek
V4 Pro 0813; 2026-08-13",
"DeepSeek API docs", "https://api-docs.deepseek.com/quick_start/pricing",
"2026-08-13", S,
"exact_version 'DeepSeek V4 Pro 0813' and model_id deepseek-v4-pro-0813 aligned with
Member A. Version stamp 0813 -> 2026-08-13."),
("deepseek-v4-pro-0813", "context_window", "1000000",
"DeepSeek API docs", "https://api-docs.deepseek.com/quick_start/pricing",
"2026-08-13", S,
"1M context length."),
("deepseek-v4-pro-0813", "max_output_tokens", "384000",
"DeepSeek API docs", "https://api-docs.deepseek.com/quick_start/pricing",
"2026-08-13", S,
"Max output 384K."),
("deepseek-v4-pro-0813", "vision_support", "no",
"DeepSeek API docs / secondary", "https://api-docs.deepseek.com/quick_start/pricing",
"2026-08-13", S,

```

```

493     "DeepSeek V4 is text-only (no multimodal) per retrieved sources."),
494     ("deepseek-v4-pro-0813", "reasoning_support", "yes",
495     "DeepSeek API docs", "https://api-docs.deepseek.com/quick_start/pricing",
496     "2026-08-13", S,
497     "Thinking mode (default) + non-thinking. AA labels Reasoning, Max Effort."),
498     ("deepseek-v4-pro-0813", "api_available", "yes",
499     "DeepSeek API docs", "https://api-docs.deepseek.com/quick_start/pricing",
500     "2026-08-13", S,
501     "Available via api.deepseek.com (OpenAI/Anthropic compatible)."),
502     ("deepseek-v4-pro-0813", "input/output/cached input price", "0.435 / 0.87 / 0.003625
503     USD per 1M",
504     "DeepSeek API docs", "https://api-docs.deepseek.com/quick_start/pricing",
505     "2026-08-13", S,
506     "Cache-miss input $0.435, output $0.87, cache-hit input $0.003625."),
507     ("deepseek-v4-pro-0813", "batch / long-context price", "NA",
508     "DeepSeek API docs", "https://api-docs.deepseek.com/quick_start/pricing",
509     "2026-08-13", S,
510     "No explicit batch discount found; 1M context at standard rate."),
511     ("deepseek-v4-pro-0813", "peak / off-peak price", "PEAK input 0.44 / output 1.32; OFF-
512     PEAK input 0.22 / output 0.66 USD per 1M",
513     "DeepSeek API docs", "https://api-docs.deepseek.com/quick_start/pricing",
514     "2026-08-13", S,
515     "Peak/off-peak billing effective 2026-08-16 16:00 UTC. peak/off-peak columns record
516     cache-miss INPUT prices; outputs in notes."),
517
518     # ---- qwen3.8-2.4t-a95b ----
519     ("qwen3.8-2.4t-a95b", "provider, exact_version, release_date", "Alibaba; Qwen3.8 2.4T
520     A95B; 2026-08-12",
521     "Qwen3.8 official blog", "https://qwen.ai/blog?id=qwen3.8", "2026-08-12", S,
522     "exact_version 'Qwen3.8 2.4T A95B' and model_id qwen3.8-2.4t-a95b aligned with Member
523     A. Release date 2026-08-12."),
524     ("qwen3.8-2.4t-a95b", "context_window", "1000000",
525     "Qwen3.8 official blog", "https://qwen.ai/blog?id=qwen3.8", "2026-08-12", S,
526     "1,000,000-token context window (config context_window: 1000000)."),
527     ("qwen3.8-2.4t-a95b", "max_output_tokens", "65536",
528     "Qwen3.8 official blog", "https://qwen.ai/blog?id=qwen3.8", "2026-08-12", S,
529     "OpenClaw config maxTokens: 65536."),
530     ("qwen3.8-2.4t-a95b", "vision_support", "yes",
531     "Qwen3.8 official blog", "https://qwen.ai/blog?id=qwen3.8", "2026-08-12", S,
532     "input_modalities text, image; multimodal agents."),
533     ("qwen3.8-2.4t-a95b", "reasoning_support", "yes",
534     "Qwen3.8 official blog", "https://qwen.ai/blog?id=qwen3.8", "2026-08-12", S,
535     "enable_thinking True; reasoning_effort xhigh/medium/low. AA does not expose a named
536     reasoning-effort level (CONDITIONAL per C audit)."),
537     ("qwen3.8-2.4t-a95b", "api_available", "yes",
538     "Qwen3.8 official blog", "https://qwen.ai/blog?id=qwen3.8", "2026-08-12", S,
539     "QwenCloud API (OpenAI/Anthropic compatible), global nodes. Alibaba Cloud first-party
540     per AA."),
541     ("qwen3.8-2.4t-a95b", "input/output/cached input price (overseas)", "2.00 / 6.00 / 0.25
542     USD per 1M",
543     "Global Times launch coverage", "https://www.globaltimes.cn/page/202608/1367420.shtml
544     ", "2026-08-12", S,
545     "Overseas price $2/$6, implicit cache-hit $0.25. Official blog omits price; domestic
546     CNY 12/36, cache-hit 1.5."),

```

```

532     ("qwen3.8-2.4t-a95b", "batch / long-context / peak-offpeak price", "NA",
533     "Qwen3.8 official blog", "https://qwen.ai/blog?id=qwen3.8", "2026-08-12", S,
534     "No batch discount, no separate long-context tier, no peak/off-peak found."),
535 ]
536
537
538 def write_csv(path: Path, fieldnames: list[str], rows: list[dict]) -> None:
539     path.parent.mkdir(parents=True, exist_ok=True)
540     with path.open("w", encoding="utf-8-sig", newline="") as handle:
541         writer = csv.DictWriter(handle, fieldnames=fieldnames)
542         writer.writeheader()
543         for row in rows:
544             writer.writerow(row)
545
546
547 def main() -> int:
548     meta_path = ROOT / "data/raw/model_metadata.csv"
549     write_csv(meta_path, list(METADATA[0].keys()), METADATA)
550     print(f"Wrote {len(METADATA)} metadata row(s) to {meta_path}")
551
552     src_path = ROOT / "data/sources/metadata_sources.csv"
553     src_fields = ["model_id", "field", "value", "source_name", "source_url",
554                  "publication_date", "retrieval_date", "notes"]
555     src_rows = [dict(zip(src_fields, rec)) for rec in SOURCES]
556     write_csv(src_path, src_fields, src_rows)
557     print(f"Wrote {len(src_rows)} source-evidence row(s) to {src_path}")
558     return 0
559
560
561 if __name__ == "__main__":
562     raise SystemExit(main())

```

source/scripts/build_paper.py

```

1  """Build the Chinese modeling report PDF with Pandoc and XeLaTeX."""
2
3  from __future__ import annotations
4
5  import shutil
6  import subprocess
7  import sys
8  import os
9  from pathlib import Path
10
11  ROOT = Path(__file__).resolve().parents[1]
12  SOURCE = ROOT / "paper" / "modeling_report.md"
13  OUTPUT = ROOT / "output" / "pdf" / "modeling_report_draft.pdf"
14
15
16  def require_program(name: str) -> str:
17     path = shutil.which(name)
18     if path is None:
19         raise RuntimeError(f"Required program is not on PATH: {name}")

```

```

20     return path
21
22
23 def main() -> int:
24     pandoc = require_program("pandoc")
25     require_program("xelatex")
26     OUTPUT.parent.mkdir(parents=True, exist_ok=True)
27     command = [
28         pandoc, str(SOURCE), "--from=markdown+tex_math_dollars+tex_math_single_backslash",
29         "--pdf-engine=xelatex", "--toc", "--toc-depth=2",
30         "--resource-path", os.pathsep.join((str(ROOT / "paper"), str(ROOT))),
31         "-V", "CJKmainfont=Microsoft YaHei",
32         "-V", "mainfont=Times New Roman", "-V", "monofont=Consolas",
33         "-V", "geometry:margin=2.2cm", "-V", "fontsize=11pt",
34         "-V", "linestretch=1.15", "-V", "colorlinks=true",
35         "-V", "linkcolor=blue", "-V", "urlcolor=blue",
36         "-o", str(OUTPUT),
37     ]
38     completed = subprocess.run(command, cwd=ROOT, check=False)
39     if completed.returncode:
40         return completed.returncode
41     print(f"Built {OUTPUT.relative_to(ROOT)} ({OUTPUT.stat().st_size:,} bytes)")
42     return 0
43
44
45 if __name__ == "__main__":
46     try:
47         raise SystemExit(main())
48     except RuntimeError as exc:
49         print(exc, file=sys.stderr)
50         raise SystemExit(2) from exc

```

source/scripts/check_coverage.py

```

1  """Report raw and strictly comparable benchmark coverage without imputation."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  from pathlib import Path
8
9  ROOT = Path(__file__).resolve().parents[1]
10 NA_VALUES = {"", "na", "n/a", "null", "none"}
11
12
13 def is_present(value: str | None) -> bool:
14     return value is not None and value.strip().lower() not in NA_VALUES
15
16
17 def read_rows(path: Path) -> list[dict[str, str]]:
18     if not path.exists():
19         raise FileNotFoundError(f"File not found: {path}")

```

```

20     with path.open(encoding="utf-8-sig", newline="") as handle:
21         return list(csv.DictReader(handle))
22
23
24 def main() -> int:
25     parser = argparse.ArgumentParser(description=__doc__)
26     parser.add_argument("--candidates", type=Path, default=ROOT / "data/model_candidates.
27         csv")
28     parser.add_argument("--scores", type=Path, default=ROOT / "data/raw/benchmark_scores.
29         csv")
30     parser.add_argument("--output", type=Path, help="Optional CSV path for the coverage
31         report")
32     args = parser.parse_args()
33
34     candidates = read_rows(args.candidates)
35     scores = read_rows(args.scores)
36     final_candidates = [
37         row for row in candidates if row.get("candidate_status", "").strip().lower() == "
38         final"
39     ]
40     active_candidates = final_candidates or [
41         row for row in candidates if row.get("candidate_status", "").strip().lower() != "
42         excluded"
43     ]
44     model_ids = {
45         row.get("model_id", "").strip()
46         for row in active_candidates
47         if is_present(row.get("model_id"))
48     }
49     by_benchmark: dict[str, set[str]] = {}
50     compatible_cohorts: dict[str, dict[tuple[str, str, str], set[str]]] = {}
51     for row in scores:
52         benchmark = row.get("benchmark", "").strip()
53         model_id = row.get("model_id", "").strip()
54         if is_present(benchmark) and model_id in model_ids and is_present(row.get("score"))
55         :
56             by_benchmark.setdefault(benchmark, set()).add(model_id)
57             if row.get("compatible", "").strip().lower() == "true":
58                 cohort = (
59                     row.get("benchmark_version", "").strip(),
60                     row.get("test_setting", "").strip(),
61                     row.get("source_name", "").strip(),
62                 )
63                 compatible_cohorts.setdefault(benchmark, {}).setdefault(cohort, set()).add(
64                     model_id)
65
66     total = len(model_ids)
67     if not by_benchmark:
68         print(f"No benchmark observations found. Active candidate models: {total}.")
69         return 0
70
71     output = []
72     for benchmark in sorted(by_benchmark):
73         raw_present = by_benchmark[benchmark]

```

```

67     cohorts = compatible_cohorts.get(benchmark, {})
68     best_key, best_present = max(
69         cohorts.items(), key=lambda item: (len(item[1]), item[0]), default=("-", "-",
70         "-"), set())
71     )
72     raw_coverage = len(raw_present) / total if total else 0.0
73     comparable_coverage = len(best_present) / total if total else 0.0
74     missing = ", ".join(sorted(model_ids - best_present)) or "-"
75     cohort_label = f"{best_key[0]} | {best_key[1]}" if best_present else "none"
76     output.append((
77         benchmark,
78         len(raw_present),
79         len(best_present),
80         total,
81         f"{raw_coverage:.1%}",
82         f"{comparable_coverage:.1%}",
83         missing,
84         cohort_label,
85     ))
86
87     headers = (
88         "Benchmark", "Raw", "Comparable", "Total", "Raw coverage",
89         "Comparable coverage", "Missing from best cohort", "Best comparable cohort",
90     )
91     widths = [max(len(str(row[i])) for row in [headers, *output]) for i in range(len(
92     headers))]
93     print(" | ".join(str(headers[i]).ljust(widths[i]) for i in range(len(headers))))
94     print("-+-".join("-" * width for width in widths))
95     for row in output:
96         print(" | ".join(str(row[i]).ljust(widths[i]) for i in range(len(headers))))
97     if args.output:
98         args.output.parent.mkdir(parents=True, exist_ok=True)
99         with args.output.open("w", encoding="utf-8", newline="") as handle:
100             writer = csv.writer(handle)
101             writer.writerow(headers)
102             writer.writerows(output)
103             print(f"Coverage report written to {args.output}.")
104     return 0
105
106 if __name__ == "__main__":
107     raise SystemExit(main())

```

source/scripts/fetch_artificial_analysis_efficiency.py

```

1  """Collect auditable Artificial Analysis efficiency candidates into staging.
2
3  This script reads only public, explicitly allow-listed model provider pages. It
4  does not call candidate LLMs, run benchmarks, infer missing values, or write any
5  raw project table. Output is review-only staging and must not be used for
6  modeling until manually reviewed and promoted in a later phase.
7
8  Examples:

```

```

9     python scripts/fetch_artificial_analysis_efficiency.py --dry-run
10    python scripts/fetch_artificial_analysis_efficiency.py --model-id kimi-k3
11    python scripts/fetch_artificial_analysis_efficiency.py --snapshot-dir PATH
12    """
13
14    from __future__ import annotations
15
16    import argparse
17    import csv
18    import hashlib
19    import html
20    import json
21    import random
22    import subprocess
23    import sys
24    import time
25    from dataclasses import dataclass
26    from datetime import date, datetime, timezone
27    from pathlib import Path
28    from typing import Any, Iterable
29    from urllib.error import HTTPError, URLError
30    from urllib.parse import urlparse
31    from urllib.request import Request, urlopen
32
33
34    ROOT = Path(__file__).resolve().parents[1]
35    DEFAULT_CONFIG = Path(__file__).with_name("artificial_analysis_targets.json")
36    DEFAULT_OUTPUT = ROOT / "results/efficiency_staging.csv"
37    DEFAULT_CANDIDATE_REF = "HEAD"
38    USER_AGENT = "TJMLL-Member-C-Efficiency-Collector/1.0 (academic research)"
39    SOURCE_NAME = "Artificial Analysis API Provider Performance Benchmarking"
40    SOURCE_TYPE = "independent_evaluation"
41    MISSING = "NA"
42    REVIEW_MARKER = "NOT FOR MODELING; REVIEW ONLY; NOT RAW DATA"
43
44    FIELDS = [
45        "model_id",
46        "candidate_exact_version",
47        "aa_display_name",
48        "configuration",
49        "reasoning_effort",
50        "provider",
51        "provider_scope",
52        "workload",
53        "metric",
54        "raw_display_value",
55        "raw_unit",
56        "normalized_value_candidate",
57        "normalized_unit_candidate",
58        "source_url",
59        "source_name",
60        "source_type",
61        "measurement_window",
62        "retrieval_date",

```

```

63     "retrieval_timestamp",
64     "http_status",
65     "content_hash",
66     "exact_version_match",
67     "configuration_identified",
68     "provider_identified",
69     "metric_label_verified",
70     "candidate_compatible",
71     "manual_review_required",
72     "review_reason",
73     "notes",
74 ]
75
76 METRICS = {
77     "Time to First Token": ("ttft_seconds", "seconds"),
78     "Output Speed": ("output_speed_tokens_per_second", "tokens/s"),
79     "End-to-End Response Time": ("end_to_end_500_tokens_seconds", "seconds"),
80 }
81
82
83 class CollectionError(RuntimeError):
84     """Raised when public page content cannot be verified safely."""
85
86
87 class FetchError(CollectionError):
88     """Raised when a public HTTP request fails, preserving a known status."""
89
90     def __init__(self, message: str, status: str = MISSING) -> None:
91         super().__init__(message)
92         self.status = status
93
94
95 @dataclass(frozen=True)
96 class FetchResult:
97     url: str
98     status: int
99     body: bytes
100     timestamp: str
101
102     @property
103     def sha256(self) -> str:
104         return hashlib.sha256(self.body).hexdigest()
105
106
107 def parse_args() -> argparse.Namespace:
108     parser = argparse.ArgumentParser(description=__doc__)
109     parser.add_argument("--config", type=Path, default=DEFAULT_CONFIG)
110     parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)
111     parser.add_argument("--candidate-ref", default=DEFAULT_CANDIDATE_REF)
112     parser.add_argument("--model-id", action="append", dest="model_ids")
113     parser.add_argument("--delay", type=float, default=2.5)
114     parser.add_argument("--jitter", type=float, default=0.5)
115     parser.add_argument("--timeout", type=float, default=30.0)
116     parser.add_argument("--retries", type=int, default=2)

```

```

117 parser.add_argument("--retrieval-date", default=date.today().isoformat())
118 parser.add_argument("--snapshot-dir", type=Path)
119 parser.add_argument("--dry-run", action="store_true")
120 parser.add_argument(
121     "--refresh",
122     action="store_true",
123     help="Revisit same-date targets; identical content is still deduplicated.",
124 )
125 args = parser.parse_args()
126 if args.delay < 2:
127     parser.error("--delay must be at least 2 seconds")
128 if args.jitter < 0:
129     parser.error("--jitter must be non-negative")
130 if args.timeout <= 0:
131     parser.error("--timeout must be positive")
132 if not 0 <= args.retries <= 5:
133     parser.error("--retries must be between 0 and 5")
134 try:
135     date.fromisoformat(args.retrieval_date)
136 except ValueError:
137     parser.error("--retrieval-date must be YYYY-MM-DD")
138 return args
139
140
141 def read_candidate_versions(ref: str) -> dict[str, str]:
142     command = ["git", "show", f"{ref}:data/model_candidates.csv"]
143     completed = subprocess.run(
144         command,
145         cwd=ROOT,
146         check=False,
147         capture_output=True,
148         text=True,
149         encoding="utf-8",
150     )
151     if completed.returncode != 0:
152         raise CollectionError(
153             f"Cannot read candidate table from {ref}: {completed.stderr.strip()}"
154         )
155     rows = csv.DictReader(completed.stdout.splitlines())
156     candidates = {
157         row["model_id"].strip(): row["exact_version"].strip()
158         for row in rows
159         if row.get("model_id", "").strip()
160     }
161     if not candidates:
162         raise CollectionError(f"Candidate table at {ref} is empty")
163     return candidates
164
165
166 def load_targets(path: Path, candidates: dict[str, str]) -> list[dict[str, Any]]:
167     try:
168         targets = json.loads(path.read_text(encoding="utf-8"))
169     except (OSError, json.JSONDecodeError) as exc:
170         raise CollectionError(f"Cannot load target configuration {path}: {exc}") from exc

```

```

171 if not isinstance(targets, list):
172     raise CollectionError("Target configuration must be a JSON array")
173
174 required = {
175     "model_id",
176     "candidate_exact_version",
177     "aa_slug",
178     "aa_display_name",
179     "configuration",
180     "reasoning_effort",
181     "provider",
182     "provider_scope",
183     "url",
184     "status",
185     "candidate_compatible",
186     "review_reason",
187 }
188 seen: set[tuple[str, str, str]] = set()
189 for index, target in enumerate(targets, start=1):
190     if not isinstance(target, dict):
191         raise CollectionError(f"Target {index} is not an object")
192     missing_keys = sorted(required - target.keys())
193     if missing_keys:
194         raise CollectionError(f"Target {index} lacks keys: {missing_keys}")
195     model_id = str(target["model_id"])
196     if model_id not in candidates:
197         raise CollectionError(f"Target model_id is absent from {DEFAULT_CANDIDATE_REF}: {model_id}")
198     if str(target["candidate_exact_version"]) != candidates[model_id]:
199         raise CollectionError(
200             f"Exact-version mismatch for {model_id}: config={target['candidate_exact_version']}!r, "
201             f"candidate={candidates[model_id]}!r"
202         )
203     parsed = urlparse(str(target["url"]))
204     if parsed.scheme != "https" or parsed.netloc != "artificialanalysis.ai":
205         raise CollectionError(f"Target URL is not an approved AA HTTPS page: {target['url']}")
206     if not parsed.path.startswith("/models/") or not parsed.path.endswith("/providers"):
207         raise CollectionError(f"Target must be an explicit model providers page: {target['url']}")
208     if target["provider_scope"] not in {
209         "model_aggregate",
210         "first_party",
211         "specific_provider",
212         "unknown",
213     }:
214         raise CollectionError(f"Invalid provider_scope for {model_id}")
215     key = (model_id, str(target["configuration"]), str(target["provider"]))
216     if key in seen:
217         raise CollectionError(f"Duplicate target configuration: {key}")
218     seen.add(key)
219 return targets

```

```

220
221
222 def fetch_public_html(url: str, timeout: float, retries: int) -> FetchResult:
223     request = Request(
224         url,
225         headers={
226             "User-Agent": USER_AGENT,
227             "Accept": "text/html,application/xhtml+xml",
228         },
229         method="GET",
230     )
231     last_error: Exception | None = None
232     for attempt in range(retries + 1):
233         try:
234             with urlopen(request, timeout=timeout) as response:
235                 final_url = response.geturl()
236                 if urlparse(final_url).netloc != "artificialanalysis.ai":
237                     raise CollectionError(f"Unexpected redirect outside AA: {final_url}")
238                 status = int(response.status)
239                 body = response.read()
240                 if status != 200:
241                     raise FetchError(f"Unexpected HTTP status {status}", str(status))
242                 content_type = response.headers.get_content_type()
243                 if content_type not in {"text/html", "application/xhtml+xml"}:
244                     raise FetchError(f"Unexpected content type: {content_type}", str(status))
245             if not body:
246                 raise FetchError("Empty response body", str(status))
247             if len(body) > 10 * 1024 * 1024:
248                 raise FetchError("Response exceeds 10 MiB safety limit", str(status))
249             timestamp = datetime.now(timezone.utc).isoformat(timespec="seconds")
250             return FetchResult(final_url, status, body, timestamp)
251         except HTTPError as exc:
252             last_error = FetchError(f"HTTP error {exc.code}", str(exc.code))
253             if attempt < retries:
254                 time.sleep(min(2 ** attempt, 4))
255         except (URLError, TimeoutError, OSError, CollectionError) as exc:
256             last_error = exc
257             if attempt < retries:
258                 time.sleep(min(2 ** attempt, 4))
259     status = last_error.status if isinstance(last_error, FetchError) else MISSING
260     raise FetchError(
261         f"Fetch failed after {retries + 1} attempt(s): {last_error}", status
262     )
263
264
265 def decode_public_html(body: bytes) -> str:
266     try:
267         decoded = body.decode("utf-8")
268     except UnicodeDecodeError as exc:
269         raise CollectionError("Response is not valid UTF-8 HTML") from exc
270     # Next.js returns public server-rendered data inside escaped script text.
271     decoded = html.unescape(decoded).replace('\\"', '"')
272     return decoded

```

```

273
274
275 def extract_balanced_object(text: str, start: int) -> str:
276     if not text.startswith("{", start):
277         raise CollectionError("JSON object start not found")
278     depth = 0
279     in_string = False
280     escaped = False
281     for index in range(start, len(text)):
282         char = text[index]
283         if in_string:
284             if escaped:
285                 escaped = False
286             elif char == "\\":
287                 escaped = True
288             elif char == "'":
289                 in_string = False
290                 continue
291         if char == "'":
292             in_string = True
293         elif char == "{":
294             depth += 1
295         elif char == "}":
296             depth -= 1
297             if depth == 0:
298                 return text[start : index + 1]
299     raise CollectionError("Unbalanced JSON object in page")
300
301
302 def provider_objects(page: str, aa_slug: str) -> list[dict[str, Any]]:
303     marker = f'"model":{{"slug":"{aa_slug}"}}'
304     results: dict[str, dict[str, Any]] = {}
305     search_from = 0
306     while True:
307         position = page.find(marker, search_from)
308         if position < 0:
309             break
310         start = page.rfind('"id":', max(0, position - 6000), position)
311         if start >= 0:
312             try:
313                 item = json.loads(extract_balanced_object(page, start))
314             except (json.JSONDecodeError, CollectionError):
315                 item = None
316             if isinstance(item, dict):
317                 model = item.get("model") or {}
318                 host = item.get("host") or {}
319                 performance = item.get("performance") or {}
320                 if model.get("slug") == aa_slug and host.get("label") and performance:
321                     results[str(item.get("slug", host.get("label")))] = item
322                 search_from = position + len(marker)
323     return list(results.values())
324
325
326 def number_string(value: Any) -> str:

```

```

327     if isinstance(value, bool) or not isinstance(value, (int, float)):
328         return MISSING
329     return format(value, ".15g")
330
331
332 def find_metric_values(item: dict[str, Any]) -> dict[str, str]:
333     performance = item.get("performance")
334     if not isinstance(performance, dict):
335         return {name: MISSING for name in METRICS}
336     output_speed = performance.get("outputSpeed") or {}
337     ttft = performance.get("timeToFirstToken") or {}
338     end_to_end = performance.get("endToEndResponseTime") or {}
339     return {
340         "Time to First Token": number_string(ttft.get("median")),
341         "Output Speed": number_string(output_speed.get("median")),
342         "End-to-End Response Time": number_string(end_to_end.get("totalTime")),
343     }
344
345
346 def metric_labels(page: str) -> dict[str, bool]:
347     return {
348         "Time to First Token": "Time to First Token" in page,
349         "Output Speed": "Output Speed" in page,
350         "End-to-End Response Time": "End-to-End Response Time" in page
351         and "Seconds to output 500 tokens" in page,
352     }
353
354
355 def bool_text(value: bool) -> str:
356     return "true" if value else "false"
357
358
359 def na_safe(value: Any) -> str:
360     text_value = str(value).strip() if value is not None else ""
361     return text_value or MISSING
362
363
364 def failure_rows(
365     target: dict[str, Any], retrieval_date: str, reason: str, status: str = MISSING
366 ) -> list[dict[str, str]]:
367     rows = []
368     for metric_name, (_, unit) in METRICS.items():
369         rows.append(
370             make_row(
371                 target=target,
372                 retrieval_date=retrieval_date,
373                 retrieval_timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
374                 http_status=status,
375                 content_hash=MISSING,
376                 metric_name=metric_name,
377                 value=MISSING,
378                 unit=unit,
379                 exact_match=False,

```

```

380         provider_found=False,
381         label_verified=False,
382         review_reason=reason,
383     )
384 )
385 return rows
386
387
388 def make_row(
389     *,
390     target: dict[str, Any],
391     retrieval_date: str,
392     retrieval_timestamp: str,
393     http_status: str,
394     content_hash: str,
395     metric_name: str,
396     value: str,
397     unit: str,
398     exact_match: bool,
399     provider_found: bool,
400     label_verified: bool,
401     review_reason: str,
402 ) -> dict[str, str]:
403     has_value = value != MISSING
404     manual_review = (
405         target.get("status") != "confirmed"
406         or not exact_match
407         or not provider_found
408         or not label_verified
409         or not has_value
410         or not bool(target.get("candidate_compatible"))
411     )
412     _, normalized_unit = METRICS[metric_name]
413     row = {
414         "model_id": target["model_id"],
415         "candidate_exact_version": target["candidate_exact_version"],
416         "aa_display_name": target["aa_display_name"],
417         "configuration": target["configuration"],
418         "reasoning_effort": target["reasoning_effort"],
419         "provider": target["provider"],
420         "provider_scope": target["provider_scope"],
421         "workload": "10k input tokens; single prompt; 500-token standardized E2E",
422         "metric": metric_name,
423         "raw_display_value": value,
424         "raw_unit": unit,
425         "normalized_value_candidate": value,
426         "normalized_unit_candidate": normalized_unit,
427         "source_url": target["url"],
428         "source_name": SOURCE_NAME,
429         "source_type": SOURCE_TYPE,
430         "measurement_window": "median (P50) over past 72 hours",
431         "retrieval_date": retrieval_date,
432         "retrieval_timestamp": retrieval_timestamp,
433         "http_status": http_status,

```

```

434         "content_hash": content_hash,
435         "exact_version_match": bool_text(exact_match),
436         "configuration_identified": bool_text(target["configuration"] != MISSING),
437         "provider_identified": bool_text(provider_found),
438         "metric_label_verified": bool_text(label_verified),
439         "candidate_compatible": bool_text(bool(target.get("candidate_compatible"))),
440         "manual_review_required": bool_text(manual_review),
441         "review_reason": review_reason,
442         "notes": (
443             f"{REVIEW_MARKER}; value read from AA public server-rendered performance data;
444             "
445             "E2E is AA's directly published standardized 500-token totalTime, not an
446             arbitrary-length answer."
447         ),
448     }
449     return {field: na_safe(row.get(field)) for field in FIELDS}
450
451 def existing_rows(path: Path) -> list[dict[str, str]]:
452     if not path.exists():
453         return []
454     with path.open(encoding="utf-8-sig", newline="") as handle:
455         rows = list(csv.DictReader(handle))
456     if rows and set(rows[0]) != set(FIELDS):
457         raise CollectionError(f"Existing staging schema does not match collector schema: {
458             path}")
459     return rows
460
461 def identity_key(row: dict[str, str]) -> tuple[str, ...]:
462     return (
463         row["model_id"],
464         row["configuration"],
465         row["provider"],
466         row["metric"],
467         row["source_url"],
468         row["retrieval_date"],
469     )
470
471 def exact_evidence_key(row: dict[str, str]) -> tuple[str, ...]:
472     return identity_key(row) + (row["content_hash"], row["raw_display_value"])
473
474
475 def write_rows(path: Path, old_rows: list[dict[str, str]], new_rows: Iterable[dict[str, str
476     ]]) -> int:
477     evidence = {exact_evidence_key(row) for row in old_rows}
478     additions = []
479     for row in new_rows:
480         key = exact_evidence_key(row)
481         if key not in evidence:
482             additions.append(row)
483             evidence.add(key)
484     if not additions:

```

```

484         return 0
485     path.parent.mkdir(parents=True, exist_ok=True)
486     with path.open("w", encoding="utf-8", newline="") as handle:
487         writer = csv.DictWriter(handle, fieldnames=FIELDS)
488         writer.writeheader()
489         writer.writerows([*old_rows, *additions])
490     return len(additions)
491
492
493 def save_snapshot(directory: Path, target: dict[str, Any], fetched: FetchResult) -> Path:
494     directory.mkdir(parents=True, exist_ok=True)
495     stamp = fetched.timestamp.replace(":", "").replace("+00:00", "Z")
496     provider = "".join(c if c.isalnum() else "-" for c in target["provider"]).strip("-")
497     filename = f"{target['model_id']}__{provider}__{stamp}__{fetched.sha256[:12]}.html"
498     path = directory / filename
499     if not path.exists():
500         path.write_bytes(fetched.body)
501     return path
502
503
504 def collect_target(
505     target: dict[str, Any], args: argparse.Namespace, candidate_versions: dict[str, str]
506 ) -> list[dict[str, str]]:
507     exact_match = candidate_versions.get(target["model_id"]) == target["
candidate_exact_version"]
508     try:
509         fetched = fetch_public_html(target["url"], args.timeout, args.retries)
510         page = decode_public_html(fetched.body)
511         if target["aa_display_name"] not in page:
512             raise CollectionError(
513                 f"AA display name not found in page: {target['aa_display_name']}"
514             )
515         items = provider_objects(page, target["aa_slug"])
516         selected = [item for item in items if (item.get("host") or {}).get("label") ==
target["provider"]]
517         if len(selected) != 1:
518             raise CollectionError(
519                 f"Expected one provider object for {target['provider']}!r}; found {len(
selected)})"
520             )
521         provider_item = selected[0]
522         values = find_metric_values(provider_item)
523         labels = metric_labels(page)
524         if args.snapshot_dir:
525             save_snapshot(args.snapshot_dir, target, fetched)
526         rows = []
527         for metric_name, (_, unit) in METRICS.items():
528             reason_parts = [target["review_reason"]]
529             if values[metric_name] == MISSING:
530                 reason_parts.append("Direct metric value is absent from the provider
performance object.")
531             if not labels[metric_name]:
532                 reason_parts.append("Expected metric label is absent from the public page
.")

```

```

533         rows.append(
534             make_row(
535                 target=target,
536                 retrieval_date=args.retrieval_date,
537                 retrieval_timestamp=fetched.timestamp,
538                 http_status=str(fetched.status),
539                 content_hash=fetched.sha256,
540                 metric_name=metric_name,
541                 value=values[metric_name],
542                 unit=unit,
543                 exact_match=exact_match,
544                 provider_found=True,
545                 label_verified=labels[metric_name],
546                 review_reason=" ".join(reason_parts),
547             )
548         )
549     return rows
550 except FetchError as exc:
551     return failure_rows(target, args.retrieval_date, str(exc), exc.status)
552 except CollectionError as exc:
553     return failure_rows(target, args.retrieval_date, str(exc))
554
555
556 def main() -> int:
557     args = parse_args()
558     try:
559         candidates = read_candidate_versions(args.candidate_ref)
560         targets = load_targets(args.config, candidates)
561         if args.model_ids:
562             requested = set(args.model_ids)
563             unknown = requested - {target["model_id"] for target in targets}
564             if unknown:
565                 raise CollectionError(f"Unknown --model-id values: {sorted(unknown)}")
566             targets = [target for target in targets if target["model_id"] in requested]
567         current_rows = existing_rows(args.output)
568     except CollectionError as exc:
569         print(f"ERROR: {exc}", file=sys.stderr)
570         return 2
571
572     print(f"Candidate source: {args.candidate_ref} ({len(candidates)} models)")
573     print(f"Allow-list: {args.config} ({len(targets)} selected targets)")
574     print(f"Output: {args.output} [{REVIEW_MARKER}]")
575     for target in targets:
576         print(
577             f"- {target['model_id']} | {target['configuration']} | "
578             f"{target['provider']} | {target['url']}"
579         )
580     if args.dry_run:
581         print("Dry-run complete: no HTTP requests sent and no staging file written.")
582         return 0
583
584     same_date_keys = {identity_key(row) for row in current_rows}
585     all_new_rows: list[dict[str, str]] = []
586     visited = 0

```

```

587     for index, target in enumerate(targets):
588         expected_keys = {
589             (
590                 target["model_id"],
591                 target["configuration"],
592                 target["provider"],
593                 metric,
594                 target["url"],
595                 args.retrieval_date,
596             )
597             for metric in METRICS
598         }
599         if not args.refresh and expected_keys.issubset(same_date_keys):
600             print(f"SKIP {target['model_id']}: same-date staging records already exist")
601             continue
602         if visited and index:
603             wait_seconds = args.delay + random.uniform(0, args.jitter)
604             time.sleep(wait_seconds)
605         print(f"FETCH {target['model_id']}: {target['url']}")
606         rows = collect_target(target, args, candidates)
607         all_new_rows.extend(rows)
608         visited += 1
609
610     try:
611         added = write_rows(args.output, current_rows, all_new_rows)
612     except (OSError, CollectionError) as exc:
613         print(f"ERROR: cannot write staging output: {exc}", file=sys.stderr)
614         return 3
615     values = [row for row in all_new_rows if row["raw_display_value"] != MISSING]
616     print(
617         f"Collection complete: visited={visited}, candidate rows={len(all_new_rows)}, "
618         f"numeric rows={len(values)}, appended={added}."
619     )
620     return 0
621
622
623 if __name__ == "__main__":
624     raise SystemExit(main())

```

source/scripts/generate_figures.py

```

1  """Generate dependency-free SVG figures from committed result tables."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  from html import escape
8  from pathlib import Path
9
10 ROOT = Path(__file__).resolve().parents[1]
11 DEFAULT_OUTPUT = ROOT / "figures"
12 COLORS = ["#2563eb", "#0f766e", "#7c3aed", "#ea580c", "#475569", "#dc2626"]

```

```

13
14
15 def read_rows(path: Path) -> list[dict[str, str]]:
16     with path.open(encoding="utf-8-sig", newline="") as handle:
17         return list(csv.DictReader(handle))
18
19
20 def save(path: Path, body: str, width: int, height: int) -> None:
21     path.parent.mkdir(parents=True, exist_ok=True)
22     svg = (
23         f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}" height="{height}" viewBox
24         ="0 0 {width} {height}">'
25         ' <rect width="100%" height="100%" fill="white"/>'
26         ' <style>text{font-family:Arial,"Microsoft YaHei",sans-serif;fill:#172033}.title{
27         font-size:22px;font-weight:700}.label{font-size:14px}.small{font-size:12px;fill
28         :#475569}</style>'
29         + body + "</svg>"
30     )
31     path.write_text(svg, encoding="utf-8")
32
33 def bar_chart(path: Path, title: str, labels: list[str], values: list[float], value_format:
34     str = ".3f") -> None:
35     width, left, right, top, row_height = 900, 210, 70, 70, 46
36     height = top + len(labels) * row_height + 45
37     maximum = max(values) if values else 1
38     plot_width = width - left - right
39     parts = [f'<text class="title" x="{width/2}" y="34" text-anchor="middle">{escape(title)}
40     </text>']
41     for index, (label, value) in enumerate(zip(labels, values)):
42         y = top + index * row_height
43         bar_width = plot_width * value / maximum if maximum else 0
44         parts.append(f'<text class="label" x="{left-12}" y="{y+20}" text-anchor="end">{
45         escape(label)}</text>')
46         parts.append(f'<rect x="{left}" y="{y}" width="{bar_width:.1f}" height="27" rx="4"
47         fill="{COLORS[index % len(COLORS)]}" />')
48         parts.append(f'<text class="label" x="{left+bar_width+8:.1f}" y="{y+20}">{format(
49         value, value_format)}</text>')
50     save(path, "".join(parts), width, height)
51
52 def generate(output_dir: Path) -> list[Path]:
53     ranking = [row for row in read_rows(ROOT / "results/phase4/general_ranking.csv") if row
54     ["topsis_score"] != "NA"]
55     ranking.sort(key=lambda row: int(row["rank"]))
56     bar_chart(
57         output_dir / "general_ranking.svg", "CRITIC - TOPSIS 综合排名",
58         [row["model_name"] for row in ranking], [float(row["topsis_score"]) for row in
59         ranking],
60     )
61     weights = read_rows(ROOT / "results/phase4/critic_weights.csv")
62     weights.sort(key=lambda row: float(row["critic_weight"]), reverse=True)
63     bar_chart(
64         output_dir / "critic_weights.svg", "CRITIC 指标权重",

```

```

57     [row["indicator"] for row in weights], [float(row["critic_weight"]) for row in
58     weights],
59 )
60
61 scenario = [row for row in read_rows(ROOT / "results/phase5/scenario_rankings.csv") if
62 row["scenario_rank"] != "NA"]
63 scenarios = list(dict.fromkeys(row["scenario"] for row in scenario))
64 models = list(dict.fromkeys(row["model_name"] for row in scenario))
65 width, height = 950, 410
66 x_positions = [190, 470, 750]
67 parts = [f'<text class="title" x="{width/2}" y="34" text-anchor="middle">三类场景排名变化
68 (1 为最佳) </text>']
69 for x, name in zip(x_positions, scenarios):
70     parts.append(f'<text class="label" x="{x}" y="68" text-anchor="middle">{escape(name
71 )}</text>')
72     parts.append(f'<line x1="{x}" y1="90" x2="{x}" y2="330" stroke="#cbd5e1"/>')
73     for rank in range(1, 6):
74         y = 90 + (rank - 1) * 60
75         parts.append(f'<text class="small" x="{x-18}" y="{y+4}" text-anchor="end">{rank
76 }</text>')
77 for index, model in enumerate(models):
78     points = []
79     for x, name in zip(x_positions, scenarios):
80         row = next(item for item in scenario if item["scenario"] == name and item["
81 model_name"] == model)
82         y = 90 + (int(row["scenario_rank"]) - 1) * 60
83         points.append((x, y))
84         color = COLORS[index]
85         parts.append(f'<polyline points="{ " ".join(f"{x},{y}" for x,y in points)}" fill="
86 none" stroke="{color}" stroke-width="3"/>')
87         for x, y in points:
88             parts.append(f'<circle cx="{x}" cy="{y}" r="5" fill="{color}"/>')
89             parts.append(f'<text class="small" x="{x_positions[-1]+18}" y="{points[-1][1]+4}"
90 fill="{color}">{escape(model)}</text>')
91 save(output_dir / "scenario_rank_changes.svg", "".join(parts), width, height)
92
93 cost = [row for row in read_rows(ROOT / "results/phase6/performance_cost.csv") if row["
94 analysis_status"] == "included"]
95 width, height, left, bottom, top, right = 900, 520, 90, 70, 60, 100
96 max_cost = max(float(row["standardized_api_cost_usd"]) for row in cost) * 1.1
97 max_score = max(float(row["general_performance_score"]) for row in cost) * 1.1
98 parts = [f'<text class="title" x="{width/2}" y="34" text-anchor="middle">性能—API 成本
99 Pareto 前沿</text>']
100 parts += [
101     f'<line x1="{left}" y1="{height-bottom}" x2="{width-right}" y2="{height-bottom}"
102 stroke="#334155"/>',
103     f'<line x1="{left}" y1="{height-bottom}" x2="{left}" y2="{top}" stroke
104 ="#334155"/>',
105     f'<text class="label" x="{(left+width-right)/2}" y="{height-20}" text-anchor="
106 middle">标准工作负载 API 成本 (USD) </text>',
107     f'<text class="label" transform="translate(24 {(top+height-bottom)/2}) rotate(-90)"
108 text-anchor="middle">综合性能得分</text>',
109 ]
110 frontier_points = []

```

```

97     for index, row in enumerate(cost):
98         x = left + float(row["standardized_api_cost_usd"]) / max_cost * (width - left -
right)
99         y = height - bottom - float(row["general_performance_score"]) / max_score * (height
- top - bottom)
100         color = "#dc2626" if row["pareto_frontier"] == "true" else "#64748b"
101         parts.append(f'<circle cx="{x:.1f}" cy="{y:.1f}" r="8" fill="{color}"/>')
102         parts.append(f'<text class="small" x="{x+10:.1f}" y="{y-9:.1f}">{escape(row["
model_name"])}</text>')
103         if row["pareto_frontier"] == "true":
104             frontier_points.append((x, y))
105     frontier_points.sort()
106     if len(frontier_points) > 1:
107         parts.append(f'<polyline points="{{" ".join(f"{x:.1f},{y:.1f}" for x,y in
frontier_points)}" fill="none" stroke="#dc2626" stroke-width="2" stroke-dasharray="6
4"/>')
108     save(output_dir / "performance_cost_pareto.svg", "".join(parts), width, height)
109     return [
110         output_dir / "general_ranking.svg", output_dir / "critic_weights.svg",
111         output_dir / "scenario_rank_changes.svg", output_dir / "performance_cost_pareto.svg
",
112     ]
113
114
115 def main() -> int:
116     parser = argparse.ArgumentParser(description=__doc__)
117     parser.add_argument("--output-dir", type=Path, default=DEFAULT_OUTPUT)
118     args = parser.parse_args()
119     paths = generate(args.output_dir)
120     print(f"Generated {len(paths)} SVG figures in {args.output_dir}.")
121     return 0
122
123
124 if __name__ == "__main__":
125     raise SystemExit(main())

```

source/scripts/merge_data.py

```

1  """Safely merge raw tables by model_id while preserving NA values.
2
3  The current scaffold writes one row per candidate and stores all matching raw
4  records as JSON arrays. It deliberately performs no imputation, aggregation,
5  deduplication, or score selection.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import csv
12 import json
13 from collections import defaultdict
14 from pathlib import Path
15

```

```

16 ROOT = Path(__file__).resolve().parents[1]
17
18
19 def read_rows(path: Path) -> list[dict[str, str]]:
20     with path.open(encoding="utf-8-sig", newline="") as handle:
21         return list(csv.DictReader(handle))
22
23
24 def grouped(rows: list[dict[str, str]]) -> dict[str, list[dict[str, str]]]:
25     result: dict[str, list[dict[str, str]]] = defaultdict(list)
26     for row in rows:
27         result[row.get("model_id", "")].append(row)
28     return result
29
30
31 def main() -> int:
32     parser = argparse.ArgumentParser(description=__doc__)
33     parser.add_argument("--output", type=Path, default=ROOT / "data/merged/model_dataset.
34         csv")
35     parser.add_argument("--force", action="store_true", help="Replace an existing output
36         file")
37     parser.add_argument("--final-only", action="store_true", help="Write only
38         candidate_status=final models")
39     args = parser.parse_args()
40     if args.output.exists() and not args.force:
41         print(f"Refusing to overwrite existing file: {args.output}. Use --force after
42         review.")
43         return 2
44
45     candidates = read_rows(ROOT / "data/model_candidates.csv")
46     if args.final_only:
47         candidates = [row for row in candidates if row.get("candidate_status") == "final"]
48     benchmarks = grouped(read_rows(ROOT / "data/raw/benchmark_scores.csv"))
49     metadata = grouped(read_rows(ROOT / "data/raw/model_metadata.csv"))
50     efficiency = grouped(read_rows(ROOT / "data/raw/cost_efficiency.csv"))
51
52     args.output.parent.mkdir(parents=True, exist_ok=True)
53     fields = ["model_id", "provider", "model_name", "exact_version", "candidate_status", "
54         benchmark_records", "metadata_records", "efficiency_records"]
55     with args.output.open("w", encoding="utf-8", newline="") as handle:
56         writer = csv.DictWriter(handle, fieldnames=fields)
57         writer.writeheader()
58         for candidate in candidates:
59             model_id = candidate.get("model_id", "")
60             writer.writerow({
61                 "model_id": model_id,
62                 "provider": candidate.get("provider", "NA") or "NA",
63                 "model_name": candidate.get("model_name", "NA") or "NA",
64                 "exact_version": candidate.get("exact_version", "NA") or "NA",
65                 "candidate_status": candidate.get("candidate_status", "NA") or "NA",
66                 "benchmark_records": json.dumps(benchmarks.get(model_id, []), ensure_ascii=
67                 False),
68                 "metadata_records": json.dumps(metadata.get(model_id, []), ensure_ascii=
69                 False),

```

```

63         "efficiency_records": json.dumps(efficiency.get(model_id, []), ensure_ascii
        =False),
64     })
65     print(f"Wrote {len(candidates)} candidate row(s) to {args.output}; no values were
        imputed or discarded.")
66     return 0
67
68
69 if __name__ == "__main__":
70     raise SystemExit(main())

```

source/scripts/model_phase4_critic_topsis.py

```

1  """Compute CRITIC weights and a no-imputation TOPSIS general ranking."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  import math
8  from pathlib import Path
9
10 try:
11     from scripts.analyze_phase3_indicators import pearson
12 except ModuleNotFoundError: # Direct execution adds scripts/, not repository root, to sys.
    path.
13     from analyze_phase3_indicators import pearson
14
15 ROOT = Path(__file__).resolve().parents[1]
16 INPUT = ROOT / "data/processed/core_benchmark_long.csv"
17 DEFAULT_OUTPUT = ROOT / "results/phase4"
18 NA = "NA"
19
20
21 def read_rows(path: Path) -> list[dict[str, str]]:
22     with path.open(encoding="utf-8-sig", newline="") as handle:
23         return list(csv.DictReader(handle))
24
25
26 def write_rows(path: Path, fields: list[str], rows: list[dict[str, str]]) -> None:
27     path.parent.mkdir(parents=True, exist_ok=True)
28     with path.open("w", encoding="utf-8", newline="") as handle:
29         writer = csv.DictWriter(handle, fieldnames=fields)
30         writer.writeheader()
31         writer.writerows(rows)
32
33
34 def minmax(values: dict[str, float], higher_is_better: bool) -> dict[str, float]:
35     low, high = min(values.values()), max(values.values())
36     if high == low:
37         return {key: 0.5 for key in values}
38     if higher_is_better:
39         return {key: (value - low) / (high - low) for key, value in values.items()}

```

```

40     return {key: (high - value) / (high - low) for key, value in values.items()}
41
42
43 def population_std(values: list[float]) -> float:
44     center = sum(values) / len(values)
45     return math.sqrt(sum((value - center) ** 2 for value in values) / len(values))
46
47
48 def compute(output_dir: Path) -> dict[str, object]:
49     rows = read_rows(INPUT)
50     indicators = list(dict.fromkeys(row["indicator_key"] for row in rows))
51     models = list(dict.fromkeys(row["model_id"] for row in rows))
52     names = {row["model_id"]: row["model_name"] for row in rows}
53     dimensions = {row["indicator_key"]: row["dimension"] for row in rows}
54     indicator_names = {row["indicator_key"]: row["indicator"] for row in rows}
55     raw: dict[str, dict[str, float]] = {indicator: {} for indicator in indicators}
56     for row in rows:
57         if row["score"] != NA:
58             raw[row["indicator_key"]][row["model_id"]] = float(row["score"])
59     normalized = {
60         indicator: minmax(raw[indicator], next(row["higher_is_better"] == "true" for row in
61         rows if row["indicator_key"] == indicator))
62         for indicator in indicators
63     }
64
65     normalization_rows = []
66     for model_id in models:
67         for indicator in indicators:
68             normalization_rows.append({
69                 "model_id": model_id,
70                 "model_name": names[model_id],
71                 "indicator_key": indicator,
72                 "raw_score": f"{raw[indicator][model_id]:.10g}" if model_id in raw[
73                 indicator] else NA,
74                 "minmax_score": f"{normalized[indicator][model_id]:.10f}" if model_id in
75                 normalized[indicator] else NA,
76                 "normalization": "benefit_minmax_available_observations",
77             })
78     write_rows(
79         output_dir / "normalized_scores.csv",
80         ["model_id", "model_name", "indicator_key", "raw_score", "minmax_score", "
81         normalization"],
82         normalization_rows,
83     )
84
85     information: dict[str, float] = {}
86     weight_rows = []
87     for left in indicators:
88         correlations = []
89         for right in indicators:
90             if left == right:
91                 continue
92             common = sorted(set(normalized[left]) & set(normalized[right]))
93             correlation = pearson(

```

```

90         [normalized[left][model] for model in common],
91         [normalized[right][model] for model in common],
92     )
93     if correlation is None:
94         raise ValueError(f"Cannot compute correlation for {left}/{right}")
95     correlations.append(correlation)
96     contrast = population_std(list(normalized[left].values()))
97     conflict = sum(1 - value for value in correlations)
98     information[left] = contrast * conflict
99     weight_rows.append({
100         "indicator_key": left,
101         "dimension": dimensions[left],
102         "indicator": indicator_names[left],
103         "available_models": str(len(normalized[left])),
104         "contrast_std": f"{contrast:.10f}",
105         "conflict_sum": f"{conflict:.10f}",
106         "information_content": f"{information[left]:.10f}",
107         "critic_weight": "",
108     })
109     total_information = sum(information.values())
110     if total_information <= 0:
111         raise ValueError("CRITIC information total must be positive")
112     weights = {key: value / total_information for key, value in information.items()}
113     for row in weight_rows:
114         row["critic_weight"] = f"{weights[row['indicator_key']]:.10f}"
115     write_rows(output_dir / "critic_weights.csv", list(weight_rows[0]), weight_rows)
116
117     complete = [model for model in models if all(model in normalized[indicator] for
118 indicator in indicators)]
119     if len(complete) < 2:
120         raise ValueError("TOPSIS requires at least two complete models")
121     ranked = []
122     for model_id in complete:
123         positive = math.sqrt(sum((weights[indicator] * normalized[indicator][model_id] -
124 weights[indicator]) ** 2 for indicator in indicators))
125         negative = math.sqrt(sum((weights[indicator] * normalized[indicator][model_id]) **
126 2 for indicator in indicators))
127         closeness = negative / (positive + negative)
128         ranked.append((model_id, closeness, positive, negative))
129     ranked.sort(key=lambda item: (-item[1], item[0]))
130     rank_lookup = {model_id: index for index, (model_id, *_rest) in enumerate(ranked, start
131 =1)}
132     score_lookup = {model_id: values for model_id, *values in ranked}
133     ranking_rows = []
134     for model_id in models:
135         available = sum(model_id in normalized[indicator] for indicator in indicators)
136         if model_id in score_lookup:
137             closeness, positive, negative = score_lookup[model_id]
138             ranking_rows.append({
139                 "model_id": model_id, "model_name": names[model_id], "available_indicators
140 ": str(available),
141                 "total_indicators": str(len(indicators)), "coverage_percent": f"{available
142 / len(indicators) * 100:.1f}",
143                 "distance_positive": f"{positive:.10f}", "distance_negative": f"{negative

```

```

:.10f}",
138     "topsis_score": f"{closeness:.10f}", "rank": str(rank_lookup[model_id]), "
ranking_status": "ranked_complete_case",
139     })
140     else:
141         ranking_rows.append({
142             "model_id": model_id, "model_name": names[model_id], "available_indicators
": str(available),
143             "total_indicators": str(len(indicators)), "coverage_percent": f"{available
/ len(indicators) * 100:.1f}",
144             "distance_positive": NA, "distance_negative": NA, "topsis_score": NA, "rank
": NA,
145             "ranking_status": "not_ranked_insufficient_coverage",
146         })
147     ranking_rows.sort(key=lambda row: (row["rank"] == NA, int(row["rank"]) if row["rank"]
!= NA else 999))
148     write_rows(
149         output_dir / "general_ranking.csv",
150         [
151             "model_id", "model_name", "available_indicators", "total_indicators", "
coverage_percent",
152             "distance_positive", "distance_negative", "topsis_score", "rank", "
ranking_status",
153         ],
154         ranking_rows,
155     )
156
157     kimi_rows = []
158     kimi_id = "kimi-k3"
159     for indicator in indicators:
160         ordered = sorted(normalized[indicator], key=lambda model: (-normalized[indicator][
model], model))
161         leader = ordered[0]
162         kimi_rank = ordered.index(kimi_id) + 1
163         kimi_rows.append({
164             "indicator_key": indicator,
165             "dimension": dimensions[indicator],
166             "indicator": indicator_names[indicator],
167             "kimi_normalized_score": f"{normalized[indicator][kimi_id]:.10f}",
168             "kimi_rank": str(kimi_rank),
169             "models_compared": str(len(ordered)),
170             "leader_model_id": leader,
171             "gap_to_leader": f"{normalized[indicator][leader] - normalized[indicator][
kimi_id]:.10f}",
172             "assessment": "strength" if kimi_rank <= 2 else "weakness" if kimi_rank >= len(
ordered) - 1 else "middle",
173         })
174     write_rows(
175         output_dir / "kimi_strengths_weaknesses.csv",
176         [
177             "indicator_key", "dimension", "indicator", "kimi_normalized_score", "kimi_rank
",
178             "models_compared", "leader_model_id", "gap_to_leader", "assessment",
179         ],

```

```

180         kimi_rows,
181     )
182     return {
183         "weights": weights,
184         "ranked_models": len(complete),
185         "unranked_models": len(models) - len(complete),
186         "winner": ranked[0][0],
187     }
188
189
190 def main() -> int:
191     parser = argparse.ArgumentParser(description=__doc__)
192     parser.add_argument("--output-dir", type=Path, default=DEFAULT_OUTPUT)
193     args = parser.parse_args()
194     try:
195         result = compute(args.output_dir)
196     except (KeyError, ValueError) as exc:
197         print(f"ERROR: {exc}")
198         return 1
199     print(
200         f"Phase 4 complete: {result['ranked_models']} complete models ranked, "
201         f"{result['unranked_models']} unranked without imputation; winner={result['winner']}"
202     )
203     return 0
204
205
206 if __name__ == "__main__":
207     raise SystemExit(main())

```

source/scripts/model_phase5_scenarios.py

```

1  """Build three scenario rankings from CRITIC and declared subjective weights."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  import math
8  from pathlib import Path
9
10 ROOT = Path(__file__).resolve().parents[1]
11 PRIORITIES = Path(__file__).with_name("phase5_scenario_priorities.csv")
12 DEFAULT_OUTPUT = ROOT / "results/phase5"
13
14
15 def read_rows(path: Path) -> list[dict[str, str]]:
16     with path.open(encoding="utf-8-sig", newline="") as handle:
17         return list(csv.DictReader(handle))
18
19
20 def write_rows(path: Path, fields: list[str], rows: list[dict[str, str]]) -> None:
21     path.parent.mkdir(parents=True, exist_ok=True)

```

```

22     with path.open("w", encoding="utf-8", newline="") as handle:
23         writer = csv.DictWriter(handle, fieldnames=fields)
24         writer.writeheader()
25         writer.writerows(rows)
26
27
28 def topsis(models: list[str], indicators: list[str], normalized: dict[tuple[str, str],
29     float], weights: dict[str, float]) -> dict[str, float]:
30     result = {}
31     for model in models:
32         positive = math.sqrt(sum((weights[key] * normalized[model, key] - weights[key]) **
33             2 for key in indicators))
34         negative = math.sqrt(sum((weights[key] * normalized[model, key]) ** 2 for key in
35             indicators))
36         result[model] = negative / (positive + negative)
37     return result
38
39 def compute(output_dir: Path, objective_share: float = 0.5) -> dict[str, object]:
40     if not 0 <= objective_share <= 1:
41         raise ValueError("objective_share must be in [0, 1]")
42     normalized_rows = read_rows(ROOT / "results/phase4/normalized_scores.csv")
43     critic_rows = read_rows(ROOT / "results/phase4/critic_weights.csv")
44     general_rows = read_rows(ROOT / "results/phase4/general_ranking.csv")
45     priority_rows = read_rows(PRIORITIES)
46     indicators = [row["indicator_key"] for row in critic_rows]
47     critic = {row["indicator_key"]: float(row["critic_weight"]) for row in critic_rows}
48     names = {row["model_id"]: row["model_name"] for row in normalized_rows}
49     general_rank = {row["model_id"]: row["rank"] for row in general_rows}
50     ranked_models = [row["model_id"] for row in general_rows if row["ranking_status"] == "
51     ranked_complete_case"]
52     normalized = {
53         (row["model_id"], row["indicator_key"]): float(row["minmax_score"])
54         for row in normalized_rows if row["minmax_score"] != "NA" and row["model_id"] in
55         ranked_models
56     }
57     scenarios = list(dict.fromkeys(row["scenario"] for row in priority_rows))
58     weight_output, ranking_output = [], []
59     winners = {}
60     for scenario in scenarios:
61         subjective = {row["indicator_key"]: float(row["subjective_weight"]) for row in
62             priority_rows if row["scenario"] == scenario}
63         if set(subjective) != set(indicators) or abs(sum(subjective.values()) - 1) > 1e-9:
64             raise ValueError(f"Scenario {scenario} weights must cover indicators and sum to
65             1")
66         combined = {key: objective_share * critic[key] + (1 - objective_share) * subjective
67             [key] for key in indicators}
68         for key in indicators:
69             weight_output.append({
70                 "scenario": scenario,
71                 "indicator_key": key,
72                 "critic_weight": f"{critic[key]:.10f}",
73                 "subjective_weight": f"{subjective[key]:.10f}",
74                 "objective_share": f"{objective_share:.2f}",

```

```

68         "combined_weight": f"{combined[key]:.10f}",
69     })
70     scores = topsis(ranked_models, indicators, normalized, combined)
71     ordered = sorted(ranked_models, key=lambda model: (-scores[model], model))
72     winners[scenario] = ordered[0]
73     for rank, model in enumerate(ordered, start=1):
74         ranking_output.append({
75             "scenario": scenario,
76             "model_id": model,
77             "model_name": names[model],
78             "scenario_score": f"{scores[model]:.10f}",
79             "scenario_rank": str(rank),
80             "general_rank": general_rank[model],
81             "rank_change_vs_general": str(int(general_rank[model]) - rank),
82             "ranking_status": "ranked_complete_case",
83         })
84     for model in names:
85         if model not in ranked_models:
86             ranking_output.append({
87                 "scenario": scenario, "model_id": model, "model_name": names[model],
88                 "scenario_score": "NA", "scenario_rank": "NA", "general_rank": "NA",
89                 "rank_change_vs_general": "NA", "ranking_status": "
not_ranked_insufficient_coverage",
90             })
91     write_rows(
92         output_dir / "scenario_weights.csv",
93         ["scenario", "indicator_key", "critic_weight", "subjective_weight", "
objective_share", "combined_weight"],
94         weight_output,
95     )
96     write_rows(
97         output_dir / "scenario_rankings.csv",
98         ["scenario", "model_id", "model_name", "scenario_score", "scenario_rank", "
general_rank", "rank_change_vs_general", "ranking_status"],
99         ranking_output,
100     )
101     return {"scenarios": len(scenarios), "ranked_models": len(ranked_models), "winners":
winners}
102
103
104 def main() -> int:
105     parser = argparse.ArgumentParser(description=__doc__)
106     parser.add_argument("--output-dir", type=Path, default=DEFAULT_OUTPUT)
107     parser.add_argument("--objective-share", type=float, default=0.5)
108     args = parser.parse_args()
109     try:
110         result = compute(args.output_dir, args.objective_share)
111     except (KeyError, ValueError) as exc:
112         print(f"ERROR: {exc}")
113         return 1
114     print(f"Phase 5 complete: {result['scenarios']} scenarios, winners={result['winners
']}.".")
115     return 0
116

```

```

117
118 if __name__ == "__main__":
119     raise SystemExit(main())

```

source/scripts/model_phase6_cost_benefit.py

```

1  """Evaluate performance, API cost, engineering efficiency, and budget choices."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  from pathlib import Path
8
9  try:
10     from scripts.model_phase4_critic_topsis import minmax
11 except ModuleNotFoundError:
12     from model_phase4_critic_topsis import minmax
13
14 ROOT = Path(__file__).resolve().parents[1]
15 DEFAULT_OUTPUT = ROOT / "results/phase6"
16 NA = "NA"
17 INPUT_MILLION_TOKENS = 1.0
18 OUTPUT_MILLION_TOKENS = 0.2
19 BUDGETS_USD = (6.0, 10.0, 12.0, 16.0, 20.0)
20
21
22 def read_rows(path: Path) -> list[dict[str, str]]:
23     with path.open(encoding="utf-8-sig", newline="") as handle:
24         return list(csv.DictReader(handle))
25
26
27 def write_rows(path: Path, fields: list[str], rows: list[dict[str, str]]) -> None:
28     path.parent.mkdir(parents=True, exist_ok=True)
29     with path.open("w", encoding="utf-8", newline="") as handle:
30         writer = csv.DictWriter(handle, fieldnames=fields)
31         writer.writeheader()
32         writer.writerows(rows)
33
34
35 def compute(output_dir: Path) -> dict[str, object]:
36     attributes = read_rows(ROOT / "data/processed/model_attributes.csv")
37     ranking = read_rows(ROOT / "results/phase4/general_ranking.csv")
38     rank_by_id = {row["model_id"]: row for row in ranking}
39     model_names = {row["model_id"]: row["model_name"] for row in attributes}
40     costs = {
41         row["model_id"]: float(row["input_price_usd_per_million"]) * INPUT_MILLION_TOKENS
42         + float(row["output_price_usd_per_million"]) * OUTPUT_MILLION_TOKENS
43         for row in attributes
44         if row["input_price_usd_per_million"] != NA and row["output_price_usd_per_million"]
45         != NA
46     }
47     performance = {

```

```

47     row["model_id"]: float(row["topsis_score"])
48     for row in ranking if row["topsis_score"] != NA
49 }
50 ranked_ids = list(performance)
51 frontier = set()
52 for model in ranked_ids:
53     dominated = any(
54         other != model
55         and costs[other] <= costs[model]
56         and performance[other] >= performance[model]
57         and (costs[other] < costs[model] or performance[other] > performance[model])
58         for other in ranked_ids
59     )
60     if not dominated:
61         frontier.add(model)
62 value_rows = []
63 for row in attributes:
64     model = row["model_id"]
65     score = performance.get(model)
66     value_rows.append({
67         "model_id": model,
68         "model_name": row["model_name"],
69         "workload_input_million_tokens": f"{INPUT_MILLION_TOKENS:.1f}",
70         "workload_output_million_tokens": f"{OUTPUT_MILLION_TOKENS:.1f}",
71         "standardized_api_cost_usd": f"{costs[model]:.4f}",
72         "general_performance_score": f"{score:.10f}" if score is not None else NA,
73         "performance_per_usd": f"{score / costs[model]:.10f}" if score is not None else
NA,
74         "pareto_frontier": str(model in frontier).lower() if score is not None else NA,
75         "analysis_status": "included" if score is not None else "
excluded_missing_performance_rank",
76     })
77 write_rows(
78     output_dir / "performance_cost.csv",
79     [
80         "model_id", "model_name", "workload_input_million_tokens", "
workload_output_million_tokens",
81         "standardized_api_cost_usd", "general_performance_score", "performance_per_usd
",
82         "pareto_frontier", "analysis_status",
83     ],
84     value_rows,
85 )
86
87 budget_rows = []
88 for budget in BUDGETS_USD:
89     feasible = [model for model in ranked_ids if costs[model] <= budget]
90     winner = max(feasible, key=lambda model: (performance[model], -costs[model])) if
feasible else None
91     budget_rows.append({
92         "budget_usd_per_standardized_workload": f"{budget:.2f}",
93         "feasible_ranked_models": str(len(feasible)),
94         "recommended_model_id": winner or NA,
95         "recommended_model_name": model_names[winner] if winner else NA,

```

```

96         "performance_score": f"{performance[winner]:.10f}" if winner else NA,
97         "cost_usd": f"{costs[winner]:.4f}" if winner else NA,
98         "rule": "highest general TOPSIS score within budget; no missing-score model
considered",
99     })
100 write_rows(
101     output_dir / "budget_recommendations.csv",
102     [
103         "budget_usd_per_standardized_workload", "feasible_ranked_models", "
recommended_model_id",
104         "recommended_model_name", "performance_score", "cost_usd", "rule",
105     ],
106     budget_rows,
107 )
108
109 compatible = [row for row in attributes if row["efficiency_compatible"] == "true" and
row["model_id"] in performance]
110 ttft = {row["model_id"]: float(row["comparable_ttft_seconds"]) for row in compatible}
111 speed = {row["model_id"]: float(row["comparable_output_speed_tokens_per_second"]) for
row in compatible}
112 latency = {row["model_id"]: float(row["comparable_total_latency_seconds"]) for row in
compatible}
113 ttft_score, speed_score, latency_score = minmax(ttft, False), minmax(speed, True),
minmax(latency, False)
114 engineering_rows = []
115 for row in attributes:
116     model = row["model_id"]
117     if model in ttft_score:
118         efficiency_score = (ttft_score[model] + speed_score[model] + latency_score[
model]) / 3
119         deployment_score = 0.7 * performance[model] + 0.3 * efficiency_score
120         engineering_rows.append({
121             "model_id": model, "model_name": row["model_name"],
122             "ttft_benefit_score": f"{ttft_score[model]:.10f}",
123             "speed_benefit_score": f"{speed_score[model]:.10f}",
124             "latency_benefit_score": f"{latency_score[model]:.10f}",
125             "engineering_efficiency_score": f"{efficiency_score:.10f}",
126             "performance_score": f"{performance[model]:.10f}",
127             "deployment_score_70p_performance_30p_efficiency": f"{deployment_score:.10f
}",
128             "analysis_status": "included_compatible_efficiency",
129         })
130     else:
131         engineering_rows.append({
132             "model_id": model, "model_name": row["model_name"],
133             "ttft_benefit_score": NA, "speed_benefit_score": NA, "latency_benefit_score
": NA,
134             "engineering_efficiency_score": NA, "performance_score": f"{performance[
model]:.10f}" if model in performance else NA,
135             "deployment_score_70p_performance_30p_efficiency": NA,
136             "analysis_status": "excluded_incompatible_efficiency" if model in
performance else "excluded_missing_performance_and_efficiency",
137         })
138 write_rows(

```

```

139         output_dir / "engineering_efficiency.csv",
140         [
141             "model_id", "model_name", "ttft_benefit_score", "speed_benefit_score", "
latency_benefit_score",
142             "engineering_efficiency_score", "performance_score", "
deployment_score_70p_performance_30p_efficiency",
143             "analysis_status",
144         ],
145         engineering_rows,
146     )
147     return {"frontier": sorted(frontier), "budget_rows": len(budget_rows), "
efficiency_models": len(compatible)}
148
149
150 def main() -> int:
151     parser = argparse.ArgumentParser(description=__doc__)
152     parser.add_argument("--output-dir", type=Path, default=DEFAULT_OUTPUT)
153     args = parser.parse_args()
154     try:
155         result = compute(args.output_dir)
156     except (KeyError, ValueError) as exc:
157         print(f"ERROR: {exc}")
158         return 1
159     print(f"Phase 6 complete: Pareto frontier={result['frontier']}; compatible efficiency
models={result['efficiency_models']}")
160     return 0
161
162
163 if __name__ == "__main__":
164     raise SystemExit(main())

```

source/scripts/process_phase2_data.py

```

1  """Build deterministic Phase 2 cleaned datasets without changing raw files."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  from collections import Counter
8  from pathlib import Path
9
10 ROOT = Path(__file__).resolve().parents[1]
11 RAW = ROOT / "data/raw"
12 DEFAULT_OUTPUT = ROOT / "data/processed"
13 COHORT_CONFIG = Path(__file__).with_name("phase2_core_cohorts.csv")
14 NA = "NA"
15
16
17 def read_rows(path: Path) -> list[dict[str, str]]:
18     with path.open(encoding="utf-8-sig", newline="") as handle:
19         return list(csv.DictReader(handle))
20

```

```

21
22 def write_rows(path: Path, fields: list[str], rows: list[dict[str, str]]) -> None:
23     path.parent.mkdir(parents=True, exist_ok=True)
24     with path.open("w", encoding="utf-8", newline="") as handle:
25         writer = csv.DictWriter(handle, fieldnames=fields)
26         writer.writeheader()
27         writer.writerows(rows)
28
29
30 def normalized(value: str | None) -> str:
31     value = (value or "").strip()
32     return value if value else NA
33
34
35 def build(output_dir: Path) -> dict[str, int]:
36     candidates = read_rows(ROOT / "data/model_candidates.csv")
37     benchmarks = read_rows(RAW / "benchmark_scores.csv")
38     metadata = read_rows(RAW / "model_metadata.csv")
39     efficiency = read_rows(RAW / "cost_efficiency.csv")
40     cohorts = read_rows(COHORT_CONFIG)
41
42     final = [row for row in candidates if row["candidate_status"] == "final"]
43     if not 6 <= len(final) <= 8:
44         raise ValueError(f"Expected 6-8 final models, found {len(final)}")
45     final_ids = [row["model_id"] for row in final]
46     metadata_by_id = {row["model_id"]: row for row in metadata}
47     efficiency_by_id = {row["model_id"]: row for row in efficiency}
48     if len(metadata_by_id) != len(metadata) or len(efficiency_by_id) != len(efficiency):
49         raise ValueError("Metadata and efficiency tables must contain one row per model")
50
51     long_rows: list[dict[str, str]] = []
52     matrix_rows = {
53         row["model_id"]: {
54             "model_id": row["model_id"],
55             "provider": row["provider"],
56             "model_name": row["model_name"],
57             "exact_version": row["exact_version"],
58         }
59         for row in final
60     }
61     score_units: dict[str, str] = {}
62     indicator_quality: list[dict[str, str]] = []
63     for cohort in cohorts:
64         matches = [
65             row for row in benchmarks
66             if row["benchmark"] == cohort["raw_benchmark"]
67             and row["benchmark_version"] == cohort["benchmark_version"]
68             and row["test_setting"] == cohort["test_setting"]
69             and row["source_name"] == cohort["source_name"]
70             and row["compatible"] == "true"
71             and row["model_id"] in final_ids
72         ]
73         counts = Counter(row["model_id"] for row in matches)
74         if any(counts[model_id] != 1 for model_id in final_ids):

```

```

75         raise ValueError(f"Cohort {cohort['indicator_key']} does not have exactly one
row per final model")
76     units = {row["score_unit"] for row in matches if row["score"] != NA}
77     if len(units) != 1:
78         raise ValueError(f"Cohort {cohort['indicator_key']} mixes score units")
79     score_units[cohort["indicator_key"]] = units.pop()
80     for row in matches:
81         score = normalized(row["score"])
82         matrix_rows[row["model_id"]][cohort["indicator_key"]] = score
83         long_rows.append({
84             "model_id": row["model_id"],
85             "model_name": row["model_name"],
86             "indicator_key": cohort["indicator_key"],
87             "dimension": cohort["dimension"],
88             "indicator": cohort["indicator"],
89             "score": score,
90             "score_unit": row["score_unit"],
91             "higher_is_better": cohort["higher_is_better"],
92             "benchmark_version": row["benchmark_version"],
93             "test_setting": row["test_setting"],
94             "source_name": row["source_name"],
95             "source_url": row["source_url"],
96             "retrieval_date": row["retrieval_date"],
97             "missing_reason": (
98                 "not applicable: frozen model is text-only"
99                 if score == NA
100                 and row["model_id"] == "glm-5.2"
101                 and cohort["dimension"] in {
102                     "multimodal", "document_understanding",
103                     "research_document_reasoning", "multimodal_math",
104                 }
105                 else "not reported in frozen cohort" if score == NA else NA
106             ),
107         })
108     present_ids = sorted(row["model_id"] for row in matches if row["score"] != NA)
109     missing_ids = sorted(set(final_ids) - set(present_ids))
110     indicator_quality.append({
111         "indicator_key": cohort["indicator_key"],
112         "dimension": cohort["dimension"],
113         "indicator": cohort["indicator"],
114         "score_unit": score_units[cohort["indicator_key"]],
115         "higher_is_better": cohort["higher_is_better"],
116         "available_models": str(len(present_ids)),
117         "final_models": str(len(final_ids)),
118         "coverage_percent": f"{len(present_ids) / len(final_ids) * 100:.1f}",
119         "missing_model_ids": ";".join(missing_ids) if missing_ids else NA,
120         "cohort_status": "pass" if len(present_ids) / len(final_ids) >= 0.75 else "fail
",
121     })
122
123     matrix_fields = ["model_id", "provider", "model_name", "exact_version"] + [
124         row["indicator_key"] for row in cohorts
125     ]
126     write_rows(output_dir / "core_benchmark_matrix.csv", matrix_fields, list(matrix_rows.

```

```

values()))
127 long_fields = [
128     "model_id", "model_name", "indicator_key", "dimension", "indicator", "score",
129     "score_unit", "higher_is_better", "benchmark_version", "test_setting", "source_name",
130     "source_url", "retrieval_date", "missing_reason",
131 ]
132 write_rows(output_dir / "core_benchmark_long.csv", long_fields, long_rows)
133 write_rows(
134     output_dir / "indicator_quality.csv",
135     [
136         "indicator_key", "dimension", "indicator", "score_unit", "higher_is_better",
137         "available_models", "final_models", "coverage_percent", "missing_model_ids", "
cohort_status",
138     ],
139     indicator_quality,
140 )
141
142 attribute_rows: list[dict[str, str]] = []
143 metadata_fields = [
144     "context_window", "max_output_tokens", "vision_support", "reasoning_support",
145     "api_available", "input_price_usd_per_million", "output_price_usd_per_million",
146     "cached_input_price_usd_per_million", "batch_input_price", "batch_output_price",
147     "long_context_price", "peak_price", "off_peak_price", "pricing_effective_date",
148 ]
149 for candidate in final:
150     model_id = candidate["model_id"]
151     meta = metadata_by_id[model_id]
152     eff = efficiency_by_id[model_id]
153     for field in ("model_name", "provider", "exact_version"):
154         if meta[field] != candidate[field]:
155             raise ValueError(f"Identity mismatch for {model_id}: {field}")
156     compatible = eff["compatible"] == "true"
157     result = {
158         "model_id": model_id,
159         "provider": candidate["provider"],
160         "model_name": candidate["model_name"],
161         "exact_version": candidate["exact_version"],
162     }
163     result.update({field: normalized(meta.get(field)) for field in metadata_fields})
164     result.update({
165         "efficiency_compatible": eff["compatible"],
166         "observed_ttft_seconds": normalized(eff.get("ttft_seconds")),
167         "observed_output_speed_tokens_per_second": normalized(eff.get("
output_speed_tokens_per_second")),
168         "observed_total_latency_seconds": normalized(eff.get("total_latency_seconds")),
169         "comparable_ttft_seconds": normalized(eff.get("ttft_seconds")) if compatible
else NA,
170         "comparable_output_speed_tokens_per_second": normalized(eff.get("
output_speed_tokens_per_second")) if compatible else NA,
171         "comparable_total_latency_seconds": normalized(eff.get("total_latency_seconds"))
if compatible else NA,
172         "efficiency_test_setting": eff["test_setting"],
173         "efficiency_source_url": eff["source_url"],

```

```

174         "efficiency_exclusion_reason": NA if compatible else eff["notes"],
175     })
176     attribute_rows.append(result)
177     attribute_fields = list(attribute_rows[0])
178     write_rows(output_dir / "model_attributes.csv", attribute_fields, attribute_rows)
179
180     quality_rows = [
181         {"check": "final_model_count", "value": str(len(final)), "status": "pass", "notes":
182         "Target range: 6-8"},
183         {"check": "core_indicator_count", "value": str(len(cohorts)), "status": "pass", "
184         notes": "Target range: 8-12"},
185         {"check": "core_matrix_rows", "value": str(len(matrix_rows)), "status": "pass", "
186         notes": "One row per final model"},
187         {"check": "core_long_rows", "value": str(len(long_rows)), "status": "pass", "notes
188         ": "Final models x core indicators"},
189         {"check": "core_missing_scores", "value": str(sum(row["score"] == NA for row in
190         long_rows)), "status": "pass", "notes": "Preserved as literal NA; no imputation"},
191         {"check": "core_cohorts_at_least_75_percent", "value": str(sum(row["cohort_status"]
192         == "pass" for row in indicator_quality)), "status": "pass", "notes": "All frozen core
193         cohorts pass"},
194         {"check": "strict_efficiency_models", "value": str(sum(row["efficiency_compatible"]
195         == "true" for row in attribute_rows)), "status": "pass", "notes": "Incompatible
196         observations retained separately"},
197         {"check": "raw_files_modified", "value": "0", "status": "pass", "notes": "Processor
198         is read-only for data/raw"},
199     ]
200     write_rows(output_dir / "phase2_quality_report.csv", ["check", "value", "status", "
201     notes"], quality_rows)
202     return {"models": len(final), "indicators": len(cohorts), "long_rows": len(long_rows)}
203
204 def main() -> int:
205     parser = argparse.ArgumentParser(description=__doc__)
206     parser.add_argument("--output-dir", type=Path, default=DEFAULT_OUTPUT)
207     args = parser.parse_args()
208     try:
209         counts = build(args.output_dir)
210     except (KeyError, ValueError) as exc:
211         print(f"ERROR: {exc}")
212         return 1
213     print(
214         f"Phase 2 processed data written: {counts['models']} final models, "
215         f"{counts['indicators']} core indicators, {counts['long_rows']} long rows; no
216         imputation."
217     )
218     return 0
219
220 if __name__ == "__main__":
221     raise SystemExit(main())

```

source/scripts/run_glm_supplement.py

```

1  """Generate the GLM-5.2 supplemental Q1 analysis required by model(1).md."""
2
3  from __future__ import annotations
4
5  import argparse
6  import sys
7  from pathlib import Path
8
9  ROOT = Path(__file__).resolve().parents[1]
10 if str(ROOT) not in sys.path:
11     sys.path.insert(0, str(ROOT))
12
13 from src.q1.glm_supplement import generate_figure, run_analysis
14
15
16 def main() -> int:
17     parser = argparse.ArgumentParser(description=__doc__)
18     parser.add_argument("--output-dir", type=Path, default=ROOT / "results/phase4b")
19     args = parser.parse_args()
20     try:
21         result = run_analysis(ROOT, args.output_dir)
22         figure = generate_figure(result, args.output_dir)
23     except (AssertionError, FileNotFoundError, KeyError, ValueError) as exc:
24         print(f"ERROR: {exc}")
25         return 1
26     print(f"GLM supplement complete: {len(result['comparison'])} directly comparable
27         metrics; figure={figure.name}.")
28     return 0
29
30 if __name__ == "__main__":
31     raise SystemExit(main())

```

source/scripts/run_pipeline.py

```

1  """Rebuild and validate all committed datasets, models, results, and figures."""
2
3  from __future__ import annotations
4
5  import subprocess
6  import sys
7  from pathlib import Path
8
9  ROOT = Path(__file__).resolve().parents[1]
10
11 COMMANDS = [
12     ["scripts/process_phase2_data.py"],
13     ["scripts/merge_data.py", "--final-only", "--force"],
14     ["scripts/validate_data.py"],
15     ["scripts/check_coverage.py", "--output", "results/benchmark_coverage.csv"],
16     ["scripts/analyze_phase3_indicators.py"],
17     ["scripts/model_phase4_critic_topsis.py"],

```

```

18     ["scripts/run_glm_supplement.py"],
19     ["scripts/model_phase5_scenarios.py"],
20     ["scripts/model_phase6_cost_benefit.py"],
21     ["scripts/analyze_phase7_robustness.py"],
22     ["scripts/generate_figures.py"],
23     ["scripts/validate_modeling.py"],
24 ]
25
26
27 def main() -> int:
28     for arguments in COMMANDS:
29         print(f"\n==> {sys.executable} {' '.join(arguments)}", flush=True)
30         completed = subprocess.run([sys.executable, *arguments], cwd=ROOT, check=False)
31         if completed.returncode:
32             print(f"Pipeline stopped with exit code {completed.returncode}: {' '.join(
33                 arguments)}")
34             return completed.returncode
35         print("\nPipeline completed successfully.")
36         return 0
37
38 if __name__ == "__main__":
39     raise SystemExit(main())

```

source/scripts/run_q1.py

```

1  """Run the complete reproducible Question 1 modeling workflow."""
2
3  from __future__ import annotations
4
5  import argparse
6  import sys
7  from pathlib import Path
8
9  ROOT = Path(__file__).resolve().parents[1]
10 if str(ROOT) not in sys.path:
11     sys.path.insert(0, str(ROOT))
12
13 from src.q1.analysis import run_analysis
14 from src.q1.reporting import generate_documents
15 from src.q1.visualization import generate_figures
16 from scripts.validate_q1 import CSV_FILES, validate
17
18
19 def main() -> int:
20     parser = argparse.ArgumentParser(description=__doc__)
21     parser.add_argument("--output-dir", type=Path, default=ROOT / "results/q1")
22     parser.add_argument("--docs-dir", type=Path, default=ROOT / "docs")
23     args = parser.parse_args()
24     try:
25         result = run_analysis(ROOT, args.output_dir)
26         figures = generate_figures(result, args.output_dir)
27         documents = generate_documents(result, args.docs_dir)

```

```

28     issues = validate(args.output_dir, args.docs_dir)
29     if issues:
30         raise AssertionError("; ".join(issues))
31 except (AssertionError, FileNotFoundError, KeyError, ValueError) as exc:
32     print(f"ERROR: {exc}")
33     return 1
34 ranked = result["ranking"]
35 winner = ranked.loc[ranked["rank"] == 1, "model"].iloc[0]
36 kimi_rank = int(ranked.loc[ranked["model_id"] == "kimi-k3", "rank"].iloc[0])
37 print(
38     f"Question 1 complete: {len(result['models'])} models, {len(result['metrics'])}
39     indicators, "
40     f"winner={winner}, Kimi rank={kimi_rank}; generated {len(CSV_FILES)} CSVs, "
41     f"{len(figures)} PNGs and {len(documents)} documents."
42 )
43 return 0
44
45 if __name__ == "__main__":
46     raise SystemExit(main())

```

source/scripts/run_q2.py

```

1  """Run the complete Question 2 scenario-evaluation workflow."""
2
3  from __future__ import annotations
4
5  import argparse
6  import sys
7  from pathlib import Path
8
9  ROOT = Path(__file__).resolve().parents[1]
10 if str(ROOT) not in sys.path:
11     sys.path.insert(0, str(ROOT))
12
13 from src.q2.analysis import run_analysis
14 from src.q2.reporting import generate_report
15 from src.q2.visualization import generate_figures
16
17
18 def main() -> int:
19     parser = argparse.ArgumentParser(description=__doc__)
20     parser.add_argument("--output-dir", type=Path, default=ROOT / "results/q2")
21     parser.add_argument("--docs-dir", type=Path, default=ROOT / "docs")
22     args = parser.parse_args()
23     try:
24         result = run_analysis(ROOT, args.output_dir)
25         figures = generate_figures(result, args.output_dir)
26         report = generate_report(result, args.docs_dir)
27     except (AssertionError, FileNotFoundError, KeyError, ValueError) as exc:
28         print(f"ERROR: {exc}")
29         return 1
30     rankings = result["scenario_rankings"]

```

```

31 winners = rankings.loc[rankings["rank"] == 1].set_index("scenario")["model"].to_dict()
32 print(f"Question 2 complete: winners={winners}; generated 9 CSVs, {len(figures)} PNGs
    and {report.name}.")
33 return 0
34
35
36 if __name__ == "__main__":
37     raise SystemExit(main())

```

source/scripts/run_q3.py

```

1  """Run the complete Question 3 performance-cost workflow."""
2
3  from __future__ import annotations
4
5  import argparse
6  import sys
7  from pathlib import Path
8
9  ROOT = Path(__file__).resolve().parents[1]
10 if str(ROOT) not in sys.path:
11     sys.path.insert(0, str(ROOT))
12
13 from src.q3.analysis import run_analysis
14 from src.q3.reporting import generate_mapping, generate_report
15 from src.q3.visualization import generate_figures
16
17
18 def main() -> int:
19     parser = argparse.ArgumentParser(description=__doc__)
20     parser.add_argument("--output-dir", type=Path, default=ROOT / "results/q3")
21     parser.add_argument("--q2-dir", type=Path, default=ROOT / "results/q2")
22     parser.add_argument("--docs-dir", type=Path, default=ROOT / "docs")
23     args = parser.parse_args()
24     try:
25         result = run_analysis(ROOT, args.output_dir, args.q2_dir)
26         figures = generate_figures(result, args.output_dir)
27         report = generate_report(result, args.docs_dir)
28         mapping = generate_mapping(args.docs_dir)
29     except (AssertionError, FileNotFoundError, KeyError, ValueError) as exc:
30         print(f"ERROR: {exc}")
31         return 1
32     frontier = result["pareto"].loc[result["pareto"]["is_pareto_frontier"], "model"].tolist
33     ()
34     print(f"Question 3 complete: Pareto frontier={frontier}; generated 8 CSVs, {len(figures
35     )} PNGs, {report.name}, {mapping.name} and final_model_summary.csv.")
36     return 0
37
38 if __name__ == "__main__":
39     raise SystemExit(main())

```

source/scripts/validate_data.py

```

1  """Validate raw CSV files and report issues without modifying data."""
2
3  from __future__ import annotations
4
5  import csv
6  import math
7  from collections import Counter, defaultdict
8  from datetime import date
9  from pathlib import Path
10
11  ROOT = Path(__file__).resolve().parents[1]
12  NA_VALUES = {"", "na", "n/a", "null", "none"}
13  DATA_CUTOFF_DATE = date.fromisoformat("2026-08-17")
14
15  FILES = {
16      "candidates": ROOT / "data/model_candidates.csv",
17      "benchmark": ROOT / "data/raw/benchmark_scores.csv",
18      "metadata": ROOT / "data/raw/model_metadata.csv",
19      "efficiency": ROOT / "data/raw/cost_efficiency.csv",
20  }
21  SOURCE_FILES = {
22      "benchmark_sources": ROOT / "data/sources/benchmark_sources.csv",
23      "metadata_sources": ROOT / "data/sources/metadata_sources.csv",
24      "efficiency_sources": ROOT / "data/sources/efficiency_sources.csv",
25  }
26  CORE_SELECTION = ROOT / "results/core_indicator_selection.csv"
27  PROCESSED_FILES = {
28      "core_matrix": ROOT / "data/processed/core_benchmark_matrix.csv",
29      "core_long": ROOT / "data/processed/core_benchmark_long.csv",
30      "model_attributes": ROOT / "data/processed/model_attributes.csv",
31      "indicator_quality": ROOT / "data/processed/indicator_quality.csv",
32      "quality_report": ROOT / "data/processed/phase2_quality_report.csv",
33  }
34
35
36  def missing(value: str | None) -> bool:
37      return value is None or value.strip().lower() in NA_VALUES
38
39
40  def read_csv(path: Path) -> tuple[list[str], list[dict[str, str]]]:
41      with path.open(encoding="utf-8-sig", newline="") as handle:
42          reader = csv.DictReader(handle)
43          return reader.fieldnames or [], list(reader)
44
45
46  def numeric_or_na(value: str | None) -> bool:
47      if value is None or not value.strip():
48          return False
49      if value.strip() == "NA":
50          return True
51      try:
52          return math.isfinite(float(value))

```

```

53     except (TypeError, ValueError):
54         return False
55
56
57 def validate_date(value: str | None) -> bool:
58     if missing(value):
59         return False
60     try:
61         return date.fromisoformat(value.strip()) <= DATA_CUTOFF_DATE
62     except ValueError:
63         return False
64
65
66 def date_or_na(value: str | None) -> bool:
67     return value == "NA" or validate_date(value)
68
69
70 def main() -> int:
71     issues: list[str] = []
72     for label, path in (FILES | SOURCE_FILES | PROCESSED_FILES).items():
73         if not path.exists():
74             issues.append(f"{label}: missing file {path}")
75     if not CORE_SELECTION.exists():
76         issues.append(f"core_selection: missing file {CORE_SELECTION}")
77     if issues:
78         print("\n".join(f"ERROR: {item}" for item in issues))
79         return 1
80
81     _, candidates = read_csv(FILES["candidates"])
82     _, benchmarks = read_csv(FILES["benchmark"])
83     _, metadata = read_csv(FILES["metadata"])
84     _, efficiency = read_csv(FILES["efficiency"])
85     source_tables = {label: read_csv(path)[1] for label, path in SOURCE_FILES.items()}
86     _, core_selection = read_csv(CORE_SELECTION)
87     processed = {label: read_csv(path)[1] for label, path in PROCESSED_FILES.items()}
88
89     candidate_ids = {r.get("model_id", "").strip() for r in candidates if not missing(r.get("model_id"))}
90     if len(candidate_ids) != len([r for r in candidates if not missing(r.get("model_id"))]):
91         :
92         issues.append("candidates: duplicate model_id")
93     allowed_statuses = {"pending", "likely", "excluded", "final"}
94     for line, row in enumerate(candidates, start=2):
95         if row.get("candidate_status") not in allowed_statuses:
96             issues.append(f"candidates:{line}: invalid candidate_status")
97         if not validate_date(row.get("release_date")):
98             issues.append(f"candidates:{line}: invalid/post-cutoff release_date")
99     final_ids = {r["model_id"] for r in candidates if r.get("candidate_status") == "final"}
100     if not 6 <= len(final_ids) <= 8:
101         issues.append(f"candidates: final model count must be 6-8, got {len(final_ids)}")
102     if "kimi-k3" not in final_ids:
103         issues.append("candidates: kimi-k3 must be final")
104
105     versions: dict[str, set[str]] = defaultdict(set)

```

```

105 for label, rows in (("benchmark", benchmarks), ("metadata", metadata), ("efficiency",
106 efficiency)):
107     for line, row in enumerate(rows, start=2):
108         model_id = row.get("model_id", "").strip()
109         if missing(model_id) or model_id not in candidate_ids:
110             issues.append(f"{label}:{line}: model_id is missing or absent from
111 candidates: {model_id or 'NA'}")
112         model_name = row.get("model_name", "").strip()
113         version = row.get("exact_version", "").strip()
114         if model_name and version and not missing(version):
115             versions[model_name].add(version)
116
117 benchmark_keys = []
118 for line, row in enumerate(benchmarks, start=2):
119     if not numeric_or_na(row.get("score")):
120         issues.append(f"benchmark:{line}: score must be numeric or NA")
121     for field in ("source_url", "retrieval_date", "benchmark_version"):
122         if missing(row.get(field)):
123             issues.append(f"benchmark:{line}: {field} is required")
124     if not missing(row.get("retrieval_date")) and not validate_date(row.get("
125 retrieval_date")):
126         issues.append(f"benchmark:{line}: invalid/post-cutoff retrieval_date")
127     benchmark_keys.append((row.get("model_id"), row.get("benchmark"), row.get("
128 benchmark_version"), row.get("test_setting"), row.get("source_url")))
129
130 price_fields = ("input_price_usd_per_million", "output_price_usd_per_million", "
131 cached_input_price_usd_per_million", "batch_input_price", "batch_output_price", "
132 long_context_price", "peak_price", "off_peak_price")
133 metadata_keys = []
134 for line, row in enumerate(metadata, start=2):
135     for field in price_fields + ("context_window", "max_output_tokens"):
136         if not numeric_or_na(row.get(field)):
137             issues.append(f"metadata:{line}: {field} must be numeric or NA")
138     for field in ("source_url", "retrieval_date", "exact_version"):
139         if missing(row.get(field)):
140             issues.append(f"metadata:{line}: {field} is required")
141     if any(not missing(row.get(field)) for field in price_fields) and missing(row.get("
142 pricing_effective_date")):
143         issues.append(f"metadata:{line}: pricing_effective_date required when a price
144 exists")
145     for field in ("retrieval_date", "pricing_effective_date"):
146         if not date_or_na(row.get(field)):
147             issues.append(f"metadata:{line}: invalid/post-cutoff {field}")
148     metadata_keys.append((row.get("model_id"), row.get("exact_version"), row.get("
149 source_url")))
150
151 metadata_by_id = {row["model_id"]: row for row in metadata}
152 required_frozen_metadata = {
153     "gpt-5.6-sol": {
154         "input_price_usd_per_million": "5.0",
155         "output_price_usd_per_million": "30.0",
156         "long_context_price": "10.0",
157     },
158     "kimi-k3": {"max_output_tokens": "1048576"},

```

```

150     "claude-fable-5": {
151         "input_price_usd_per_million": "10.0",
152         "output_price_usd_per_million": "50.0",
153         "cached_input_price_usd_per_million": "1.0",
154         "batch_input_price": "5.0",
155         "batch_output_price": "25.0",
156     },
157     "glm-5.2": {"vision_support": "no", "max_output_tokens": "131072"},
158 }
159 for model_id, expected_fields in required_frozen_metadata.items():
160     row = metadata_by_id.get(model_id, {})
161     for field, expected in expected_fields.items():
162         if row.get(field) != expected:
163             issues.append(f"metadata: {model_id}.{field} must equal frozen value {
expected}")
164
165 required_glm_scores = {
166     ("HLE-Full (no tools)", "Z.ai GLM-5.2 official launch report"): "40.5",
167     ("HLE-Full (with tools)", "Z.ai GLM-5.2 official launch report"): "54.7",
168     ("GPQA Diamond", "Z.ai GLM-5.2 official launch report"): "91.2",
169 }
170 for key, expected in required_glm_scores.items():
171     matches = [
172         row for row in benchmarks
173         if row.get("model_id") == "glm-5.2"
174         and (row.get("benchmark"), row.get("source_name")) == key
175     ]
176     if len(matches) != 1 or matches[0].get("score") != expected:
177         issues.append(f"benchmark: GLM-5.2 {key[0]} official supplemental score must
equal {expected}")
178
179 efficiency_keys = []
180 for line, row in enumerate(efficiency, start=2):
181     for field in ("ttft_seconds", "output_speed_tokens_per_second", "
total_latency_seconds"):
182         if not numeric_or_na(row.get(field)):
183             issues.append(f"efficiency:{line}: {field} must be numeric or NA")
184     for field in ("source_url", "retrieval_date"):
185         if missing(row.get(field)):
186             issues.append(f"efficiency:{line}: {field} is required")
187     if row.get("compatible") not in {"true", "false"}:
188         issues.append(f"efficiency:{line}: compatible must be true or false")
189     if not date_or_na(row.get("measurement_date")):
190         issues.append(f"efficiency:{line}: invalid/post-cutoff measurement_date")
191     if not validate_date(row.get("retrieval_date")):
192         issues.append(f"efficiency:{line}: invalid/post-cutoff retrieval_date")
193     efficiency_keys.append((row.get("model_id"), row.get("platform"), row.get("
measurement_date"), row.get("test_setting"), row.get("source_url")))
194
195 for label, rows in source_tables.items():
196     seen = []
197     for line, row in enumerate(rows, start=2):
198         if not row.get("source_url", row.get("url", "")).startswith("https://"):
199             issues.append(f"{label}:{line}: direct HTTPS source URL is required")

```

```

200         if not validate_date(row.get("retrieval_date")):
201             issues.append(f"{label}:{line}: invalid/post-cutoff retrieval_date")
202         if "model_id" in row and row.get("model_id") not in candidate_ids:
203             issues.append(f"{label}:{line}: model_id absent from candidates")
204         seen.append(tuple(row.values()))
205     if len(seen) != len(set(seen)):
206         issues.append(f"{label}: exact duplicate source row")
207
208     if not 8 <= len(core_selection) <= 12:
209         issues.append(f"core_selection: expected 8-12 indicators, got {len(core_selection)}")
210     for line, row in enumerate(core_selection, start=2):
211         try:
212             best = int(row.get("best_comparable_models", ""))
213             total = int(row.get("final_models", ""))
214             coverage = float(row.get("coverage", "").rstrip("%"))
215         except ValueError:
216             issues.append(f"core_selection:{line}: invalid coverage fields")
217             continue
218         if total != len(final_ids) or best > total:
219             issues.append(f"core_selection:{line}: counts do not match final pool")
220         if abs(coverage - best / total * 100) > 0.11 or coverage < 75:
221             issues.append(f"core_selection:{line}: coverage must be accurate and >=75%")
222         if row.get("status") != "core":
223             issues.append(f"core_selection:{line}: status must be core")
224
225     core_keys = {row["indicator_key"] for row in processed["core_long"]}
226     if len(processed["core_matrix"]) != len(final_ids):
227         issues.append("core_matrix: must contain one row per final model")
228     if len(processed["core_long"]) != len(final_ids) * len(core_selection):
229         issues.append("core_long: must contain final_models x core_indicators rows")
230     if {row["model_id"] for row in processed["core_matrix"]} != final_ids:
231         issues.append("core_matrix: model ids do not match final pool")
232     if len(core_keys) != len(core_selection):
233         issues.append("core_long: indicator count does not match core selection")
234     if len(processed["indicator_quality"]) != len(core_selection):
235         issues.append("indicator_quality: must contain one row per core indicator")
236     for line, row in enumerate(processed["indicator_quality"], start=2):
237         try:
238             available = int(row.get("available_models", ""))
239             total = int(row.get("final_models", ""))
240             coverage = float(row.get("coverage_percent", ""))
241         except ValueError:
242             issues.append(f"indicator_quality:{line}: invalid coverage fields")
243             continue
244         if total != len(final_ids) or abs(coverage - available / total * 100) > 0.11:
245             issues.append(f"indicator_quality:{line}: inaccurate coverage")
246         if coverage < 75 or row.get("cohort_status") != "pass":
247             issues.append(f"indicator_quality:{line}: frozen cohort must pass 75%")
248     if len({(row["model_id"], row["indicator_key"]) for row in processed["core_long"]}) != len(processed["core_long"]):
249         issues.append("core_long: duplicate model/indicator row")
250     for line, row in enumerate(processed["core_long"], start=2):
251         if not numeric_or_na(row.get("score")):

```

```

252         issues.append(f"core_long:{line}: score must be numeric or literal NA")
253     if row.get("score") == "NA" and missing(row.get("missing_reason")):
254         issues.append(f"core_long:{line}: missing score requires a reason")
255     if (
256         row.get("model_id") == "glm-5.2"
257         and row.get("dimension") in {
258             "multimodal", "document_understanding",
259             "research_document_reasoning", "multimodal_math",
260         }
261         and row.get("missing_reason") != "not applicable: frozen model is text-only"
262     ):
263         issues.append(f"core_long:{line}: GLM visual gap must be marked structurally
not applicable")
264     if len(processed["model_attributes"]) != len(final_ids):
265         issues.append("model_attributes: must contain one row per final model")
266     for line, row in enumerate(processed["model_attributes"], start=2):
267         if row.get("model_id") not in final_ids:
268             issues.append(f"model_attributes:{line}: model is not final")
269         if row.get("efficiency_compatible") == "false":
270             comparable = (
271                 row.get("comparable_ttft_seconds"),
272                 row.get("comparable_output_speed_tokens_per_second"),
273                 row.get("comparable_total_latency_seconds"),
274             )
275             if any(value != "NA" for value in comparable):
276                 issues.append(f"model_attributes:{line}: incompatible efficiency must be NA
in comparable fields")
277     if any(row.get("status") != "pass" for row in processed["quality_report"]):
278         issues.append("quality_report: all Phase 2 checks must pass")
279
280     for label, keys in (("benchmark", benchmark_keys), ("metadata", metadata_keys), ("
efficiency", efficiency_keys)):
281         for key, count in Counter(keys).items():
282             if count > 1:
283                 issues.append(f"{label}: duplicate record key ({count} rows): {key}")
284     for model_name, model_versions in versions.items():
285         if len(model_versions) > 1:
286             issues.append(f"version mix: {model_name} has versions {sorted(model_versions)
}")
287
288     row_count = len(candidates) + len(benchmarks) + len(metadata) + len(efficiency) + sum(
map(len, source_tables.values())) + len(core_selection) + sum(map(len, processed.values
()))
289     if issues:
290         print(f"Validation completed: {len(issues)} issue(s) across {row_count} row(s).")
291         print("\n".join(f"- {item}" for item in issues))
292         return 1
293     print(f"Validation passed: {row_count} data row(s), no issues found.")
294     return 0
295
296
297 if __name__ == "__main__":
298     raise SystemExit(main())

```

source/scripts/validate_modeling.py

```

1  """Validate cross-phase modeling outputs and fail closed on inconsistent results."""
2
3  from __future__ import annotations
4
5  import csv
6  from pathlib import Path
7
8  ROOT = Path(__file__).resolve().parents[1]
9
10
11 def read_rows(relative: str) -> list[dict[str, str]]:
12     path = ROOT / relative
13     if not path.exists():
14         raise ValueError(f"Missing output: {relative}")
15     with path.open(encoding="utf-8-sig", newline="") as handle:
16         return list(csv.DictReader(handle))
17
18
19 def validate() -> list[str]:
20     issues: list[str] = []
21     screening = read_rows("results/phase3/indicator_screening.csv")
22     if len(screening) != 9 or any(row["phase3_decision"] != "retain" for row in screening):
23         issues.append("Phase 3 must document nine retained indicators")
24
25     weights = read_rows("results/phase4/critic_weights.csv")
26     if len(weights) != 9 or abs(sum(float(row["critic_weight"]) for row in weights) - 1) >
27         1e-8:
28         issues.append("CRITIC weights must contain nine indicators and sum to one")
29     ranking = read_rows("results/phase4/general_ranking.csv")
30     ranked = [row for row in ranking if row["ranking_status"] == "ranked_complete_case"]
31     if sorted(int(row["rank"]) for row in ranked) != list(range(1, len(ranked) + 1)):
32         issues.append("General ranks must be unique and consecutive")
33     if len(ranked) != 5 or sum(row["ranking_status"] == "not_ranked_insufficient_coverage"
34         for row in ranked) != 1:
35         issues.append("General ranking must have five complete cases and one explicit
36             exclusion")
37
38     scenario_weights = read_rows("results/phase5/scenario_weights.csv")
39     scenarios = {row["scenario"] for row in scenario_weights}
40     if len(scenarios) != 3:
41         issues.append("Exactly three scenarios are required")
42     for scenario in scenarios:
43         total = sum(float(row["combined_weight"]) for row in scenario_weights if row["
44             scenario"] == scenario)
45         if abs(total - 1) > 1e-8:
46             issues.append(f"Scenario weights do not sum to one: {scenario}")
47
48     cost = read_rows("results/phase6/performance_cost.csv")
49     if not any(row["pareto_frontier"] == "true" for row in cost):
50         issues.append("Performance-cost analysis has no Pareto model")
51     budgets = read_rows("results/phase6/budget_recommendations.csv")
52     for row in budgets:

```

```

49     if row["recommended_model_id"] != "NA" and float(row["cost_usd"]) > float(row["
budget_usd_per_standardized_workload"]):
50         issues.append("Budget recommendation exceeds its budget")
51
52     correlations = read_rows("results/phase7/rank_correlation.csv")
53     if len(correlations) != 36:
54         issues.append("Expected 36 weight perturbation scenarios")
55     if any(not -1 <= float(row["kendall_tau_vs_baseline"]) <= 1 for row in correlations):
56         issues.append("Kendall tau outside [-1, 1]")
57
58     for name in ("general_ranking.svg", "critic_weights.svg", "scenario_rank_changes.svg",
"performance_cost_pareto.svg"):
59         path = ROOT / "figures" / name
60         if not path.exists() or not path.read_text(encoding="utf-8").startswith("<svg"):
61             issues.append(f"Missing or invalid figure: {name}")
62     return issues
63
64
65 def main() -> int:
66     try:
67         issues = validate()
68     except (KeyError, ValueError) as exc:
69         print(f"ERROR: {exc}")
70         return 1
71     if issues:
72         print(f"Modeling validation failed: {len(issues)} issue(s).")
73         for issue in issues:
74             print(f"- {issue}")
75         return 1
76     print("Modeling validation passed: Phases 3-7 outputs are internally consistent.")
77     return 0
78
79
80 if __name__ == "__main__":
81     raise SystemExit(main())

```

source/scripts/validate_paper.py

```

1  """Check that the report covers the original problem and has a readable PDF."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  ROOT = Path(__file__).resolve().parents[1]
8  REPORT = ROOT / "paper" / "modeling_report.md"
9  PROBLEM = ROOT / "references" / "problem" / "TJMML_B.pdf"
10 PDF = ROOT / "output" / "pdf" / "modeling_report_draft.pdf"
11 REQUIRED_TEXT = [
12     "CRITIC—TOPSIS 综合评价", "Kimi K3 优势与短板", "三类场景评价",
13     "性能—成本与工程效率", "energy_data_available=False", "88.89%",
14     "R^2=0.3147", "数据来源与可追溯性", "GLM-5.2 的同口径局部成绩",
15 ]

```

```

16
17
18 def main() -> int:
19     failures: list[str] = []
20     if not PROBLEM.exists():
21         failures.append("original problem PDF is missing")
22     report_text = REPORT.read_text(encoding="utf-8") if REPORT.exists() else ""
23     if not report_text:
24         failures.append("Markdown report is missing")
25     failures.extend(f"report marker is missing: {m}" for m in REQUIRED_TEXT if m not in
report_text)
26     if not PDF.exists() or PDF.stat().st_size < 100_000:
27         failures.append("built PDF is missing or unexpectedly small")
28     else:
29         try:
30             import fitz
31             document = fitz.open(PDF)
32             if len(document) < 8:
33                 failures.append(f"built PDF has too few pages: {len(document)}")
34             extracted = "".join(page.get_text() for page in document)
35             if "Kimi K3" not in extracted or "Pareto" not in extracted:
36                 failures.append("built PDF text extraction misses key content")
37             document.close()
38         except Exception as exc:
39             failures.append(f"could not inspect built PDF: {exc}")
40     if failures:
41         for failure in failures:
42             print(f"FAIL: {failure}")
43         return 1
44     print("Paper validation passed: original, coverage markers, and PDF are present.")
45     return 0
46
47
48 if __name__ == "__main__":
49     raise SystemExit(main())

```

source/scripts/validate_q1.py

```

1  """Validate the complete Question 1 deliverable without changing source data."""
2
3  from __future__ import annotations
4
5  import argparse
6  from pathlib import Path
7
8  import pandas as pd
9  from PIL import Image
10
11  ROOT = Path(__file__).resolve().parents[1]
12
13  CSV_FILES = (
14      "data_quality_report.csv",
15      "normalized_data.csv",

```

```

16     "pearson_correlation.csv",
17     "spearman_correlation.csv",
18     "high_correlation_pairs.csv",
19     "selected_metrics.csv",
20     "removed_metrics.csv",
21     "final_indicator_system.csv",
22     "critic_weights.csv",
23     "topsis_ranking.csv",
24     "kimi_k3_metric_analysis.csv",
25     "sensitivity_analysis.csv",
26     "rank_stability.csv",
27 )
28 PNG_FILES = (
29     "pearson_heatmap.png",
30     "spearman_heatmap.png",
31     "critic_weights_bar.png",
32     "topsis_ranking_bar.png",
33     "kimi_k3_radar.png",
34     "kimi_k3_gap.png",
35 )
36 DOC_FILES = (
37     "data_dictionary.md",
38     "q1_metric_selection.md",
39     "q1_results_summary.md",
40 )
41
42
43 def validate(output_dir: Path, docs_dir: Path) -> list[str]:
44     issues: list[str] = []
45     for name in CSV_FILES:
46         path = output_dir / name
47         if not path.is_file() or path.stat().st_size == 0:
48             issues.append(f"missing or empty CSV: {path}")
49     for name in DOC_FILES:
50         path = docs_dir / name
51         if not path.is_file() or path.stat().st_size == 0:
52             issues.append(f"missing or empty document: {path}")
53     for name in PNG_FILES:
54         path = output_dir / name
55         if not path.is_file() or path.stat().st_size == 0:
56             issues.append(f"missing or empty PNG: {path}")
57         continue
58         try:
59             with Image.open(path) as image:
60                 dpi = image.info.get("dpi", (0, 0))
61                 if image.format != "PNG":
62                     issues.append(f"not a PNG: {path}")
63                 if min(dpi) < 299:
64                     issues.append(f"PNG is below 300 dpi: {path} ({dpi})")
65         except OSError as exc:
66             issues.append(f"unreadable PNG: {path} ({exc})")
67
68     if issues:
69         return issues

```

```

70
71 weights = pd.read_csv(output_dir / "critic_weights.csv", encoding="utf-8-sig")
72 ranking = pd.read_csv(output_dir / "topsis_ranking.csv", encoding="utf-8-sig")
73 normalized = pd.read_csv(output_dir / "normalized_data.csv", encoding="utf-8-sig")
74 sensitivity = pd.read_csv(output_dir / "sensitivity_analysis.csv", encoding="utf-8-sig")
75
76 stability = pd.read_csv(output_dir / "rank_stability.csv", encoding="utf-8-sig")
77
78 if abs(float(weights["weight"].sum()) - 1.0) > 1e-10:
79     issues.append("CRITIC weights do not sum to one")
80 if len(weights) != 9 or weights["metric"].nunique() != 9:
81     issues.append("final CRITIC weight table must contain 9 unique metrics")
82 if ranking["model_id"].nunique() != 6:
83     issues.append("TOPSIS table must contain 6 unique models")
84 ranked = ranking.loc[ranking["ranking_status"] == "ranked_complete_case"]
85 if len(ranked) != 5 or not ranked["topsis_score"].between(0, 1).all():
86     issues.append("TOPSIS must rank 5 complete models with scores in [0, 1]")
87 glm = ranking.loc[ranking["model_id"] == "glm-5.2"]
88 if len(glm) != 1 or glm["rank"].notna().any():
89     issues.append("GLM-5 must remain explicitly unranked because of incomplete coverage")
90
91 metric_columns = [column for column in normalized if column not in {"model_id", "model"}]
92 values = normalized[metric_columns].stack()
93 if len(normalized) != 6 or len(metric_columns) != 9 or not values.between(0, 1).all():
94     issues.append("normalized matrix must be 6 x 9 with observed values in [0, 1]")
95 monte_carlo = sensitivity.loc[sensitivity["analysis_type"] == "monte_carlo_weight_perturbation"]
96 if monte_carlo["iteration"].nunique() != 1000:
97     issues.append("sensitivity analysis must contain 1000 Monte Carlo iterations")
98 kimi = stability.loc[stability["model_id"] == "kimi-k3"]
99 if len(kimi) != 1 or float(kimi.iloc[0]["top3_probability"]) < 0.95:
100     issues.append("Kimi K3 Top3 stability is unexpectedly below 95%")
101
102 return issues
103
104 def main() -> int:
105     parser = argparse.ArgumentParser(description=__doc__)
106     parser.add_argument("--output-dir", type=Path, default=ROOT / "results/q1")
107     parser.add_argument("--docs-dir", type=Path, default=ROOT / "docs")
108     args = parser.parse_args()
109     issues = validate(args.output_dir, args.docs_dir)
110     if issues:
111         print("Question 1 validation failed:")
112         for issue in issues:
113             print(f"- {issue}")
114         return 1
115     print(f"Question 1 validation passed: {len(CSV_FILES)} CSVs, {len(PNG_FILES)} PNGs, {len(DOC_FILES)} documents.")
116     return 0
117
118 if __name__ == "__main__":
119     raise SystemExit(main())

```

source/scripts/validate_q2_q3.py

```

1  """Validate Question 2 and Question 3 deliverables and mathematical invariants."""
2
3  from __future__ import annotations
4
5  import argparse
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10 from PIL import Image
11
12 ROOT = Path(__file__).resolve().parents[1]
13 Q2_CSV = (
14     "scenario_weights.csv", "scenario_rankings.csv", "research_ranking.csv", "
15     dialogue_ranking.csv",
16     "coding_ranking.csv", "rank_change_analysis.csv", "kimi_k3_scenario_analysis.csv",
17     "scenario_sensitivity.csv", "scenario_stability.csv",
18 )
19 Q2_PNG = (
20     "scenario_rank_comparison.png", "scenario_score_heatmap.png", "rank_change.png",
21     "kimi_k3_scenario_radar.png", "scenario_weights_heatmap.png",
22 )
23 Q3_CSV = (
24     "cost_data_audit.csv", "performance_cost.csv", "pareto_analysis.csv", "budget_selection
25     .csv",
26     "performance_cost_regression.csv", "deploy_scores.csv", "kimi_k3_cost_analysis.csv", "
27     sensitivity_analysis.csv",
28 )
29 Q3_PNG = ("performance_cost_pareto.png", "budget_selection.png", "performance_cost_fit.png",
30     "engineering_efficiency.png")
31 DOCS = ("q2_results_summary.md", "q3_results_summary.md", "model_implementation_mapping.md",
32     "")
33
34 def validate_png(path: Path, issues: list[str]) -> None:
35     if not path.is_file() or path.stat().st_size == 0:
36         issues.append(f"missing or empty PNG: {path}")
37         return
38     try:
39         with Image.open(path) as image:
40             dpi = image.info.get("dpi", (0, 0))
41             if image.format != "PNG" or min(dpi) < 299:
42                 issues.append(f"invalid PNG format or DPI: {path} ({image.format}, {dpi})")
43     except OSError as exc:
44         issues.append(f"unreadable PNG: {path} ({exc})")
45
46 def validate(q2_dir: Path, q3_dir: Path, docs_dir: Path, summary_path: Path) -> list[str]:
47     issues: list[str] = []

```

```

45 for directory, names in ((q2_dir, Q2_CSV), (q3_dir, Q3_CSV)):
46     for name in names:
47         path = directory / name
48         if not path.is_file() or path.stat().st_size == 0:
49             issues.append(f"missing or empty CSV: {path}")
50 for directory, names in ((q2_dir, Q2_PNG), (q3_dir, Q3_PNG)):
51     for name in names:
52         validate_png(directory / name, issues)
53 for name in DOCS:
54     path = docs_dir / name
55     if not path.is_file() or path.stat().st_size == 0:
56         issues.append(f"missing or empty document: {path}")
57 if not summary_path.is_file() or summary_path.stat().st_size == 0:
58     issues.append(f"missing final summary: {summary_path}")
59 if issues:
60     return issues
61
62 weights = pd.read_csv(q2_dir / "scenario_weights.csv", encoding="utf-8-sig")
63 rankings = pd.read_csv(q2_dir / "scenario_rankings.csv", encoding="utf-8-sig")
64 q2_sensitivity = pd.read_csv(q2_dir / "scenario_sensitivity.csv", encoding="utf-8-sig")
65 totals = weights.groupby("scenario")["combined_weight"].sum()
66 if len(totals) != 3 or not np.allclose(totals.to_numpy(), 1.0, atol=1e-10):
67     issues.append("Q2 must contain three scenario weight vectors summing to one")
68 ranked = rankings.loc[rankings["ranking_status"] == "ranked_complete_case"]
69 if not (ranked.groupby("scenario")["model_id"].nunique() == 5).all():
70     issues.append("Q2 must rank exactly five complete models in each scenario")
71 if not ranked["topsis_score"].between(0, 1).all():
72     issues.append("Q2 TOPSIS scores must be within [0, 1]")
73 glm = rankings.loc[rankings["model_id"] == "glm-5.2"]
74 if len(glm) != 3 or glm["rank"].notna().any():
75     issues.append("GLM-5.2 must remain unranked in all Q2 scenarios")
76 if len(q2_sensitivity) != 540 or q2_sensitivity[["scenario", "perturbed_metric", "
factor"]].drop_duplicates().shape[0] != 108:
77     issues.append("Q2 sensitivity must contain 36 perturbations x 3 scenarios x 5
models")
78
79 audit = pd.read_csv(q3_dir / "cost_data_audit.csv", encoding="utf-8-sig")
80 value = pd.read_csv(q3_dir / "performance_cost.csv", encoding="utf-8-sig")
81 pareto = pd.read_csv(q3_dir / "pareto_analysis.csv", encoding="utf-8-sig")
82 budgets = pd.read_csv(q3_dir / "budget_selection.csv", encoding="utf-8-sig")
83 regression = pd.read_csv(q3_dir / "performance_cost_regression.csv", encoding="utf-8-
sig")
84 q3_sensitivity = pd.read_csv(q3_dir / "sensitivity_analysis.csv", encoding="utf-8-sig")
85 summary = pd.read_csv(summary_path, encoding="utf-8-sig")
86 expected_cost = audit["input_price_usd_per_million"] + 0.2 * audit["
output_price_usd_per_million"]
87 if len(audit) != 6 or not np.allclose(expected_cost, audit["standard_workload_cost_usd
"]):
88     issues.append("Q3 standard cost must equal input price + 0.2 x output price for six
models")
89 included = value.loc[value["analysis_status"] == "included_ranked_model"]
90 if len(included) != 5 or not included["q1_performance_score"].between(0, 1).all():
91     issues.append("Q3 must use five valid Q1 performance scores")
92 for row in budgets.itertuples():

```

```

93         if row.cost_usd > row.budget_limit_usd + 1e-10:
94             issues.append(f"budget recommendation violates constraint at {row.
budget_limit_usd}")
95     for item in pareto.itertuples():
96         others = pareto.loc[pareto["model_id"] != item.model_id]
97         dominated = (
98             (others["standard_workload_cost_usd"] <= item.standard_workload_cost_usd)
99             & (others["q1_performance_score"] >= item.q1_performance_score)
100             & ((others["standard_workload_cost_usd"] < item.standard_workload_cost_usd) | (
others["q1_performance_score"] > item.q1_performance_score))
101         ).any()
102         if bool(item.is_pareto_frontier) == bool(dominated):
103             issues.append(f"incorrect Pareto status for {item.model_id}")
104     if regression["sample_size"].nunique() != 1 or int(regression["sample_size"].iloc[0])
!= 5:
105         issues.append("Q3 regression must use the five ranked models")
106     if len(q3_sensitivity) != 36:
107         issues.append("Q3 sensitivity must contain 36 model-specified perturbations")
108     if len(summary) != 6 or summary["model_id"].nunique() != 6:
109         issues.append("final_model_summary.csv must contain six unique models")
110     return issues
111
112
113 def main() -> int:
114     parser = argparse.ArgumentParser(description=__doc__)
115     parser.add_argument("--q2-dir", type=Path, default=ROOT / "results/q2")
116     parser.add_argument("--q3-dir", type=Path, default=ROOT / "results/q3")
117     parser.add_argument("--docs-dir", type=Path, default=ROOT / "docs")
118     parser.add_argument("--summary", type=Path, default=ROOT / "results/final_model_summary
.csv")
119     args = parser.parse_args()
120     issues = validate(args.q2_dir, args.q3_dir, args.docs_dir, args.summary)
121     if issues:
122         print("Q2/Q3 validation failed:")
123         for issue in issues:
124             print(f"- {issue}")
125         return 1
126     print(f"Q2/Q3 validation passed: {len(Q2_CSV)+len(Q3_CSV)+1} CSVs, {len(Q2_PNG)+len(
Q3_PNG)} PNGs, {len(DOCS)} documents.")
127     return 0
128
129
130 if __name__ == "__main__":
131     raise SystemExit(main())

```

source/src/q1/__init__.py

```

1 """Question 1 reproducible evaluation pipeline."""
2
3 from .analysis import run_analysis
4
5 __all__ = ["run_analysis"]

```

source/src/q1/analysis.py

```

1  """Numerical analysis for Question 1 without modifying frozen input data."""
2
3  from __future__ import annotations
4
5  import csv
6  import math
7  import random
8  from collections import defaultdict
9  from pathlib import Path
10
11 import numpy as np
12 import pandas as pd
13
14 NA = "NA"
15 RANDOM_SEED = 20260817
16 CORRELATION_THRESHOLDS = (0.80, 0.85, 0.90)
17 MONTE_CARLO_RUNS = 1000
18 WEIGHT_PERTURBATION = 0.10
19
20 DIMENSION_NAMES = {
21     "high_difficulty_reasoning": "高难度知识与逻辑推理",
22     "long_context": "长文本理解与推理",
23     "coding": "代码与科学编程",
24     "professional_tasks": "专业任务能力",
25     "multimodal": "多模态理解",
26     "document_understanding": "文档理解",
27     "research_document_reasoning": "科研文档推理",
28     "multimodal_math": "多模态数学推理",
29 }
30
31 METRIC_MEANINGS = {
32     "gpqa_diamond": "研究生水平科学问答, 衡量高难度知识推理能力",
33     "hle_full_no_tools": "Humanity's Last Exam 无工具成绩, 衡量极高难度开放推理",
34     "aa_lcr": "Artificial Analysis 长上下文推理成绩",
35     "scicode": "科学研究代码生成与求解能力",
36     "gdpval_aa_v2": "专业知识工作任务的相对能力评分",
37     "mmmu_pro_no_tools": "无工具多学科多模态理解能力",
38     "omnidocbench": "复杂文档解析与理解能力",
39     "charxiv_rq_no_tools": "科研论文图表与问题推理能力",
40     "mathvision_no_tools": "无工具视觉数学推理能力",
41 }
42
43
44 def read_csv(path: Path) -> pd.DataFrame:
45     if not path.exists():
46         raise FileNotFoundError(f"Required input is missing: {path}")
47     return pd.read_csv(path, encoding="utf-8-sig", keep_default_na=False, na_values=[NA])
48
49
50 def write_csv(frame: pd.DataFrame, path: Path) -> None:
51     path.parent.mkdir(parents=True, exist_ok=True)
52     frame.to_csv(path, index=False, encoding="utf-8-sig", na_rep=NA)

```

```

53
54
55 def load_frozen_data(root: Path) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
56     long = read_csv(root / "data/processed/core_benchmark_long.csv")
57     quality = read_csv(root / "data/processed/indicator_quality.csv")
58     attributes = read_csv(root / "data/processed/model_attributes.csv")
59     required = {
60         "model_id", "model_name", "indicator_key", "dimension", "indicator",
61         "score", "score_unit", "higher_is_better", "source_url",
62     }
63     missing = required - set(long.columns)
64     if missing:
65         raise ValueError(f"Frozen long table lacks columns: {sorted(missing)}")
66     long["score"] = pd.to_numeric(long["score"], errors="coerce")
67     return long, quality, attributes
68
69
70 def ordered_metadata(long: pd.DataFrame) -> tuple[list[str], list[str], dict[str, str],
71     dict[str, str], dict[str, str], dict[str, bool]]:
72     models = long["model_id"].drop_duplicates().tolist()
73     metrics = long["indicator_key"].drop_duplicates().tolist()
74     model_names = long.drop_duplicates("model_id").set_index("model_id")["model_name"].
75     to_dict()
76     metric_names = long.drop_duplicates("indicator_key").set_index("indicator_key")["
77     indicator"].to_dict()
78     dimensions = long.drop_duplicates("indicator_key").set_index("indicator_key")["
79     dimension"].to_dict()
80     directions = {
81         row.indicator_key: str(row.higher_is_better).lower() == "true"
82         for row in long.drop_duplicates("indicator_key").itertuples()
83     }
84     return models, metrics, model_names, metric_names, dimensions, directions
85
86
87
88 def raw_matrix(long: pd.DataFrame, models: list[str], metrics: list[str]) -> pd.DataFrame:
89     matrix = long.pivot(index="model_id", columns="indicator_key", values="score")
90     return matrix.reindex(index=models, columns=metrics)
91
92
93
94 def data_quality_report(
95     long: pd.DataFrame,
96     matrix: pd.DataFrame,
97     quality: pd.DataFrame,
98     metrics: list[str],
99     directions: dict[str, bool],
100 ) -> pd.DataFrame:
101     rows: list[dict[str, object]] = []
102
103     def add(scope: str, item: str, status: str, value: object, details: str) -> None:
104         rows.append({"scope": scope, "item": item, "status": status, "value": value, "
105         details": details})
106
107     add("dataset", "model_count", "pass" if matrix.index.nunique() == len(matrix) else "
108     fail", len(matrix), "One row per final model")

```

```

101 add("dataset", "indicator_count", "pass" if 8 <= len(metrics) <= 12 else "review", len(
102 metrics), "Frozen Question 1 indicator set")
103 duplicate_pairs = int(long.duplicated(["model_id", "indicator_key"]).sum())
104 add("dataset", "duplicate_model_indicator_pairs", "pass" if duplicate_pairs == 0 else "
fail", duplicate_pairs, "Must be zero")
105 add("dataset", "text_identifier_columns", "pass", "model_id/model_name", "Identifiers
are excluded from numerical modeling")
106 add("dataset", "cost_latency_energy_excluded", "pass", "yes", "Engineering and price
fields are reserved for Question 3")
107 for metric in metrics:
108     values = matrix[metric].dropna().astype(float)
109     qrow = quality.loc[quality["indicator_key"] == metric].iloc[0]
110     missing_count = int(matrix[metric].isna().sum())
111     constant = bool(values.nunique() <= 1)
112     status = "pass" if len(values) >= 5 and not constant and np.isfinite(values).all()
else "review"
113     add(
114         "indicator",
115         metric,
116         status,
117         f"available={len(values)}; missing={missing_count}",
118         (
119             f"dtype=numeric; unit={qrow['score_unit']}; direction={'positive' if
directions[metric] else 'negative'}; "
120             f"min={values.min():.6g}; max={values.max():.6g}; constant={str(constant).
lower()}"
121         ),
122     )
123 return pd.DataFrame(rows)
124
125 def minmax_normalize(matrix: pd.DataFrame, directions: dict[str, bool]) -> pd.DataFrame:
126     normalized = pd.DataFrame(index=matrix.index, columns=matrix.columns, dtype=float)
127     for metric in matrix.columns:
128         values = matrix[metric].astype(float)
129         low, high = values.min(skipna=True), values.max(skipna=True)
130         if not np.isfinite(low) or not np.isfinite(high):
131             raise ValueError(f"Indicator {metric} has no usable values")
132         if math.isclose(low, high):
133             normalized[metric] = values.where(values.isna(), 0.5)
134         elif directions[metric]:
135             normalized[metric] = (values - low) / (high - low)
136         else:
137             normalized[metric] = (high - values) / (high - low)
138     return normalized
139
140
141 def correlation_matrices(matrix: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame, pd.
DataFrame]:
142     pearson = matrix.corr(method="pearson", min_periods=3)
143     spearman = matrix.corr(method="spearman", min_periods=3)
144     counts = pd.DataFrame(index=matrix.columns, columns=matrix.columns, dtype=int)
145     for left in matrix.columns:
146         for right in matrix.columns:

```

```

147         counts.loc[left, right] = int(matrix[[left, right]].dropna().shape[0])
148     return pearson, spearman, counts.astype(int)
149
150
151 def select_metrics(
152     matrix: pd.DataFrame,
153     pearson: pd.DataFrame,
154     spearman: pd.DataFrame,
155     counts: pd.DataFrame,
156     metrics: list[str],
157     metric_names: dict[str, str],
158     dimensions: dict[str, str],
159     threshold: float = 0.85,
160 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, list[str]]:
161     pair_rows: list[dict[str, object]] = []
162     for index, left in enumerate(metrics):
163         for right in metrics[index + 1:]:
164             p, s = float(pearson.loc[left, right]), float(spearman.loc[left, right])
165             if max(abs(p), abs(s)) < threshold:
166                 continue
167             same_dimension = dimensions[left] == dimensions[right]
168             reason = (
169                 "保留二者: 共同样本仅5个, 且分别衡量不同能力维度; 高相关性不足以证明语义冗余, 后续
170                 场景评价仍分别需要"
171                 if not same_dimension
172                 else "共同样本有限: 两指标虽同属一级维度但测量任务不同, 保留并通过稳健性分析检验"
173             )
174             pair_rows.append({
175                 "metric_a": left,
176                 "metric_b": right,
177                 "metric_a_name": metric_names[left],
178                 "metric_b_name": metric_names[right],
179                 "pairwise_models": int(counts.loc[left, right]),
180                 "pearson_r": p,
181                 "spearman_r": s,
182                 "threshold": threshold,
183                 "same_dimension": same_dimension,
184                 "decision": "retain_both",
185                 "reason": reason,
186             })
187     pair_columns = [
188         "metric_a", "metric_b", "metric_a_name", "metric_b_name", "pairwise_models",
189         "pearson_r", "spearman_r", "threshold", "same_dimension", "decision", "reason",
190     ]
191     pairs = pd.DataFrame(pair_rows, columns=pair_columns)
192     selected_rows = []
193     for metric in metrics:
194         values = matrix[metric].dropna().astype(float)
195         selected_rows.append({
196             "metric": metric,
197             "indicator": metric_names[metric],
198             "dimension": dimensions[metric],
199             "available_models": len(values),
200             "missing_rate": 1 - len(values) / len(matrix),

```

```

200         "sample_std": values.std(ddof=1),
201         "coefficient_of_variation": values.std(ddof=1) / abs(values.mean()) if values.
mean() else np.nan,
202         "decision": "retain",
203         "reason": "覆盖率≥75%、非零变异、来源可追溯且具有独立能力含义；小样本高相关不作机械删
除",
204     })
205     selected = pd.DataFrame(selected_rows)
206     removed = pd.DataFrame(columns=["removed_metric", "retained_metric", "pearson_r", "
spearman_r", "reason"])
207     return pairs, selected, removed, metrics.copy()
208
209
210 def critic_weights(normalized: pd.DataFrame, metrics: list[str]) -> pd.DataFrame:
211     info_rows = []
212     information: dict[str, float] = {}
213     for left in metrics:
214         correlations = []
215         for right in metrics:
216             if left == right:
217                 continue
218             paired = normalized[[left, right]].dropna()
219             if len(paired) < 3:
220                 raise ValueError(f"CRITIC correlation lacks observations: {left}/{right}")
221             correlation = float(paired[left].corr(paired[right], method="pearson"))
222             if not np.isfinite(correlation):
223                 raise ValueError(f"CRITIC correlation is invalid: {left}/{right}")
224             correlations.append(correlation)
225             std = float(normalized[left].dropna().std(ddof=0))
226             conflict = float(sum(1 - value for value in correlations))
227             information[left] = std * conflict
228             info_rows.append({"metric": left, "std": std, "conflict": conflict, "information":
information[left]})
229     total = sum(information.values())
230     if total <= 0:
231         raise ValueError("CRITIC information must be positive")
232     frame = pd.DataFrame(info_rows)
233     frame["weight"] = frame["metric"].map(lambda item: information[item] / total)
234     frame["rank"] = frame["weight"].rank(method="min", ascending=False).astype(int)
235     return frame.sort_values("rank").reset_index(drop=True)
236
237
238 def topsis_scores(normalized: pd.DataFrame, metrics: list[str], weights: dict[str, float])
-> pd.DataFrame:
239     complete = normalized[metrics].dropna(axis=0, how="any")
240     if len(complete) < 2:
241         raise ValueError("TOPSIS requires at least two complete models")
242     weight_vector = np.array([weights[metric] for metric in metrics], dtype=float)
243     weighted = complete.to_numpy(dtype=float) * weight_vector
244     positive = weight_vector
245     negative = np.zeros_like(weight_vector)
246     d_positive = np.sqrt(((weighted - positive) ** 2).sum(axis=1))
247     d_negative = np.sqrt(((weighted - negative) ** 2).sum(axis=1))
248     scores = d_negative / (d_positive + d_negative)

```

```

249     frame = pd.DataFrame({
250         "model_id": complete.index,
251         "distance_positive": d_positive,
252         "distance_negative": d_negative,
253         "topsis_score": scores,
254     })
255     frame = frame.sort_values(["topsis_score", "model_id"], ascending=[False, True]).
reset_index(drop=True)
256     frame["rank"] = np.arange(1, len(frame) + 1)
257     return frame
258
259
260 def full_ranking(
261     normalized: pd.DataFrame,
262     metrics: list[str],
263     weights: dict[str, float],
264     model_names: dict[str, str],
265 ) -> pd.DataFrame:
266     ranked = topsis_scores(normalized, metrics, weights)
267     rank_by_id = ranked.set_index("model_id").to_dict("index")
268     rows = []
269     for model_id in normalized.index:
270         available = int(normalized.loc[model_id, metrics].notna().sum())
271         base = {
272             "model": model_names[model_id],
273             "model_id": model_id,
274             "available_indicators": available,
275             "total_indicators": len(metrics),
276             "coverage_percent": 100 * available / len(metrics),
277         }
278         if model_id in rank_by_id:
279             base.update(rank_by_id[model_id])
280             base["ranking_status"] = "ranked_complete_case"
281         else:
282             base.update({"distance_positive": np.nan, "distance_negative": np.nan, "
topsis_score": np.nan, "rank": np.nan})
283             base["ranking_status"] = "not_ranked_insufficient_coverage"
284             rows.append(base)
285     return pd.DataFrame(rows).sort_values(["rank", "model_id"], na_position="last").
reset_index(drop=True)
286
287
288 def kimi_metric_analysis(
289     raw: pd.DataFrame,
290     normalized: pd.DataFrame,
291     metrics: list[str],
292     metric_names: dict[str, str],
293     dimensions: dict[str, str],
294     model_names: dict[str, str],
295 ) -> pd.DataFrame:
296     kimi_id = "kimi-k3"
297     rows = []
298     for metric in metrics:
299         raw_values = raw[metric].dropna().astype(float)

```

```

300     norm_values = normalized[metric].dropna().astype(float)
301     order = norm_values.sort_values(ascending=False, kind="mergesort")
302     # Competition ranking: tied values receive the same rank.
303     rank = int((norm_values > norm_values.loc[kimi_id]).sum()) + 1
304     leader_id = str(order.index[0])
305     gap = float(order.iloc[0] - norm_values.loc[kimi_id])
306     mean_norm = float(norm_values.mean())
307     if rank <= 2 and norm_values.loc[kimi_id] >= mean_norm and gap <= 0.35:
308         assessment = "advantage"
309     elif rank >= math.ceil(len(order) * 0.60) and norm_values.loc[kimi_id] < mean_norm
and gap >= 0.25:
310         assessment = "weakness"
311     else:
312         assessment = "middle"
313     rows.append({
314         "metric": metric,
315         "indicator": metric_names[metric],
316         "dimension": dimensions[metric],
317         "raw_value": float(raw.loc[kimi_id, metric]),
318         "normalized_value": float(norm_values.loc[kimi_id]),
319         "rank": rank,
320         "models_compared": len(order),
321         "model_mean_raw": float(raw_values.mean()),
322         "difference_from_mean_raw": float(raw.loc[kimi_id, metric] - raw_values.mean())
323     },
324     {
325         "model_mean_normalized": mean_norm,
326         "difference_from_mean_normalized": float(norm_values.loc[kimi_id] - mean_norm),
327         "leader_model_id": leader_id,
328         "leader_model": model_names[leader_id],
329         "gap_to_leader_raw": float(raw.loc[leader_id, metric] - raw.loc[kimi_id, metric
330     ]),
331     {
332         "gap_to_leader_normalized": gap,
333         "assessment": assessment,
334     })
335     return pd.DataFrame(rows)
336
337 def threshold_sensitivity(
338     normalized: pd.DataFrame,
339     metrics: list[str],
340     weights: dict[str, float],
341     model_names: dict[str, str],
342     pearson: pd.DataFrame,
343     spearman: pd.DataFrame,
344 ) -> list[dict[str, object]]:
345     rows = []
346     for threshold in CORRELATION_THRESHOLDS:
347         high_correlation_pair_count = sum(
348             max(
349                 abs(float(pearson.loc[left_metric, right_metric])),
350                 abs(float(spearman.loc[left_metric, right_metric])),
351             ) >= threshold
352             for left_index, left_metric in enumerate(metrics)
353             for right_metric in metrics[left_index + 1 :])

```

```

351     )
352     # Semantic screening keeps all metrics at every threshold; the threshold only
changes review flags.
353     ranking = topsis_scores(normalized, metrics, weights)
354     for item in ranking.itertuples():
355         rows.append({
356             "analysis_type": "correlation_threshold",
357             "scenario": f"threshold_{threshold:.2f}",
358             "iteration": 0,
359             "threshold": threshold,
360             "high_correlation_pair_count": high_correlation_pair_count,
361             "perturbation_bound": 0.0,
362             "selected_metric_count": len(metrics),
363             "model_id": item.model_id,
364             "model": model_names[item.model_id],
365             "score": item.topsis_score,
366             "rank": item.rank,
367         })
368     return rows
369
370
371 def monte_carlo_sensitivity(
372     normalized: pd.DataFrame,
373     metrics: list[str],
374     base_weights: dict[str, float],
375     model_names: dict[str, str],
376 ) -> tuple[list[dict[str, object]], pd.DataFrame]:
377     rng = random.Random(RANDOM_SEED)
378     rows: list[dict[str, object]] = []
379     histories: dict[str, list[int]] = defaultdict(list)
380     for iteration in range(1, MONTE_CARLO_RUNS + 1):
381         perturbed = {
382             metric: base_weights[metric] * rng.uniform(1 - WEIGHT_PERTURBATION, 1 +
WEIGHT_PERTURBATION)
383             for metric in metrics
384         }
385         total = sum(perturbed.values())
386         perturbed = {metric: value / total for metric, value in perturbed.items()}
387         ranking = topsis_scores(normalized, metrics, perturbed)
388         for item in ranking.itertuples():
389             rank = int(item.rank)
390             histories[item.model_id].append(rank)
391             rows.append({
392                 "analysis_type": "monte_carlo_weight_perturbation",
393                 "scenario": "uniform_independent_pm10pct",
394                 "iteration": iteration,
395                 "threshold": np.nan,
396                 "high_correlation_pair_count": np.nan,
397                 "perturbation_bound": WEIGHT_PERTURBATION,
398                 "selected_metric_count": len(metrics),
399                 "model_id": item.model_id,
400                 "model": model_names[item.model_id],
401                 "score": item.topsis_score,
402                 "rank": rank,

```

```

403         })
404     stability = []
405     for model_id, ranks in histories.items():
406         array = np.asarray(ranks, dtype=float)
407         probabilities = {rank: float(np.mean(array == rank)) for rank in range(1, len(
408             histories) + 1)}
409         stability.append({
410             "model_id": model_id,
411             "model": model_names[model_id],
412             "mean_rank": float(array.mean()),
413             "rank_std": float(array.std(ddof=0)),
414             "minimum_rank": int(array.min()),
415             "maximum_rank": int(array.max()),
416             "top1_probability": float(np.mean(array <= 1)),
417             "top3_probability": float(np.mean(array <= 3)),
418             "top5_probability": float(np.mean(array <= 5)),
419             **{f"rank_{rank}_probability": probabilities[rank] for rank in probabilities},
420             "simulations": MONTE_CARLO_RUNS,
421             "random_seed": RANDOM_SEED,
422         })
423     return rows, pd.DataFrame(stability).sort_values("mean_rank").reset_index(drop=True)
424
425 def final_indicator_system(
426     metrics: list[str],
427     metric_names: dict[str, str],
428     dimensions: dict[str, str],
429     directions: dict[str, bool],
430     long: pd.DataFrame,
431 ) -> pd.DataFrame:
432     rows = []
433     for metric in metrics:
434         sample = long.loc[long["indicator_key"] == metric].iloc[0]
435         rows.append({
436             "一级指标": DIMENSION_NAMES[dimensions[metric]],
437             "二级指标": metric_names[metric],
438             "原数据字段": metric,
439             "指标含义": METRIC_MEANINGS[metric],
440             "单位": sample["score_unit"],
441             "正负向": "正向" if directions[metric] else "负向",
442             "是否保留": "是",
443             "问题1用途": "CRITIC赋权与TOPSIS综合评价",
444         })
445     return pd.DataFrame(rows)
446
447
448 def run_analysis(root: Path, output_dir: Path) -> dict[str, object]:
449     long, quality, attributes = load_frozen_data(root)
450     models, metrics, model_names, metric_names, dimensions, directions = ordered_metadata(
451         long)
452     raw = raw_matrix(long, models, metrics)
453     quality_report = data_quality_report(long, raw, quality, metrics, directions)
454     normalized = minmax_normalize(raw, directions)
455     pearson, spearman, counts = correlation_matrices(raw)

```

```

455 pairs, selected, removed, selected_metrics = select_metrics(
456     raw, pearson, spearman, counts, metrics, metric_names, dimensions
457 )
458 weights_frame = critic_weights(normalized, selected_metrics)
459 weights = weights_frame.set_index("metric")["weight"].to_dict()
460 ranking = full_ranking(normalized, selected_metrics, weights, model_names)
461 kimi = kimi_metric_analysis(raw, normalized, selected_metrics, metric_names, dimensions
    , model_names)
462 sensitivity_rows = threshold_sensitivity(
463     normalized,
464     selected_metrics,
465     weights,
466     model_names,
467     pearson,
468     spearman,
469 )
470 monte_carlo_rows, stability = monte_carlo_sensitivity(normalized, selected_metrics,
    weights, model_names)
471 sensitivity = pd.DataFrame(sensitivity_rows + monte_carlo_rows)
472 indicator_system = final_indicator_system(selected_metrics, metric_names, dimensions,
    directions, long)
473
474 normalized_out = normalized.copy()
475 normalized_out.insert(0, "model", [model_names[index] for index in normalized_out.index
    ])
476 normalized_out.insert(0, "model_id", normalized_out.index)
477 normalized_out = normalized_out.reset_index(drop=True)
478 correlation_columns = ["metric"] + metrics
479 pearson_out = pearson.copy().rename_axis("metric").reset_index()[correlation_columns]
480 spearman_out = spearman.copy().rename_axis("metric").reset_index()[correlation_columns]
481 weights_frame.insert(1, "indicator", weights_frame["metric"].map(metric_names))
482 weights_frame.insert(2, "dimension", weights_frame["metric"].map(dimensions))
483
484 outputs = {
485     "data_quality_report.csv": quality_report,
486     "normalized_data.csv": normalized_out,
487     "pearson_correlation.csv": pearson_out,
488     "spearman_correlation.csv": spearman_out,
489     "high_correlation_pairs.csv": pairs,
490     "selected_metrics.csv": selected,
491     "removed_metrics.csv": removed,
492     "final_indicator_system.csv": indicator_system,
493     "critic_weights.csv": weights_frame,
494     "topsis_ranking.csv": ranking,
495     "kimi_k3_metric_analysis.csv": kimi,
496     "sensitivity_analysis.csv": sensitivity,
497     "rank_stability.csv": stability,
498 }
499 for name, frame in outputs.items():
500     write_csv(frame, output_dir / name)
501
502 if not math.isclose(float(weights_frame["weight"].sum()), 1.0, abs_tol=1e-10):
503     raise AssertionError("CRITIC weights do not sum to one")
504 ranked_scores = ranking.loc[ranking["ranking_status"] == "ranked_complete_case", "

```

```

topsis_score"]
505 if not ranked_scores.between(0, 1).all():
506     raise AssertionError("TOPSIS scores must be within [0, 1]")
507 if ranking["model_id"].nunique() != len(models):
508     raise AssertionError("A model was lost or duplicated")
509
510 return {
511     "long": long,
512     "attributes": attributes,
513     "raw": raw,
514     "normalized": normalized,
515     "pearson": pearson,
516     "spearman": spearman,
517     "pairs": pairs,
518     "selected": selected,
519     "weights": weights_frame,
520     "ranking": ranking,
521     "kimi": kimi,
522     "stability": stability,
523     "indicator_system": indicator_system,
524     "models": models,
525     "metrics": metrics,
526     "selected_metrics": selected_metrics,
527     "model_names": model_names,
528     "metric_names": metric_names,
529     "dimensions": dimensions,
530     "directions": directions,
531 }

```

source/src/q1/glm_supplement.py

```

1  """GLM-5.2 supplemental analysis required by model(1).md section 2.6."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import matplotlib.pyplot as plt
8  import numpy as np
9  import pandas as pd
10
11  from src.q1.analysis import write_csv
12  from src.q1.visualization import configure_style, save
13
14  METRICS = {
15      "gpqa_diamond": "GPQA Diamond",
16      "aa_lcr": "AA-LCR",
17      "scicode": "SciCode",
18      "gdpval_aa_v2": "GDPval-AA v2",
19  }
20
21
22  def run_analysis(root: Path, output_dir: Path) -> dict[str, object]:

```

```

23 matrix = pd.read_csv(root / "data/processed/core_benchmark_matrix.csv", encoding="utf
-8-sig")
24 raw = pd.read_csv(root / "data/raw/benchmark_scores.csv", encoding="utf-8-sig")
25 names = matrix.set_index("model_id")["model_name"].to_dict()
26 rows = []
27 for metric, indicator in METRICS.items():
28     values = matrix.set_index("model_id")[metric].dropna().astype(float)
29     glm_value = float(values.loc["glm-5.2"])
30     leader_id = str(values.idxmax())
31     rank = int((values > glm_value).sum()) + 1
32     rows.append({
33         "metric": metric,
34         "indicator": indicator,
35         "glm_score": glm_value,
36         "score_unit": "elo" if metric == "gdpval_aa_v2" else "percent",
37         "rank_among_six": rank,
38         "models_compared": len(values),
39         "leader_model_id": leader_id,
40         "leader_model": names[leader_id],
41         "leader_score": float(values.loc[leader_id]),
42         "gap_to_leader": float(values.loc[leader_id] - glm_value),
43         "relative_gap_to_leader_percent": float(100 * (values.loc[leader_id] -
glm_value) / values.loc[leader_id]),
44         "comparison_scope": "frozen_main_cohort_directly_comparable",
45     })
46 comparison = pd.DataFrame(rows)
47 official = raw.loc[
48     (raw["model_id"] == "glm-5.2")
49     & (raw["benchmark"] == "HLE-Full (no tools)")
50     & (raw["source_name"] == "Z.ai GLM-5.2 official launch report")
51 ].copy()
52 if len(official) != 1 or float(official.iloc[0]["score"]) != 40.5:
53     raise AssertionError("Expected one GLM-5.2 official HLE no-tools score of 40.5")
54 hle = pd.DataFrame([
55     {
56         "model_id": "glm-5.2",
57         "model": "GLM-5.2",
58         "benchmark": "HLE-Full (no tools)",
59         "official_score": 40.5,
60         "score_unit": "percent",
61         "source_name": official.iloc[0]["source_name"],
62         "source_url": official.iloc[0]["source_url"],
63         "test_setting": official.iloc[0]["test_setting"],
64         "directly_comparable_to_frozen_cohort": False,
65         "usage_rule": "qualitative_scale_reference_only; never fill frozen-cohort NA or
assign rank",
66     }
67 ])
68 write_csv(comparison, output_dir / "glm_partial_comparison.csv")
69 write_csv(hle, output_dir / "glm_official_hle_note.csv")
70 return {"matrix": matrix, "comparison": comparison, "hle": hle, "names": names}
71
72 def generate_figure(result: dict[str, object], output_dir: Path) -> Path:
73     configure_style()
74     matrix: pd.DataFrame = result["matrix"] # type: ignore[assignment]

```

```

74 comparison: pd.DataFrame = result["comparison"] # type: ignore[assignment]
75 names: dict[str, str] = result["names"] # type: ignore[assignment]
76 model_order = matrix["model_id"].tolist()
77 fig, axes = plt.subplots(2, 2, figsize=(13, 8.5))
78 for ax, (metric, indicator) in zip(axes.flat, METRICS.items()):
79     values = matrix.set_index("model_id")[metric].reindex(model_order).astype(float)
80     colors = ["#dc2626" if model == "glm-5.2" else "#94a3b8" for model in model_order]
81     bars = ax.bar([names[model] for model in model_order], values, color=colors)
82     glm_row = comparison.loc[comparison["metric"] == metric].iloc[0]
83     ax.bar_label(bars, fmt="%.1f", padding=2, fontsize=8)
84     ax.set_title(f"{indicator} (GLM 第{int(glm_row['rank_among_six'])}/{int(glm_row['models_compared'])})")
85     ax.tick_params(axis="x", rotation=24, labelsz=8)
86     if metric == "gdpval_aa_v2":
87         ax.axhline(1000, color="#2563eb", linestyle="--", linewidth=1.2, label="Human baseline=1000")
88         ax.legend(fontsize=8)
89         ax.set_ylabel("Elo")
90     else:
91         ax.set_ylabel("得分 (%)")
92     ax.spines[["top", "right"]].set_visible(False)
93 fig.suptitle("GLM-5.2 四项可比文本能力局部定位", fontsize=18, fontweight="bold")
94 fig.tight_layout(rect=(0, 0, 1, 0.95))
95 path = output_dir / "glm_partial_comparison.png"
96 save(fig, path)
97 return path

```

source/src/q1/reporting.py

```

1 """Generate traceable Markdown materials for Question 1."""
2
3 from __future__ import annotations
4
5 from pathlib import Path
6
7 import pandas as pd
8
9 from .analysis import DIMENSION_NAMES, METRIC_MEANINGS, MONTE_CARLO_RUNS, RANDOM_SEED
10
11
12 def fmt(value: float, digits: int = 4) -> str:
13     return f"{value:.{digits}f}"
14
15
16 def write_data_dictionary(result: dict[str, object], docs_dir: Path) -> Path:
17     long: pd.DataFrame = result["long"] # type: ignore[assignment]
18     attributes: pd.DataFrame = result["attributes"] # type: ignore[assignment]
19     metrics: list[str] = result["metrics"] # type: ignore[assignment]
20     metric_names: dict[str, str] = result["metric_names"] # type: ignore[assignment]
21     dimensions: dict[str, str] = result["dimensions"] # type: ignore[assignment]
22     directions: dict[str, bool] = result["directions"] # type: ignore[assignment]
23     lines = [
24         "# 数据字典",

```

```

25     "",
26     "问题1建模输入为 `data/processed/core_benchmark_long.csv`。每行表示一个“最终模型—冻结
指标”观测；模型标识和来源字段为文本，`score` 为数值或 `NA`。成本、时延和吞吐来自 `
model_attributes.csv`，只为问题3预留，不进入问题1纯性能排名。",
27     "",
28     "## 问题1性能指标",
29     "",
30     "| 指标名称 | 英文/代码字段名 | 指标含义 | 指标类别 | 单位 | 方向 | 数据来源 | 是否进入问
题1 | 备注 |",
31     "|---|---|---|---|---|---|---|---|---|",
32 ]
33 for metric in metrics:
34     row = long.loc[long["indicator_key"] == metric].iloc[0]
35     source = str(row["source_name"])
36     lines.append(
37         f"| {metric_names[metric]} | `{metric}` | {METRIC_MEANINGS[metric]} | {
DIMENSION_NAMES[dimensions[metric]]} | "
38         f"{row['score_unit']} | {'正向' if directions[metric] else '负向'} | {source} |
是 | 缺失不插补；按冻结 cohort 比较 |"
39     )
40 lines += [
41     "",
42     "## 标识、来源与工程字段",
43     "",
44     "| 字段名 | 含义 | 类别 | 单位 | 方向 | 是否进入问题1 | 备注 |",
45     "|---|---|---|---|---|---|---|",
46     "| `model_id` / `model_name` | 模型唯一标识与展示名 | 文本标识 | — | — | 仅保留标识 |
不参与数值计算 |",
47     "| `source_name` / `source_url` / `retrieval_date` | 指标来源与检索日期 | 来源字段 |
— | — | 否 | 用于追溯 |",
48     "| `input_price_usd_per_million` / `output_price_usd_per_million` | API 输入/输出价
格 | 成本 | USD/1M tokens | 负向 | 否 | 问题3使用 |",
49     "| `comparable_ttft_seconds` | 首 token 延迟 | 工程效率 | seconds | 负向 | 否 | 问题3
使用；仅 compatible 记录 |",
50     "| `comparable_output_speed_tokens_per_second` | 输出速度 | 工程效率 | tokens/s | 正
向 | 否 | 问题3使用；仅 compatible 记录 |",
51     "| `comparable_total_latency_seconds` | 标准响应总延迟 | 工程效率 | seconds | 负向 |
否 | 问题3使用；仅 compatible 记录 |",
52     "",
53     f"当前工程属性表含 {len(attributes)} 个最终模型。仓库没有可严格横向比较的能耗字段，因此问
题1和现阶段问题3接口均不虚构能耗数值。",
54 ]
55 path = docs_dir / "data_dictionary.md"
56 path.parent.mkdir(parents=True, exist_ok=True)
57 path.write_text("\n".join(lines) + "\n", encoding="utf-8")
58 return path
59
60
61 def write_metric_selection(result: dict[str, object], docs_dir: Path) -> Path:
62     pairs: pd.DataFrame = result["pairs"] # type: ignore[assignment]
63     pearson: pd.DataFrame = result["pearson"] # type: ignore[assignment]
64     spearman: pd.DataFrame = result["spearman"] # type: ignore[assignment]
65     selected_metrics: list[str] = result["selected_metrics"] # type: ignore[assignment]
66     selected: pd.DataFrame = result["selected"] # type: ignore[assignment]

```

```

67 lines = [
68     "# 问题1指标筛选说明",
69     "",
70     "## 可复现规则",
71     "",
72     "1. 仅使用 Phase 2 已冻结且覆盖率不低于 75% 的指标；不重新采集、不插补。",
73     "2. 排除常数列，并检查单位与方向；本轮九项均为正向能力指标。",
74     "3. 同时计算 Pearson 与 Spearman；任一绝对值达到 0.85 即进入冗余复核。",
75     "4. 复核覆盖率、样本标准差、语义维度、来源和后续场景用途。由于最终样本仅 6 个、含缺失指标的
共同样本仅 5 个，高相关不作为自动删除依据。",
76     "5. 只有在语义重复、来源/口径不弱于替代指标且删除不损害场景能力表示时才删除。本轮没有指标同
时满足这些条件。",
77     "",
78     f"初始指标 {len(selected)} 项，删除 0 项，最终保留 {len(selected)} 项。`
removed_metrics.csv` 因此只有表头，这是明确筛选结论而非漏算。",
79     "",
80     "## 高相关候选对（阈值 0.85）",
81     "",
82     "| 指标A | 指标B | Pearson | Spearman | 共同模型 | 决策 |",
83     "|---|---|---:|---:|---:|---|",
84 ]
85 for row in pairs.itertuples():
86     lines.append(f"| {row.metric_a_name} | {row.metric_b_name} | {row.pearson_r:.4f} |
{row.spearman_r:.4f} | {row.pairwise_models} | 保留二者 |")
87 lines += [
88     "",
89     "这些高相关关系跨越不同能力构念，且共同样本量很小。机械删除会把科学推理、多模态或科研文档能
力错误地视作同一指标；因此主模型保留九项，并在 0.80/0.85/0.90 阈值下重复检查排名。",
90 ]
91 path = docs_dir / "q1_metric_selection.md"
92 path.write_text("\n".join(lines) + "\n", encoding="utf-8")
93 return path
94
95
96 def write_results_summary(result: dict[str, object], docs_dir: Path) -> Path:
97     weights: pd.DataFrame = result["weights"] # type: ignore[assignment]
98     ranking: pd.DataFrame = result["ranking"] # type: ignore[assignment]
99     kimi: pd.DataFrame = result["kimi"] # type: ignore[assignment]
100     stability: pd.DataFrame = result["stability"] # type: ignore[assignment]
101     pairs: pd.DataFrame = result["pairs"] # type: ignore[assignment]
102     pearson: pd.DataFrame = result["pearson"] # type: ignore[assignment]
103     spearman: pd.DataFrame = result["spearman"] # type: ignore[assignment]
104     selected_metrics: list[str] = result["selected_metrics"] # type: ignore[assignment]
105     ranked = ranking.loc[ranking["ranking_status"] == "ranked_complete_case"].sort_values("
rank")
106     kimi_rank = int(ranked.loc[ranked["model_id"] == "kimi-k3", "rank"].iloc[0])
107     strengths = kimi.loc[kimi["assessment"] == "advantage"].sort_values(["rank", "
gap_to_leader_normalized"])
108     weaknesses = kimi.loc[kimi["assessment"] == "weakness"].sort_values("
gap_to_leader_normalized", ascending=False)
109     largest_gap = kimi.sort_values("gap_to_leader_normalized", ascending=False).iloc[0]
110     kimi_stability = stability.loc[stability["model_id"] == "kimi-k3"].iloc[0]
111     threshold_pair_counts = {
112         threshold: sum(

```

```

113         max(
114             abs(float(pearson.loc[left_metric, right_metric])),
115             abs(float(spearman.loc[left_metric, right_metric])),
116         ) >= threshold
117         for left_index, left_metric in enumerate(selected_metrics)
118         for right_metric in selected_metrics[left_index + 1 :]
119     )
120     for threshold in (0.80, 0.85, 0.90)
121 }
122 lines = [
123     "# 问题1建模结果摘要",
124     "",
125     "## 1. 模型选择理由",
126     "",
127     "Pearson 用于识别线性冗余, Spearman 用于检验对异常值和非线性单调关系更稳健的秩相关; 二者共
同形成指标复核清单。CRITIC 在方向统一的 Min - Max 数据上同时利用离散程度与指标冲突性确定客观权
重, 避免人为迎合排名。TOPSIS 再计算各模型到正、负理想解的距离, 将多指标信息汇总为可解释的相对接
近度。",
128     "",
129     "## 2. 指标筛选结果",
130     "",
131     f"问题1从 Phase 2 冻结的 9 项核心指标开始。0.85 阈值下发现 {len(pairs)} 组高相关候选, 但
共同样本仅 5 个且语义维度不同, 故删除 0 项、保留 9 项。具体数值和理由见 `
high_correlation_pairs.csv` 与 `q1_metric_selection.md`。",
132     "",
133     "## 3. CRITIC 权重",
134     "",
135     "| 排名 | 指标 | 权重 | 标准差 | 冲突性 |",
136     "|---:|---|---:|---:|---:|",
137 ]
138 for row in weights.head(5).itertuples():
139     lines.append(f"| {row.rank} | {row.indicator} | {row.weight:.6f} | {row.std:.6f} |
{row.conflict:.6f} |")
140 top = weights.iloc[0]
141 low = weights.sort_values("weight").iloc[0]
142 lines += [
143     "",
144     f"最高权重为 {top['indicator']} ({top['weight']:.6f}), 其标准化离散度与对其他指标的冲突
性乘积最大; 最低为 {low['indicator']} ({low['weight']:.6f})。权重只反映当前样本的信息贡献,
不代表永久重要性。",
145     "",
146     "## 4. 综合排名",
147     "",
148     "| Rank | Model | Score |",
149     "|---:|---|---:|",
150 ]
151 for row in ranked.itertuples():
152     lines.append(f"| {int(row.rank)} | {row.model} | {row.topsis_score:.10f} |")
153 unranked = ranking.loc[ranking["ranking_status"] != "ranked_complete_case"]
154 if not unranked.empty:
155     row = unranked.iloc[0]
156     lines.append(f"\n{row['model']} 仅覆盖 {row['available_indicators']}/{row['
total_indicators']} 项, 未插补且不参与主排名。")
157 lines += [

```

```

158     "",
159     "## 5. Kimi K3",
160     "",
161     f"Kimi K3 综合排名第 {kimi_rank}。优势指标为: " + "、".join(str(item) for item in
strengths["indicator"].tolist()) + "。",
162     ("主要短板为: " + "、".join(str(item) for item in weaknesses["indicator"].tolist())
+ "。") if not weaknesses.empty else "没有被组合规则判定为显著短板的指标。",
163     f"与单指标最优模型差距最大的是 {largest_gap['indicator']}, 标准化差距为 {largest_gap['
gap_to_leader_normalized']:.6f}, 领先模型为 {largest_gap['leader_model']}。综合定位是长文
本与文档理解突出、整体第二, 但极高难度开放推理仍明显落后于领先者。",
164     "",
165     "## 6. 稳健性",
166     "",
167     "相关性阈值调整为 0.80、0.85、0.90 时, 高相关候选对分别为 "
168     f"{threshold_pair_counts[0.80]}、{threshold_pair_counts[0.85]}、{
threshold_pair_counts[0.90]} 对; "
169     "语义复核规则均保留九项指标, 主排名不变。",
170     f"固定随机种子 {RANDOM_SEED}, 对 CRITIC 权重独立施加  $\pm 10\%$  均匀扰动并重新归一化, 共运行 {
MONTE_CARLO_RUNS} 次。Kimi K3 平均排名 {kimi_stability['mean_rank']:.4f}、排名标准差 {
kimi_stability['rank_std']:.4f}、Top3 概率 {kimi_stability['top3_probability']:.2%}。",
171     "",
172     "## 给问题2的输入",
173     "",
174     "问题2可直接读取 `results/q1/selected_metrics.csv`、`final_indicator_system.csv`、`
normalized_data.csv` 和 `critic_weights.csv`, 在此基础上叠加三个场景的主观偏好或组合权重。
",
175     "",
176     "## 给问题3的输入",
177     "",
178     "问题3可读取 `results/q1/topsis_ranking.csv` 的综合性能得分, 再按 `model_id` 连接 `
data/processed/model_attributes.csv` 中的价格和 compatible 效率字段。当前没有严格可比能耗数
据, 不应自行补造。",
179 ]
180 path = docs_dir / "q1_results_summary.md"
181 path.write_text("\n".join(lines) + "\n", encoding="utf-8")
182 return path
183
184
185 def generate_documents(result: dict[str, object], docs_dir: Path) -> list[Path]:
186     docs_dir.mkdir(parents=True, exist_ok=True)
187     return [
188         write_data_dictionary(result, docs_dir),
189         write_metric_selection(result, docs_dir),
190         write_results_summary(result, docs_dir),
191     ]

```

source/src/q1/visualization.py

```

1 """Paper-ready 300 dpi PNG figures for Question 1."""
2
3 from __future__ import annotations
4
5 import math

```

```

6 from pathlib import Path
7
8 import matplotlib
9
10 matplotlib.use("Agg")
11 import matplotlib.pyplot as plt
12 import numpy as np
13 import pandas as pd
14
15 from .analysis import DIMENSION_NAMES
16
17
18 def configure_style() -> None:
19     plt.rcParams.update({
20         "font.family": "sans-serif",
21         "font.sans-serif": ["Microsoft YaHei", "SimHei", "Arial Unicode MS", "DejaVu Sans
22         "],
23         "axes.unicode_minus": False,
24         "figure.facecolor": "white",
25         "axes.facecolor": "white",
26         "axes.edgecolor": "#334155",
27         "axes.titleweight": "bold",
28         "axes.grid": False,
29     })
30
31 def save(fig: plt.Figure, path: Path) -> None:
32     path.parent.mkdir(parents=True, exist_ok=True)
33     fig.savefig(path, dpi=300, bbox_inches="tight", facecolor="white")
34     plt.close(fig)
35
36
37 def heatmap(matrix: pd.DataFrame, labels: list[str], title: str, path: Path) -> None:
38     size = max(8.0, 0.82 * len(labels) + 2.2)
39     fig, ax = plt.subplots(figsize=(size, size * 0.86))
40     image = ax.imshow(matrix.to_numpy(dtype=float), cmap="RdBu_r", vmin=-1, vmax=1)
41     ax.set_xticks(range(len(labels)), labels=labels, rotation=42, ha="right")
42     ax.set_yticks(range(len(labels)), labels=labels)
43     for i in range(len(labels)):
44         for j in range(len(labels)):
45             value = matrix.iloc[i, j]
46             color = "white" if abs(value) >= 0.65 else "#172033"
47             ax.text(j, i, f"{value:.2f}", ha="center", va="center", fontsize=8, color=color
48         )
49     ax.set_title(title, pad=16)
50     colorbar = fig.colorbar(image, ax=ax, fraction=0.046, pad=0.04)
51     colorbar.set_label("相关系数")
52     fig.tight_layout()
53     save(fig, path)
54
55 def horizontal_bar(labels: list[str], values: list[float], title: str, xlabel: str, path:
56     Path, highlight: str | None = None) -> None:
57     order = np.argsort(values)

```

```

57 labels_ordered = [labels[index] for index in order]
58 values_ordered = [values[index] for index in order]
59 colors = ["#dc2626" if label == highlight else "#2563eb" for label in labels_ordered]
60 fig, ax = plt.subplots(figsize=(9.2, max(4.8, 0.52 * len(labels) + 1.8)))
61 bars = ax.barh(labels_ordered, values_ordered, color=colors, alpha=0.9)
62 ax.bar_label(bars, labels=[f"{value:.4f}" for value in values_ordered], padding=4,
63              fontsize=9)
64 ax.set_title(title, pad=14)
65 ax.set_xlabel(xlabel)
66 ax.set_xlim(0, max(values_ordered) * 1.16 if values_ordered else 1)
67 ax.spines[["top", "right"]].set_visible(False)
68 fig.tight_layout()
69 save(fig, path)
70
71 def radar_chart(
72     normalized: pd.DataFrame,
73     weights: pd.DataFrame,
74     dimensions: dict[str, str],
75     ranking: pd.DataFrame,
76     model_names: dict[str, str],
77     path: Path,
78 ) -> None:
79     ranked = ranking.loc[ranking["ranking_status"] == "ranked_complete_case"].sort_values("rank")
80     leader_id = str(ranked.iloc[0]["model_id"])
81     complete_ids = ranked["model_id"].tolist()
82     metric_weights = weights.set_index("metric")["weight"].to_dict()
83     dimension_order = list(dict.fromkeys(dimensions.values()))
84
85     def aggregate(model_id: str) -> list[float]:
86         result = []
87         for dimension in dimension_order:
88             metric_list = [metric for metric, value in dimensions.items() if value ==
89                             dimension]
90             denom = sum(metric_weights[metric] for metric in metric_list)
91             result.append(sum(normalized.loc[model_id, metric] * metric_weights[metric] for
92                                metric in metric_list) / denom)
93         return result
94
95     kimi = aggregate("kimi-k3")
96     leader = aggregate(leader_id)
97     average = np.mean([aggregate(model_id) for model_id in complete_ids], axis=0).tolist()
98     labels = [DIMENSION_NAMES[item] for item in dimension_order]
99     angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False).tolist()
100     angles += angles[:1]
101     fig, ax = plt.subplots(figsize=(9, 8), subplot_kw={"polar": True})
102     for values, label, color, width in (
103         (kimi, "Kimi K3", "#dc2626", 2.7),
104         (leader, model_names[leader_id], "#2563eb", 2.2),
105         (average, "完整模型均值", "#64748b", 1.8),
106     ):
107         closed = values + values[:1]
108         ax.plot(angles, closed, color=color, linewidth=width, label=label)

```

```

107         ax.fill(angles, closed, color=color, alpha=0.07)
108     ax.set_xticks(angles[:-1], labels, fontsize=9)
109     ax.set_ylim(0, 1)
110     ax.set_yticks([0.2, 0.4, 0.6, 0.8, 1.0])
111     ax.set_yticklabels(["0.2", "0.4", "0.6", "0.8", "1.0"], fontsize=8)
112     ax.set_title("Kimi K3 一级能力维度对比", pad=24)
113     ax.legend(loc="upper right", bbox_to_anchor=(1.28, 1.14), frameon=False)
114     fig.tight_layout()
115     save(fig, path)
116
117
118 def gap_chart(kimi: pd.DataFrame, path: Path) -> None:
119     frame = kimi.sort_values("gap_to_leader_normalized", ascending=True)
120     colors = frame["assessment"].map({"advantage": "#16a34a", "middle": "#64748b", "
121     weakness": "#dc2626"}).tolist()
122     fig, ax = plt.subplots(figsize=(9.4, 6.4))
123     bars = ax.barh(frame["indicator"], frame["gap_to_leader_normalized"], color=colors)
124     ax.bar_label(bars, labels=[f"{value:.3f}" for value in frame["gap_to_leader_normalized
125     "]], padding=4, fontsize=9)
126     ax.set_xlim(0, 1.05)
127     ax.set_xlabel("与该指标最优模型的标准化差距（越小越好）")
128     ax.set_title("Kimi K3 单指标差距")
129     ax.spines[["top", "right"]].set_visible(False)
130     fig.tight_layout()
131     save(fig, path)
132
133 def generate_figures(result: dict[str, object], output_dir: Path) -> list[Path]:
134     configure_style()
135     metric_names: dict[str, str] = result["metric_names"] # type: ignore[assignment]
136     metrics: list[str] = result["metrics"] # type: ignore[assignment]
137     labels = [metric_names[item] for item in metrics]
138     paths = [
139         output_dir / "pearson_heatmap.png",
140         output_dir / "spearman_heatmap.png",
141         output_dir / "critic_weights_bar.png",
142         output_dir / "topsis_ranking_bar.png",
143         output_dir / "kimi_k3_radar.png",
144         output_dir / "kimi_k3_gap.png",
145     ]
146     heatmap(result["pearson"], labels, "问题1指标 Pearson 相关性", paths[0]) # type: ignore
147     [arg-type]
148     heatmap(result["spearman"], labels, "问题1指标 Spearman 相关性", paths[1]) # type:
149     ignore[arg-type]
150     weights: pd.DataFrame = result["weights"] # type: ignore[assignment]
151     horizontal_bar(weights["indicator"].tolist(), weights["weight"].tolist(), "CRITIC 客观权
152     重", "权重", paths[2])
153     ranking: pd.DataFrame = result["ranking"] # type: ignore[assignment]
154     ranked = ranking.loc[ranking["ranking_status"] == "ranked_complete_case"]
155     horizontal_bar(
156         ranked["model"].tolist(), ranked["topsis_score"].tolist(),
157         "CRITIC - TOPSIS 综合性能排名", "TOPSIS 相对接近度", paths[3], highlight="Kimi K3",
158     )
159     radar_chart(

```

```

156         result["normalized"], weights, result["dimensions"], ranking,
157         result["model_names"], paths[4], # type: ignore[arg-type]
158     )
159     gap_chart(result["kimi"], paths[5]) # type: ignore[arg-type]
160     return paths

```

source/src/q2/__init__.py

```

1 """Question 2 scenario evaluation package."""

```

source/src/q2/analysis.py

```

1 """Question 2 scenario evaluation specified by model(1).md section 3."""
2
3 from __future__ import annotations
4
5 import math
6 from pathlib import Path
7
8 import numpy as np
9 import pandas as pd
10
11 from src.q1.analysis import topsis_scores, write_csv
12
13 OBJECTIVE_SHARE = 0.5
14 PERTURBATION_FACTORS = (0.8, 0.9, 1.1, 1.2)
15 SCENARIO_LABELS = {
16     "research_long_text": "科研长文本分析",
17     "daily_dialogue": "大众日常通用对话",
18     "code_development": "计算机代码开发",
19 }
20
21
22 def kendall_tau(left: dict[str, int], right: dict[str, int]) -> float:
23     concordant = discordant = 0
24     models = sorted(left)
25     for index, first in enumerate(models):
26         for second in models[index + 1 :]:
27             product = (left[first] - left[second]) * (right[first] - right[second])
28             concordant += int(product > 0)
29             discordant += int(product < 0)
30     return (concordant - discordant) / (concordant + discordant)
31
32
33 def load_inputs(root: Path) -> dict[str, object]:
34     normalized = pd.read_csv(root / "results/q1/normalized_data.csv", encoding="utf-8-sig")
35     normalized = normalized.set_index("model_id")
36     weights_frame = pd.read_csv(root / "results/q1/critic_weights.csv", encoding="utf-8-sig")
37     q1_ranking = pd.read_csv(root / "results/q1/topsis_ranking.csv", encoding="utf-8-sig")
38     priorities = pd.read_csv(root / "scripts/phase5_scenario_priorities.csv", encoding="utf-8-sig")
39     metrics = weights_frame["metric"].tolist()

```

```

40 critic = weights_frame.set_index("metric")["weight"].astype(float).to_dict()
41 if not math.isclose(sum(critic.values()), 1.0, abs_tol=1e-10):
42     raise ValueError("Q1 CRITIC weights do not sum to one")
43 if set(priorities["indicator_key"]) != set(metrics):
44     raise ValueError("Scenario priority metrics do not match Q1 final metrics")
45 for scenario, frame in priorities.groupby("scenario"):
46     if len(frame) != len(metrics) or not math.isclose(float(frame["subjective_weight"].
47 sum()), 1.0, abs_tol=1e-10):
48         raise ValueError(f"Subjective weights for {scenario} must cover all metrics and
49 sum to one")
50 return {
51     "normalized": normalized,
52     "weights_frame": weights_frame,
53     "q1_ranking": q1_ranking,
54     "priorities": priorities,
55     "metrics": metrics,
56     "critic": critic,
57 }
58
59 def combined_weights(priorities: pd.DataFrame, critic: dict[str, float]) -> pd.DataFrame:
60     rows: list[dict[str, object]] = []
61     for scenario, group in priorities.groupby("scenario", sort=False):
62         for row in group.itertuples():
63             objective = critic[row.indicator_key]
64             subjective = float(row.subjective_weight)
65             combined = OBJECTIVE_SHARE * objective + (1 - OBJECTIVE_SHARE) * subjective
66             rows.append({
67                 "scenario": scenario,
68                 "scenario_name": SCENARIO_LABELS[scenario],
69                 "metric": row.indicator_key,
70                 "critic_weight": objective,
71                 "subjective_weight": subjective,
72                 "objective_share": OBJECTIVE_SHARE,
73                 "combined_weight": combined,
74                 "weight_change_vs_q1": combined - objective,
75                 "rationale": row.rationale,
76             })
77     frame = pd.DataFrame(rows)
78     totals = frame.groupby("scenario")["combined_weight"].sum()
79     if not np.allclose(totals.to_numpy(), 1.0, atol=1e-10):
80         raise AssertionError("Combined scenario weights do not sum to one")
81     return frame
82
83 def scenario_rankings(
84     normalized: pd.DataFrame,
85     metrics: list[str],
86     weight_frame: pd.DataFrame,
87     q1_ranking: pd.DataFrame,
88 ) -> pd.DataFrame:
89     names = q1_ranking.set_index("model_id")["model"].to_dict()
90     q1_ranks = q1_ranking.set_index("model_id")["rank"].to_dict()
91     rows: list[dict[str, object]] = []

```

```

92 for scenario, group in weight_frame.groupby("scenario", sort=False):
93     weights = group.set_index("metric")["combined_weight"].to_dict()
94     ranked = topsis_scores(normalized, metrics, weights)
95     ranked_by_id = ranked.set_index("model_id").to_dict("index")
96     for model_id in normalized.index:
97         base = {
98             "scenario": scenario,
99             "scenario_name": SCENARIO_LABELS[scenario],
100             "model_id": model_id,
101             "model": names[model_id],
102             "q1_rank": q1_ranks.get(model_id, np.nan),
103         }
104         if model_id in ranked_by_id:
105             item = ranked_by_id[model_id]
106             base.update(item)
107             base["rank_change_vs_q1"] = int(item["rank"] - q1_ranks[model_id])
108             base["rank_improvement_vs_q1"] = int(q1_ranks[model_id] - item["rank"])
109             base["ranking_status"] = "ranked_complete_case"
110         else:
111             base.update({
112                 "distance_positive": np.nan,
113                 "distance_negative": np.nan,
114                 "topsis_score": np.nan,
115                 "rank": np.nan,
116                 "rank_change_vs_q1": np.nan,
117                 "rank_improvement_vs_q1": np.nan,
118                 "ranking_status": "not_ranked_insufficient_coverage",
119             })
120         rows.append(base)
121     return pd.DataFrame(rows).sort_values(["scenario", "rank", "model_id"], na_position="
122     last").reset_index(drop=True)
123
124 def rank_change_analysis(rankings: pd.DataFrame, q1_ranking: pd.DataFrame) -> pd.DataFrame:
125     ranked = rankings.loc[rankings["ranking_status"] == "ranked_complete_case"]
126     pivot = ranked.pivot(index=["model_id", "model", "q1_rank"], columns="scenario", values
127     ="rank").reset_index()
128     rows = []
129     scenario_keys = list(SCENARIO_LABELS)
130     for row in pivot.itertuples(index=False):
131         values = {scenario: int(getattr(row, scenario)) for scenario in scenario_keys}
132         best_rank, worst_rank = min(values.values()), max(values.values())
133         best = "\ ".join(SCENARIO_LABELS[key] for key, value in values.items() if value ==
134         best_rank)
135         worst = "\ ".join(SCENARIO_LABELS[key] for key, value in values.items() if value ==
136         worst_rank)
137         ranks_all = [int(row.q1_rank), *values.values()]
138         rows.append({
139             "model_id": row.model_id,
140             "model": row.model,
141             "q1_rank": int(row.q1_rank),
142             "research_rank": values["research_long_text"],
143             "dialogue_rank": values["daily_dialogue"],
144             "coding_rank": values["code_development"],

```

```

142         "research_rank_change": values["research_long_text"] - int(row.q1_rank),
143         "dialogue_rank_change": values["daily_dialogue"] - int(row.q1_rank),
144         "coding_rank_change": values["code_development"] - int(row.q1_rank),
145         "best_scenario": best,
146         "worst_scenario": worst,
147         "rank_range": max(ranks_all) - min(ranks_all),
148         "rank_std": float(np.std(ranks_all, ddof=0)),
149         "scenario_sensitivity": "场景敏感型" if max(ranks_all) - min(ranks_all) > 0 else
"场景稳定型",
150     })
151     return pd.DataFrame(rows).sort_values(["rank_range", "q1_rank"], ascending=[False, True
]).reset_index(drop=True)
152
153
154 def contribution_analysis(
155     normalized: pd.DataFrame,
156     metrics: list[str],
157     weights: pd.DataFrame,
158     rankings: pd.DataFrame,
159     metric_names: dict[str, str],
160 ) -> pd.DataFrame:
161     rows = []
162     rank_lookup = rankings.set_index(["scenario", "model_id"])[["topsis_score", "rank"]].
to_dict("index")
163     for item in weights.itertuples():
164         value = float(normalized.loc["kimi-k3", item.metric])
165         result = rank_lookup[(item.scenario, "kimi-k3")]
166         rows.append({
167             "scenario": item.scenario,
168             "scenario_name": item.scenario_name,
169             "metric": item.metric,
170             "indicator": metric_names[item.metric],
171             "kimi_normalized_value": value,
172             "critic_weight": item.critic_weight,
173             "combined_weight": item.combined_weight,
174             "weight_change_vs_q1": item.weight_change_vs_q1,
175             "weighted_normalized_component": value * item.combined_weight,
176             "weight_shift_profile_effect": value * item.weight_change_vs_q1,
177             "scenario_score": result["topsis_score"],
178             "scenario_rank": result["rank"],
179         })
180     return pd.DataFrame(rows)
181
182
183 def sensitivity_analysis(
184     normalized: pd.DataFrame,
185     metrics: list[str],
186     critic: dict[str, float],
187     priorities: pd.DataFrame,
188     baseline: pd.DataFrame,
189 ) -> tuple[pd.DataFrame, pd.DataFrame]:
190     baseline_rank = {
191         scenario: group.set_index("model_id")["rank"].astype(int).to_dict()
192         for scenario, group in baseline.loc[baseline["ranking_status"] == "

```

```

ranked_complete_case"].groupby("scenario")
193 }
194 rows = []
195 for metric in metrics:
196     for factor in PERTURBATION_FACTORS:
197         adjusted = dict(critic)
198         adjusted[metric] *= factor
199         total = sum(adjusted.values())
200         adjusted = {key: value / total for key, value in adjusted.items()}
201         for scenario, group in priorities.groupby("scenario", sort=False):
202             subjective = group.set_index("indicator_key")["subjective_weight"].astype(
float).to_dict()
203             combined = {
204                 key: OBJECTIVE_SHARE * adjusted[key] + (1 - OBJECTIVE_SHARE) *
subjective[key]
205                 for key in metrics
206             }
207             ranked = topsis_scores(normalized, metrics, combined)
208             ranks = ranked.set_index("model_id")["rank"].astype(int).to_dict()
209             tau = kendall_tau(baseline_rank[scenario], ranks)
210             for result in ranked.itertuples():
211                 rows.append({
212                     "scenario": scenario,
213                     "scenario_name": SCENARIO_LABELS[scenario],
214                     "perturbed_metric": metric,
215                     "factor": factor,
216                     "model_id": result.model_id,
217                     "score": result.topsis_score,
218                     "rank": result.rank,
219                     "baseline_rank": baseline_rank[scenario][result.model_id],
220                     "rank_change": result.rank - baseline_rank[scenario][result.
model_id],
221                     "kendall_tau_vs_scenario_baseline": tau,
222                 })
223 detail = pd.DataFrame(rows)
224 summary_rows = []
225 for (scenario, model_id), group in detail.groupby(["scenario", "model_id"]):
226     summary_rows.append({
227         "scenario": scenario,
228         "scenario_name": SCENARIO_LABELS[scenario],
229         "model_id": model_id,
230         "baseline_rank": int(group["baseline_rank"].iloc[0]),
231         "minimum_rank": int(group["rank"].min()),
232         "maximum_rank": int(group["rank"].max()),
233         "mean_rank": float(group["rank"].mean()),
234         "rank_std": float(group["rank"].std(ddof=0)),
235         "top1_probability": float((group["rank"] <= 1).mean()),
236         "top3_probability": float((group["rank"] <= 3).mean()),
237         "minimum_kendall_tau": float(group["kendall_tau_vs_scenario_baseline"].min()),
238         "scenarios": int(len(group)),
239     })
240 return detail, pd.DataFrame(summary_rows)
241
242

```

```

243 def run_analysis(root: Path, output_dir: Path) -> dict[str, object]:
244     inputs = load_inputs(root)
245     normalized: pd.DataFrame = inputs["normalized"] # type: ignore[assignment]
246     weights_frame: pd.DataFrame = inputs["weights_frame"] # type: ignore[assignment]
247     q1_ranking: pd.DataFrame = inputs["q1_ranking"] # type: ignore[assignment]
248     priorities: pd.DataFrame = inputs["priorities"] # type: ignore[assignment]
249     metrics: list[str] = inputs["metrics"] # type: ignore[assignment]
250     critic: dict[str, float] = inputs["critic"] # type: ignore[assignment]
251     metric_names = weights_frame.set_index("metric")["indicator"].to_dict()
252     weights = combined_weights(priorities, critic)
253     rankings = scenario_rankings(normalized, metrics, weights, q1_ranking)
254     changes = rank_change_analysis(rankings, q1_ranking)
255     kimi = contribution_analysis(normalized, metrics, weights, rankings, metric_names)
256     sensitivity, stability = sensitivity_analysis(normalized, metrics, critic, priorities,
257     rankings)
258     outputs = {
259         "scenario_weights.csv": weights,
260         "scenario_rankings.csv": rankings,
261         "research_ranking.csv": rankings.loc[rankings["scenario"] == "research_long_text"].
262         reset_index(drop=True),
263         "dialogue_ranking.csv": rankings.loc[rankings["scenario"] == "daily_dialogue"].
264         reset_index(drop=True),
265         "coding_ranking.csv": rankings.loc[rankings["scenario"] == "code_development"].
266         reset_index(drop=True),
267         "rank_change_analysis.csv": changes,
268         "kimi_k3_scenario_analysis.csv": kimi,
269         "scenario_sensitivity.csv": sensitivity,
270         "scenario_stability.csv": stability,
271     }
272     for name, frame in outputs.items():
273         write_csv(frame, output_dir / name)
274     return {
275         **inputs,
276         "scenario_weights": weights,
277         "scenario_rankings": rankings,
278         "rank_changes": changes,
279         "kimi": kimi,
280         "sensitivity": sensitivity,
281         "stability": stability,
282         "metric_names": metric_names,
283     }

```

source/src/q2/reporting.py

```

1  """Generate Question 2 paper-writing material."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import pandas as pd
8
9  from src.q2.analysis import OBJECTIVE_SHARE, SCENARIO_LABELS

```

```

10
11
12 def generate_report(result: dict[str, object], docs_dir: Path) -> Path:
13     docs_dir.mkdir(parents=True, exist_ok=True)
14     rankings: pd.DataFrame = result["scenario_rankings"] # type: ignore[assignment]
15     weights: pd.DataFrame = result["scenario_weights"] # type: ignore[assignment]
16     changes: pd.DataFrame = result["rank_changes"] # type: ignore[assignment]
17     kimi: pd.DataFrame = result["kimi"] # type: ignore[assignment]
18     stability: pd.DataFrame = result["stability"] # type: ignore[assignment]
19     lines = [
20         "# 问题2场景化评价结果摘要",
21         "",
22         "## 1. 建模方法及与问题1的衔接",
23         "",
24         "直接复用问题1的九项最终指标、Min - Max 标准化矩阵和 CRITIC 权重, 不重新筛选、标准化或插
25         补。严格按 `model(1).md` §3, 采用 "
26         f"$w*_{{s,j}}={{OBJECTIVE_SHARE}}w_j+{1-OBJECTIVE_SHARE}a_{{s,j}}$" 组合赋权, 再使
27         用与问题1相同的 TOPSIS。GLM-5.2 因仅覆盖 4/9 项继续不参与完整矩阵排名。",
28         "",
29         "## 2. 三类场景权重",
30         ""
31     ]
32     for scenario, label in SCENARIO_LABELS.items():
33         group = weights.loc[weights["scenario"] == scenario].sort_values("combined_weight",
34                                 ascending=False)
35         top = ", ".join(f"{row.metric}={row.combined_weight:.4f}" for row in group.head(3).
36                         itertuples())
37         lines.append(f"- {label}: 权重最高的三项为 {top}; 权重和为 {group['combined_weight'].
38                         sum():.6f}。")
39     for section, (scenario, label) in enumerate(SCENARIO_LABELS.items(), start=3):
40         group = rankings.loc[(rankings["scenario"] == scenario) & (rankings["ranking_status"]
41                                 == "ranked_complete_case")].sort_values("rank")
42         lines += [f"## {section}. {label}排名", "", "| 排名 | 模型 | TOPSIS 得分 | 相对Q1
43         名次变化 |", "|---:|---:|---:|---:|"]
44         for row in group.itertuples():
45             lines.append(f"| {int(row.rank)} | {row.model} | {row.topsis_score:.10f} | {int
46             (row.rank_improvement_vs_q1):+d} |")
47         sensitive = changes.loc[changes["scenario_sensitivity"] == "场景敏感型"]
48         stable = changes.loc[changes["scenario_sensitivity"] == "场景稳定型"]
49         lines += [
50             "",
51             "## 6. 排名变化及机理",
52             "",
53             "场景敏感型模型为: " + ("、".join(sensitive["model"].tolist()) if not sensitive.empty
54             else "无") + "; 场景稳定型模型为: " + ("、".join(stable["model"].tolist()) + "。",
55             "科研场景提升 AA-LCR 与 CharXiv RQ 的组合权重, 而 Kimi K3 在 AA-LCR、OmniDocBench 等指
56             标上的标准化表现领先, 使其由问题1第2升至科研场景第1。代码场景显著提升 SciCode 权重, Claude
57             Fable 5 在该指标取全体最高标准化值, 因此继续保持第1。日常场景提高 GDPval、MMU-Pro 与
58             MathVision 权重, Claude Fable 5 的综合匹配度仍最高。以上权重变化和逐项乘积可在 `
59             kimi_k3_scenario_analysis.csv` 追溯。",
60             "",
61             "## 7. Kimi K3场景表现",
62             "",
63             ""
64         ]

```

```

51     kimi_scenarios = kimi[["scenario", "scenario_name", "scenario_score", "scenario_rank
52     ]].drop_duplicates()
53     for row in kimi_scenarios.itertuples():
54         lines.append(f"- {row.scenario_name}: 得分 {row.scenario_score:.10f}, 排名第 {int(
55         row.scenario_rank)}。")
56     best_rank = kimi_scenarios["scenario_rank"].min()
57     best = "\n".join(kimi_scenarios.loc[kimi_scenarios["scenario_rank"] == best_rank, "
58     scenario_name"])
59     worst_rank = kimi_scenarios["scenario_rank"].max()
60     worst = "\n".join(kimi_scenarios.loc[kimi_scenarios["scenario_rank"] == worst_rank, "
61     scenario_name"])
62     kimi_stability = stability.loc[stability["model_id"] == "kimi-k3"]
63     lines += [
64         f"最强场景为 {best}, 最弱场景为 {worst}。这里按场景内名次判断, 不把不同权重体系下的
65         TOPSIS 绝对值直接解释为跨场景效用大小。",
66         "",
67         "## 8. 稳健性",
68         "",
69         "严格沿用 `model(1).md` §5.2 的 9 项 CRITIC 权重逐项乘以 0.8、0.9、1.1、1.2 后归一化的
70         36 种扰动, 并将扰动后的客观权重重新与固定场景主观权重按 0.5/0.5 组合。",
71         ]
72     for row in kimi_stability.itertuples():
73         lines.append(f"- {row.scenario_name}: Kimi 平均排名 {row.mean_rank:.4f}, 范围 {int(
74         row.minimum_rank)} - {int(row.maximum_rank)}, Top3 概率 {row.top3_probability:.2%}, 最低
75         Kendall  $\tau$ ={row.minimum_kendall_tau:.3f}。")
76     lines += [
77         "",
78         "## 9. 可直接用于论文的结论",
79         "",
80         "固定模型能力不变时, 场景需求通过组合权重改变理想解距离, 从而改变排序。Kimi K3 的长上下文与
81         文档能力与科研长文本场景最匹配; Claude Fable 5 在日常综合与代码开发场景保持首位。该结论是给定
82         主观权重方案下的场景化比较, 不应外推为所有用户的唯一选择。",
83         ]
84     path = docs_dir / "q2_results_summary.md"
85     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
86     return path

```

source/src/q2/visualization.py

```

1  """Paper-ready Question 2 visualizations."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import matplotlib.pyplot as plt
8  import numpy as np
9  import pandas as pd
10
11  from src.q1.visualization import configure_style, save
12  from src.q2.analysis import SCENARIO_LABELS
13
14

```

```

15 def rank_comparison(rankings: pd.DataFrame, q1: pd.DataFrame, path: Path) -> None:
16     ranked_q1 = q1.loc[q1["ranking_status"] == "ranked_complete_case"]
17     stages = ["问题1综合", *SCENARIO_LABELS.values()]
18     scenarios = list(SCENARIO_LABELS)
19     palette = {
20         "claude-fable-5": "#2563eb",
21         "kimi-k3": "#dc2626",
22         "gpt-5.6-sol": "#f59e0b",
23         "gpt-5.5": "#16a34a",
24         "claude-opus-4.8": "#7c3aed",
25     }
26     fig, ax = plt.subplots(figsize=(11, 6.8))
27     for item in ranked_q1.sort_values("rank").itertuples():
28         ranks = [int(item.rank)] + [
29             int(rankings.loc[(rankings["scenario"] == scenario) & (rankings["model_id"] ==
30 item.model_id), "rank"].iloc[0])
31             for scenario in scenarios
32         ]
33         color = palette[item.model_id]
34         ax.plot(stages, ranks, marker="o", linewidth=3 if item.model_id == "kimi-k3" else
35 1.8, label=item.model, color=color)
36         for x, value in enumerate(ranks):
37             ax.text(x, value - 0.10, str(value), ha="center", fontsize=9)
38     ax.set_ylim(5.45, 0.55)
39     ax.set_ylabel("排名（越高越好）")
40     ax.set_title("问题1与三类场景排名变化")
41     ax.grid(axis="y", alpha=0.25)
42     ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
43     fig.tight_layout()
44     save(fig, path)
45
46 def heatmap(frame: pd.DataFrame, path: Path, value_column: str, title: str, fmt: str) ->
47 None:
48     pivot = frame.pivot(index="scenario_name", columns="model", values=value_column)
49     order = [SCENARIO_LABELS[key] for key in SCENARIO_LABELS]
50     pivot = pivot.reindex(order)
51     fig, ax = plt.subplots(figsize=(10.5, 4.7))
52     image = ax.imshow(pivot.to_numpy(float), cmap="YlGnBu", aspect="auto")
53     ax.set_xticks(range(len(pivot.columns)), pivot.columns, rotation=20, ha="right")
54     ax.set_yticks(range(len(pivot.index)), pivot.index)
55     for y in range(len(pivot.index)):
56         for x in range(len(pivot.columns)):
57             value = pivot.iloc[y, x]
58             ax.text(x, y, format(value, fmt), ha="center", va="center", color="white" if
59 value > np.nanmean(pivot.to_numpy()) else "#111827")
60     fig.colorbar(image, ax=ax, shrink=0.8)
61     ax.set_title(title)
62     fig.tight_layout()
63     save(fig, path)
64
65 def rank_change(changes: pd.DataFrame, path: Path) -> None:
66     labels = ["科研长文本", "日常对话", "代码开发"]

```

```

65     columns = ["research_rank_change", "dialogue_rank_change", "coding_rank_change"]
66     x = np.arange(len(changes))
67     width = 0.24
68     fig, ax = plt.subplots(figsize=(11, 6.3))
69     for index, (label, column) in enumerate(zip(labels, columns)):
70         # Positive means rank improved, so invert the stored new-minus-old change.
71         ax.bar(x + (index - 1) * width, -changes[column], width, label=label)
72     ax.axhline(0, color="#334155", linewidth=1)
73     ax.set_xticks(x, changes["model"], rotation=18, ha="right")
74     ax.set_ylabel("相对问题1排名提升（正值为上升）")
75     ax.set_title("场景化评价对模型排名的影响")
76     ax.legend()
77     fig.tight_layout()
78     save(fig, path)
79
80
81 def kimi_radar(kimi: pd.DataFrame, path: Path) -> None:
82     metrics = kimi.loc[kimi["scenario"] == "research_long_text", "indicator"].tolist()
83     angles = np.linspace(0, 2 * np.pi, len(metrics), endpoint=False).tolist()
84     angles += angles[:1]
85     fig, ax = plt.subplots(figsize=(9, 8), subplot_kw={"polar": True})
86     colors = ["#2563eb", "#16a34a", "#dc2626"]
87     for color, (scenario, label) in zip(colors, SCENARIO_LABELS.items()):
88         group = kimi.loc[kimi["scenario"] == scenario]
89         values = group["weighted_normalized_component"].tolist()
90         values += values[:1]
91         ax.plot(angles, values, linewidth=2.3, color=color, label=label)
92         ax.fill(angles, values, alpha=0.08, color=color)
93     ax.set_xticks(angles[:-1], metrics)
94     ax.set_ylim(0, max(0.25, float(kimi["weighted_normalized_component"].max()) * 1.12))
95     ax.set_title("Kimi K3: 标准化能力 × 场景组合权重", pad=25)
96     ax.legend(bbox_to_anchor=(1.28, 1.10))
97     fig.tight_layout()
98     save(fig, path)
99
100
101 def weight_heatmap(weights: pd.DataFrame, path: Path) -> None:
102     pivot = weights.pivot(index="scenario_name", columns="metric", values="combined_weight")
103     order = [SCENARIO_LABELS[key] for key in SCENARIO_LABELS]
104     pivot = pivot.reindex(order)
105     fig, ax = plt.subplots(figsize=(12.5, 4.5))
106     image = ax.imshow(pivot.to_numpy(float), cmap="OrRd", aspect="auto")
107     ax.set_xticks(range(len(pivot.columns)), pivot.columns, rotation=28, ha="right")
108     ax.set_yticks(range(len(pivot.index)), [f"{name} (Σ=1)" for name in pivot.index])
109     for y in range(len(pivot.index)):
110         for x in range(len(pivot.columns)):
111             ax.text(x, y, f"{pivot.iloc[y, x]:.3f}", ha="center", va="center", fontsize=8)
112     fig.colorbar(image, ax=ax, shrink=0.8, label="组合权重")
113     ax.set_title("三类场景组合权重")
114     fig.tight_layout()
115     save(fig, path)
116
117

```

```

118 def generate_figures(result: dict[str, object], output_dir: Path) -> list[Path]:
119     configure_style()
120     paths = [
121         output_dir / "scenario_rank_comparison.png",
122         output_dir / "scenario_score_heatmap.png",
123         output_dir / "rank_change.png",
124         output_dir / "kimi_k3_scenario_radar.png",
125         output_dir / "scenario_weights_heatmap.png",
126     ]
127     rankings: pd.DataFrame = result["scenario_rankings"] # type: ignore[assignment]
128     ranked = rankings.loc[rankings["ranking_status"] == "ranked_complete_case"]
129     rank_comparison(rankings, result["q1_ranking"], paths[0]) # type: ignore[arg-type]
130     heatmap(ranked, paths[1], "topsis_score", "三类场景 TOPSIS 得分", ".3f")
131     rank_change(result["rank_changes"], paths[2]) # type: ignore[arg-type]
132     kimi_radar(result["kimi"], paths[3]) # type: ignore[arg-type]
133     weight_heatmap(result["scenario_weights"], paths[4]) # type: ignore[arg-type]
134     return paths

```

source/src/q3/__init__.py

```

1 """Question 3 performance-cost evaluation package."""

```

source/src/q3/analysis.py

```

1 """Question 3 models specified by model(1).md section 4."""
2
3 from __future__ import annotations
4
5 import math
6 from pathlib import Path
7
8 import numpy as np
9 import pandas as pd
10
11 from src.q1.analysis import topsis_scores, write_csv
12
13 INPUT_MILLION_TOKENS = 1.0
14 OUTPUT_MILLION_TOKENS = 0.2
15 BUDGETS = (6.0, 10.0, 12.0, 16.0, 20.0)
16 PERTURBATION_FACTORS = (0.8, 0.9, 1.1, 1.2)
17
18
19 def benefit_minmax(series: pd.Series, higher_is_better: bool) -> pd.Series:
20     minimum, maximum = float(series.min()), float(series.max())
21     if math.isclose(minimum, maximum):
22         raise ValueError(f"Cannot normalize constant engineering field {series.name}")
23     if higher_is_better:
24         return (series - minimum) / (maximum - minimum)
25     return (maximum - series) / (maximum - minimum)
26
27
28 def pareto_table(performance_cost: pd.DataFrame) -> pd.DataFrame:

```

```

29 included = performance_cost.loc[performance_cost["analysis_status"] == "
included_ranked_model"].copy()
30 rows = []
31 for item in included.itertuples():
32     dominators = included.loc[
33         (included["model_id"] != item.model_id)
34         & (included["standard_workload_cost_usd"] <= item.standard_workload_cost_usd)
35         & (included["q1_performance_score"] >= item.q1_performance_score)
36         & (
37             (included["standard_workload_cost_usd"] < item.standard_workload_cost_usd)
38             | (included["q1_performance_score"] > item.q1_performance_score)
39         )
40     ]
41     rows.append({
42         "model_id": item.model_id,
43         "model": item.model,
44         "q1_performance_score": item.q1_performance_score,
45         "standard_workload_cost_usd": item.standard_workload_cost_usd,
46         "pareto_status": "frontier" if dominators.empty else "dominated",
47         "is_pareto_frontier": dominators.empty,
48         "dominated_by_model_ids": "" if dominators.empty else ";".join(dominators["
model_id"]),
49         "dominated_by_models": "" if dominators.empty else "; ".join(dominators["model
"]),
50     })
51 return pd.DataFrame(rows).sort_values(["standard_workload_cost_usd", "
q1_performance_score"], ascending=[True, False])
52
53
54 def cost_audit(attributes: pd.DataFrame, q1: pd.DataFrame) -> pd.DataFrame:
55     q1_status = q1.set_index("model_id")["ranking_status"].to_dict()
56     rows = []
57     for item in attributes.itertuples():
58         has_prices = pd.notna(item.input_price_usd_per_million) and pd.notna(item.
output_price_usd_per_million)
59         cost = (
60             float(item.input_price_usd_per_million) * INPUT_MILLION_TOKENS
61             + float(item.output_price_usd_per_million) * OUTPUT_MILLION_TOKENS
62             if has_prices else np.nan
63         )
64         rows.append({
65             "model_id": item.model_id,
66             "model": item.model_name,
67             "input_price_usd_per_million": item.input_price_usd_per_million,
68             "output_price_usd_per_million": item.output_price_usd_per_million,
69             "price_unit": "USD / 1M tokens",
70             "workload_input_million_tokens": INPUT_MILLION_TOKENS,
71             "workload_output_million_tokens": OUTPUT_MILLION_TOKENS,
72             "standard_workload_cost_usd": cost,
73             "price_data_complete": has_prices,
74             "efficiency_compatible": bool(item.efficiency_compatible),
75             "comparable_ttft_seconds": item.comparable_ttft_seconds,
76             "comparable_output_speed_tokens_per_second": item.
comparable_output_speed_tokens_per_second,

```

```

77         "comparable_total_latency_seconds": item.comparable_total_latency_seconds,
78         "energy_data_available": False,
79         "q1_ranking_status": q1_status[item.model_id],
80         "q3_cost_analysis_status": "included" if has_prices else "
excluded_missing_price",
81         "notes": "能耗字段不存在; 不估算。效率仅在 compatible=true 时横向比较。",
82     })
83     return pd.DataFrame(rows)
84
85
86 def performance_cost_table(audit: pd.DataFrame, q1: pd.DataFrame) -> pd.DataFrame:
87     q1_columns = q1[["model_id", "topsis_score", "rank", "ranking_status"]].rename(
88         columns={"topsis_score": "q1_performance_score", "rank": "q1_rank"}
89     )
90     frame = audit[["model_id", "model", "standard_workload_cost_usd"]].merge(q1_columns, on
="model_id", how="left")
91     frame["analysis_status"] = np.where(
92         frame["ranking_status"] == "ranked_complete_case",
93         "included_ranked_model",
94         "excluded_insufficient_q1_coverage",
95     )
96     included = frame["analysis_status"] == "included_ranked_model"
97     frame.loc[included, "performance_per_usd"] = (
98         frame.loc[included, "q1_performance_score"] / frame.loc[included, "
standard_workload_cost_usd"]
99     )
100     frame.loc[included, "cost_rank_among_ranked"] = frame.loc[included, "
standard_workload_cost_usd"].rank(method="min").astype(int)
101     frame.loc[included, "value_rank_among_ranked"] = frame.loc[included, "
performance_per_usd"].rank(method="min", ascending=False).astype(int)
102     costs = frame.loc[included, "standard_workload_cost_usd"]
103     frame.loc[included, "cost_benefit_index"] = (costs.max() - costs) / (costs.max() -
costs.min())
104     return frame
105
106
107 def budget_selection(performance_cost: pd.DataFrame) -> pd.DataFrame:
108     included = performance_cost.loc[performance_cost["analysis_status"] == "
included_ranked_model"]
109     rows = []
110     for budget in BUDGETS:
111         feasible = included.loc[included["standard_workload_cost_usd"] <= budget]
112         if feasible.empty:
113             rows.append({"budget_level": budget, "budget_limit_usd": budget, "
feasible_models": 0})
114             continue
115         winner = feasible.sort_values(["q1_performance_score", "standard_workload_cost_usd
"], ascending=[False, True]).iloc[0]
116         rows.append({
117             "budget_level": budget,
118             "budget_limit_usd": budget,
119             "feasible_models": len(feasible),
120             "recommended_model_id": winner["model_id"],
121             "recommended_model": winner["model"],

```

```

122         "performance": winner["q1_performance_score"],
123         "cost_usd": winner["standard_workload_cost_usd"],
124         "unused_budget_usd": budget - winner["standard_workload_cost_usd"],
125         "selection_rule": "预算内Q1 TOPSIS得分最高; 不含Q1未排名模型",
126     })
127     return pd.DataFrame(rows)
128
129
130 def regression(performance_cost: pd.DataFrame) -> pd.DataFrame:
131     frame = performance_cost.loc[performance_cost["analysis_status"] == "
included_ranked_model"].copy()
132     x = np.log(frame["standard_workload_cost_usd"].to_numpy(float))
133     y = frame["q1_performance_score"].to_numpy(float)
134     design = np.column_stack([np.ones(len(x)), x])
135     beta = np.linalg.lstsq(design, y, rcond=None)[0]
136     fitted = design @ beta
137     residuals = y - fitted
138     sse = float(np.sum(residuals**2))
139     sst = float(np.sum((y - y.mean()) ** 2))
140     r_squared = 1 - sse / sst
141     adjusted = 1 - (1 - r_squared) * (len(y) - 1) / (len(y) - 2)
142     rmse = math.sqrt(sse / len(y))
143     frame["ln_cost"] = x
144     frame["fitted_performance"] = fitted
145     frame["residual"] = residuals
146     frame["intercept"] = beta[0]
147     frame["slope_log_cost"] = beta[1]
148     frame["r_squared"] = r_squared
149     frame["adjusted_r_squared"] = adjusted
150     frame["rmse"] = rmse
151     frame["sample_size"] = len(y)
152     frame["model_formula"] = "S = beta0 + beta1 * ln(Cost)"
153     frame["inference_scope"] = "exploratory_description_only_n_equals_5"
154     return frame[[
155         "model_id", "model", "standard_workload_cost_usd", "q1_performance_score", "ln_cost
",
156         "fitted_performance", "residual", "intercept", "slope_log_cost", "r_squared",
157         "adjusted_r_squared", "rmse", "sample_size", "model_formula", "inference_scope",
158     ]]
159
160
161 def engineering_efficiency(attributes: pd.DataFrame, q1: pd.DataFrame) -> pd.DataFrame:
162     performance = q1.set_index("model_id")["topsis_score"].to_dict()
163     compatible = attributes.loc[
164         attributes["efficiency_compatible"]
165         & attributes["model_id"].map(lambda key: pd.notna(performance[key]))
166     ].copy()
167     compatible["ttft_benefit_score"] = benefit_minmax(compatible["comparable_ttft_seconds
"], False)
168     compatible["speed_benefit_score"] = benefit_minmax(compatible["
comparable_output_speed_tokens_per_second"], True)
169     compatible["latency_benefit_score"] = benefit_minmax(compatible["
comparable_total_latency_seconds"], False)
170     compatible["engineering_efficiency_score"] = compatible[

```

```

171     ["ttft_benefit_score", "speed_benefit_score", "latency_benefit_score"]
172 ].mean(axis=1)
173 compatible["q1_performance_score"] = compatible["model_id"].map(performance)
174 compatible["deployment_score"] = 0.7 * compatible["q1_performance_score"] + 0.3 *
175 compatible["engineering_efficiency_score"]
176 compatible["efficiency_rank"] = compatible["engineering_efficiency_score"].rank(method
177 = "min", ascending=False).astype(int)
178 compatible["deployment_rank"] = compatible["deployment_score"].rank(method="min",
179 ascending=False).astype(int)
180 compatible["analysis_status"] = "included_compatible_efficiency"
181 result = attributes[["model_id", "model_name"]].rename(columns={"model_name": "model"})
182 .merge(
183     compatible[[
184         "model_id", "ttft_benefit_score", "speed_benefit_score", "latency_benefit_score",
185         "engineering_efficiency_score", "q1_performance_score", "deployment_score",
186         "efficiency_rank", "deployment_rank", "analysis_status",
187     ]], on="model_id", how="left",
188 )
189 result["analysis_status"] = result["analysis_status"].fillna("
190 excluded_incompatible_or_missing_performance")
191 return result
192
193 def perturbed_q1_scores(normalized: pd.DataFrame, metrics: list[str], critic: dict[str,
194 float]):
195     for metric in metrics:
196         for factor in PERTURBATION_FACTORS:
197             adjusted = dict(critic)
198             adjusted[metric] *= factor
199             total = sum(adjusted.values())
200             adjusted = {key: value / total for key, value in adjusted.items()}
201             yield metric, factor, topsis_scores(normalized, metrics, adjusted)
202
203 def q3_sensitivity(
204     normalized: pd.DataFrame,
205     metrics: list[str],
206     critic: dict[str, float],
207     costs: dict[str, float],
208 ) -> pd.DataFrame:
209     rows = []
210     for metric, factor, ranking in perturbed_q1_scores(normalized, metrics, critic):
211         performance = ranking.set_index("model_id")["topsis_score"].to_dict()
212         ranks = ranking.set_index("model_id")["rank"].astype(int).to_dict()
213         frontier = []
214         for model in performance:
215             dominated = any(
216                 other != model and costs[other] <= costs[model] and performance[other] >=
217                 performance[model]
218                 and (costs[other] < costs[model] or performance[other] > performance[model]
219             )
220             for other in performance
221         )

```

```

216         if not dominated:
217             frontier.append(model)
218     row: dict[str, object] = {
219         "perturbed_metric": metric,
220         "factor": factor,
221         "kimi_score": performance["kimi-k3"],
222         "kimi_rank": ranks["kimi-k3"],
223         "kimi_on_pareto_frontier": "kimi-k3" in frontier,
224         "pareto_frontier_model_ids": ";".join(sorted(frontier)),
225     }
226     for budget in BUDGETS:
227         feasible = [model for model in performance if costs[model] <= budget]
228         winner = max(feasible, key=lambda model: (performance[model], -costs[model]))
229         row[f"budget_{int(budget)}_recommendation"] = winner
230     rows.append(row)
231     return pd.DataFrame(rows)
232
233
234 def kimi_cost_analysis(
235     performance_cost: pd.DataFrame,
236     pareto: pd.DataFrame,
237     budgets: pd.DataFrame,
238     engineering: pd.DataFrame,
239     sensitivity: pd.DataFrame,
240 ) -> pd.DataFrame:
241     base = performance_cost.loc[performance_cost["model_id"] == "kimi-k3"].iloc[0]
242     pareto_row = pareto.loc[pareto["model_id"] == "kimi-k3"].iloc[0]
243     engineering_row = engineering.loc[engineering["model_id"] == "kimi-k3"].iloc[0]
244     recommended = budgets.loc[budgets["recommended_model_id"] == "kimi-k3", "
    budget_limit_usd"].tolist()
245     return pd.DataFrame([
246         {
247             "model_id": "kimi-k3",
248             "model": "Kimi K3",
249             "q1_performance_score": base["q1_performance_score"],
250             "q1_rank": base["q1_rank"],
251             "standard_workload_cost_usd": base["standard_workload_cost_usd"],
252             "cost_rank_among_ranked": base["cost_rank_among_ranked"],
253             "performance_per_usd": base["performance_per_usd"],
254             "value_rank_among_ranked": base["value_rank_among_ranked"],
255             "pareto_status": pareto_row["pareto_status"],
256             "recommended_budget_levels_usd": ";".join(str(int(value)) for value in recommended)
257         },
258         {
259             "engineering_efficiency_score": engineering_row["engineering_efficiency_score"],
260             "efficiency_rank": engineering_row["efficiency_rank"],
261             "deployment_score": engineering_row["deployment_score"],
262             "deployment_rank": engineering_row["deployment_rank"],
263             "main_frontier_competitor": "Claude Fable 5",
264             "same_cost_competitor": "GPT-5.5",
265             "sensitivity_pareto_probability": float(sensitivity["kimi_on_pareto_frontier"].mean(
266                 )),
267             "sensitivity_budget_recommendation_rate": float(
268                 sensitivity[[column for column in sensitivity if column.startswith("budget_")
269                 ]].eq("kimi-k3").to_numpy().mean()
270             ),

```

```

266         "positioning": "高性能、同档最低成本、Pareto前沿; 预算达到20美元时由Claude Fable 5取代",
267     ]])
268
269
270 def final_summary(
271     root: Path,
272     q2_dir: Path,
273     q1: pd.DataFrame,
274     performance_cost: pd.DataFrame,
275     pareto: pd.DataFrame,
276     budgets: pd.DataFrame,
277     engineering: pd.DataFrame,
278 ) -> pd.DataFrame:
279     q2 = pd.read_csv(q2_dir / "scenario_rankings.csv", encoding="utf-8-sig")
280     summary = q1[["model_id", "model", "topsis_score", "rank"]].rename(columns={"
281     topsis_score": "q1_score", "rank": "q1_rank"})
282     for scenario, prefix in (("research_long_text", "research"), ("daily_dialogue", "
283     dialogue"), ("code_development", "coding")):
284         subset = q2.loc[q2["scenario"] == scenario, ["model_id", "topsis_score", "rank"]].
285         rename(
286             columns={"topsis_score": f"{prefix}_score", "rank": f"{prefix}_rank"}
287         )
288         summary = summary.merge(subset, on="model_id", how="left")
289     summary = summary.merge(performance_cost[["
290     "model_id", "standard_workload_cost_usd", "cost_rank_among_ranked", "
291     cost_benefit_index",
292     "performance_per_usd", "value_rank_among_ranked",
293     ]], on="model_id", how="left")
294     summary = summary.merge(pareto[["model_id", "pareto_status"]], on="model_id", how="left
295     ")
296     recommendations = budgets.groupby("recommended_model_id")["budget_limit_usd"].apply(
297         lambda values: ";".join(str(int(value)) for value in values)
298     ).to_dict()
299     summary["recommended_budget_levels_usd"] = summary["model_id"].map(recommendations).
300     fillna("")
301     summary = summary.merge(engineering[["
302     "model_id", "engineering_efficiency_score", "efficiency_rank", "deployment_score",
303     "deployment_rank",
304     ]], on="model_id", how="left")
305     return summary
306
307 def run_analysis(
308     root: Path,
309     output_dir: Path,
310     q2_dir: Path | None = None,
311     summary_path: Path | None = None,
312 ) -> dict[str, object]:
313     q2_dir = q2_dir or root / "results/q2"
314     summary_path = summary_path or root / "results/final_model_summary.csv"
315     attributes = pd.read_csv(root / "data/processed/model_attributes.csv", encoding="utf-8-
316     sig")
317     attributes["efficiency_compatible"] = attributes["efficiency_compatible"].astype(str).
318     str.lower().eq("true")

```

```

311 q1 = pd.read_csv(root / "results/q1/topsis_ranking.csv", encoding="utf-8-sig")
312 normalized = pd.read_csv(root / "results/q1/normalized_data.csv", encoding="utf-8-sig")
    .set_index("model_id")
313 weight_frame = pd.read_csv(root / "results/q1/critic_weights.csv", encoding="utf-8-sig")
314 metrics = weight_frame["metric"].tolist()
315 critic = weight_frame.set_index("metric")["weight"].astype(float).to_dict()
316 audit = cost_audit(attributes, q1)
317 performance_cost = performance_cost_table(audit, q1)
318 pareto = pareto_table(performance_cost)
319 budgets = budget_selection(performance_cost)
320 regression_frame = regression(performance_cost)
321 engineering = engineering_efficiency(attributes, q1)
322 costs = performance_cost.set_index("model_id")["standard_workload_cost_usd"].astype(
    float).to_dict()
323 sensitivity = q3_sensitivity(normalized, metrics, critic, costs)
324 kimi = kimi_cost_analysis(performance_cost, pareto, budgets, engineering, sensitivity)
325 summary = final_summary(root, q2_dir, q1, performance_cost, pareto, budgets,
    engineering)
326 outputs = {
327     "cost_data_audit.csv": audit,
328     "performance_cost.csv": performance_cost,
329     "pareto_analysis.csv": pareto,
330     "budget_selection.csv": budgets,
331     "performance_cost_regression.csv": regression_frame,
332     "deploy_scores.csv": engineering,
333     "kimi_k3_cost_analysis.csv": kimi,
334     "sensitivity_analysis.csv": sensitivity,
335 }
336 for name, frame in outputs.items():
337     write_csv(frame, output_dir / name)
338 write_csv(summary, summary_path)
339 return {
340     "attributes": attributes,
341     "q1": q1,
342     "normalized": normalized,
343     "weights": weight_frame,
344     "audit": audit,
345     "performance_cost": performance_cost,
346     "pareto": pareto,
347     "budgets": budgets,
348     "regression": regression_frame,
349     "engineering": engineering,
350     "kimi": kimi,
351     "sensitivity": sensitivity,
352     "summary": summary,
353 }

```

source/src/q3/reporting.py

```

1 """Question 3 report and model-to-code traceability mapping."""
2
3 from __future__ import annotations

```

```

4
5 from pathlib import Path
6
7 import pandas as pd
8
9 from src.q3.analysis import BUDGETS, INPUT_MILLION_TOKENS, OUTPUT_MILLION_TOKENS
10
11
12 def generate_report(result: dict[str, object], docs_dir: Path) -> Path:
13     docs_dir.mkdir(parents=True, exist_ok=True)
14     audit: pd.DataFrame = result["audit"] # type: ignore[assignment]
15     value: pd.DataFrame = result["performance_cost"] # type: ignore[assignment]
16     pareto: pd.DataFrame = result["pareto"] # type: ignore[assignment]
17     budgets: pd.DataFrame = result["budgets"] # type: ignore[assignment]
18     regression: pd.DataFrame = result["regression"] # type: ignore[assignment]
19     engineering: pd.DataFrame = result["engineering"] # type: ignore[assignment]
20     kimi: pd.Series = result["kimi"].iloc[0] # type: ignore[index,union-attr]
21     sensitivity: pd.DataFrame = result["sensitivity"] # type: ignore[assignment]
22     included = value.loc[value["analysis_status"] == "included_ranked_model"].sort_values("
performance_per_usd", ascending=False)
23     fit = regression.iloc[0]
24     frontier_names = ", ".join(pareto.loc[pareto["is_pareto_frontier"], "model"])
25     lines = [
26         "# 问题3性能—成本效益结果摘要",
27         "",
28         "## 1. 建模方法",
29         "",
30         "严格按 `model(1).md` §4: 直接采用问题1 TOPSIS 得分作为性能, 定义标准工作负载, 进行性能—
成本 Pareto 判定、预算约束选型、对数回归, 以及兼容配置模型的工程效率和 0.7/0.3 部署综合评价。
",
31         "",
32         "## 2. 成本数据与假设",
33         "",
34         f"标准工作负载为 {INPUT_MILLION_TOKENS:.1f} 百万输入 token + {OUTPUT_MILLION_TOKENS
:.1f} 百万输出 token; 成本公式为 `输入单价×1 + 输出单价×0.2`, 单位统一为 USD。6 个最终模型价
格均完整, 兼容效率记录为 {int(audit['efficiency_compatible'].sum())}/6。仓库没有可比能耗字
段, 因此未估算能耗。",
35         "",
36         "## 3. 性能成本评价",
37         "",
38         "| 模型 | Q1性能 | 标准成本 | 性能/美元 | 性价比排名 |",
39         "|---|---:|---:|---:|---:|",
40     ]
41     for row in included.itertuples():
42         lines.append(f"| {row.model} | {row.q1_performance_score:.10f} | ${row.
standard_workload_cost_usd:.3f} | {row.performance_per_usd:.6f} | {int(row.
value_rank_among_ranked)} |")
43     lines += [
44         "",
45         "## 4. Pareto前沿",
46         "",
47         f"Pareto 前沿模型为: {frontier_names}。被支配模型及其支配者记录在 `pareto_analysis.csv`
, 只有5个问题1完整排名模型参与判定。GLM-5.2 的标准成本为 $1.918, 但没有完整Q1性能得分, 只作低
价文本模型参照, 不进入前沿。",

```

```

48     "",
49     "## 5. 预算选型",
50     "",
51     "| 预算上限 | 可选模型数 | 推荐模型 | 性能 | 实际成本 |",
52     "|---:|---:|---|---:|---:|",
53 ]
54 for row in budgets.itertuples():
55     lines.append(f"| ${row.budget_limit_usd:.0f} | {row.feasible_models} | {row.
56 recommended_model} | {row.performance:.10f} | ${row.cost_usd:.3f} |")
57 lines += [
58     "",
59     "## 6. 性能成本关系",
60     "",
61     f"探索性回归为  $RS = \{fit['intercept']:.6f\} + \{fit['slope\_log\_cost']:.6f\} \ln(Cost)$ ,  $RS^2 = \{fit['r\_squared']:.6f\}$ , 调整  $RS^2 = \{fit['adjusted\_r\_squared']:.6f\}$ ,  $RMSE = \{fit['rmse']:.6f\}$ 。样本仅5个且拟合度偏低, 最多说明本样本中价格与性能的正相关较弱; 不能推断因果或普遍边际规律。",
62     "",
63     "## 7. 工程效率",
64     "",
65 ]
66 for row in engineering.loc[engineering["analysis_status"] == "
67 included_compatible_efficiency"].sort_values("deployment_rank").itertuples():
68     lines.append(f"- {row.model}: 效率分 {row.engineering_efficiency_score:.6f}, 部署综合
69 分 {row.deployment_score:.6f}, 部署排名第 {int(row.deployment_rank)}。")
70 lines += [
71     "",
72     "## 8. Kimi K3成本效益定位",
73     "",
74     f"Kimi K3 的Q1性能为 {kimi['q1_performance_score']:.10f} (第{int(kimi['q1_rank'])}
75 ), 标准成本 ${kimi['standard_workload_cost_usd']:.3f}, 在完整排名模型中成本并列第{int(kimi
76 ['cost_rank_among_ranked'])}、性能/美元第{int(kimi['value_rank_among_ranked'])}, 位于
77 Pareto 前沿。预算 6/10/12/16 美元时均推荐 Kimi K3; 预算达到20美元后推荐 Claude Fable 5。与
78 同价 GPT-5.5 相比, Kimi 性能更高并直接支配后者。",
79     "",
80     "## 9. 稳健性",
81     "",
82     "沿用 `model(1).md` §5.2 的36个 CRITIC 权重扰动场景, 逐次重算问题1性能、Pareto和预算推
83 荐。",
84     f"Kimi K3 保持 Pareto 前沿的比例为 {sensitivity['kimi_on_pareto_frontier'].mean()
85 :.2%}, 排名范围 {int(sensitivity['kimi_rank'].min())} - {int(sensitivity['kimi_rank'].max
86 ())}。",
87 ]
88 for budget in BUDGETS:
89     column = f"budget_{int(budget)}_recommendation"
90     dominant = sensitivity[column].value_counts(normalize=True)
91     model_id, probability = dominant.index[0], dominant.iloc[0]
92     lines.append(f"- ${budget:.0f} 预算: 最常推荐 {model_id}, 稳定率 {probability:.2%}。")
93 )
94 lines += [
95     "",
96     "## 10. 局限与论文结论",
97     "",
98     "成本结论依赖固定 token 比例及公开标价, 不含缓存、批处理、折扣、峰谷价和内部部署成本; 效率

```

分析只有4个兼容模型；无能耗数据；回归 $n=5$ 。给定这些边界，Kimi K3 是中低预算下的高性能高性价比选择，Claude Fable 5 是预算充足时的纯性能优先选择。”，

]

path = docs_dir / "q3_results_summary.md"

path.write_text("\n".join(lines) + "\n", encoding="utf-8")

return path

def generate_mapping(docs_dir: Path) -> Path:

lines = [

"# 建模方法—代码—结果映射",

""

"最高优先级方法规范为仓库根目录 `model(1).md` v1.1。",

""

"| 问题/步骤 | model 中的数学方法 | 代码文件 | 主要函数 | 输出文件 | 状态 |",

"|---|---|---|---|---|---|",

"| Q1 标准化 | 正/负向 Min-Max | `src/q1/analysis.py` | `minmax\_normalize` | `results/q1/normalized\_data.csv` | 已实现 |",

"| Q1 相关性与筛选 | Pearson + Spearman, 语义复核 | `src/q1/analysis.py` | `correlation\_matrices`, `select\_metrics` | `results/q1/\*correlation.csv`, `selected\_metrics.csv` | 已实现; 候选阈值按既有Q1任务为0.85, model §2.2 为0.9同维度删除门槛; 最终均保留9项 |",

"| Q1 客观赋权 | CRITIC: 标准差×冲突性 | `src/q1/analysis.py` | `critic\_weights` | `results/q1/critic\_weights.csv` | 已实现 |",

"| Q1 综合评价 | TOPSIS 正负理想解距离 | `src/q1/analysis.py` | `topsis\_scores`, `full\_ranking` | `results/q1/topsis\_ranking.csv` | 已实现 |",

"| Q1 Kimi分析 | 单项位次与领先差距 | `src/q1/analysis.py` | `kimi\_metric\_analysis` | `results/q1/kimi\_k3\_metric\_analysis.csv` | 已实现 |",

"| Q1 GLM-5.2补充评价 | 4项局部位次 + 独立HLE定性参照 | `src/q1/glm\_supplement.py` | `run\_analysis`, `generate\_figure` | `results/phase4b/glm\_partial\_comparison.csv`, `glm\_official\_hle\_note.csv` | 已实现; 不插补、不分配综合排名 |",

"| Q2 组合赋权 | $w^*_s = 0.5w_j + 0.5a_{s,j}$ | `src/q2/analysis.py` | `combined\_weights` | `results/q2/scenario\_weights.csv` | 已实现 |",

"| Q2 场景TOPSIS | 组合权重替代CRITIC权重 | `src/q2/analysis.py` | `scenario\_rankings` (复用Q1 `topsis\_scores`) | `results/q2/\*ranking.csv` | 已实现 |",

"| Q2 差异机理 | 权重变化×模型标准化值 | `src/q2/analysis.py` | `rank\_change\_analysis`, `contribution\_analysis` | `rank\_change\_analysis.csv`, `kimi\_k3\_scenario\_analysis.csv` | 已实现 |",

"| Q2 稳健性 | model §5.2 的36种客观权重扰动向场景组合传播 | `src/q2/analysis.py` | `sensitivity\_analysis` | `scenario\_sensitivity.csv`, `scenario\_stability.csv` | 已实现 |",

"| Q3 标准成本 | $Cost = P_{in} + 0.2P_{out}$ | `src/q3/analysis.py` | `cost\_audit`, `performance\_cost\_table` | `cost\_data\_audit.csv`, `performance\_cost.csv` | 已实现 |",

"| Q3 Pareto | 成本更低且性能更高的两两支配 | `src/q3/analysis.py` | `pareto\_table` | `pareto\_analysis.csv` | 已实现 |",

"| Q3 预算选型 | $\arg\max S_i, Cost_i \leq B$ | `src/q3/analysis.py` | `budget\_selection` | `budget\_selection.csv` | 已实现 |",

"| Q3 成本规律 | $S = \beta_0 + \beta_1 \ln(Cost)$ 最小二乘 | `src/q3/analysis.py` | `regression` | `performance\_cost\_regression.csv` | 已实现 |",

"| Q3 工程效率 | 三项正向化等权, $Deploy = 0.7S + 0.3E$ | `src/q3/analysis.py` | `engineering\_efficiency` | `deploy\_scores.csv` | 已实现 |",

"| Q3 稳健性 | 36种权重扰动下重算前沿与预算推荐 | `src/q3/analysis.py` | `q3\_sensitivity` | `sensitivity\_analysis.csv` | 已实现 |",

"| 全局稳健性: 熵权法 | 熵权重重跑TOPSIS并与CRITIC对照 | `scripts/`

```

analyze_phase7_robustness.py` | `entropy_weights`, `analyze` | `results/phase7/
method_comparison.csv`, `weight_method_comparison.csv` | 已实现 (输出名与model建议名不同)
|",
119     "| 全局稳健性: 逐项扰动 | 9指标x4因子, 共36场景, Kendall  $\tau$  | `scripts/
analyze_phase7_robustness.py` | `kendall_tau`, `analyze` | `
weight_perturbation_rankings.csv`, `rank_correlation.csv` | 已实现 |",
120     "| Q1 - Q3 汇总 | 按 `model_id` 对齐全部评价 | `src/q3/analysis.py` | `final_summary`
| `results/final_model_summary.csv` | 已实现 |",
121     "",
122     "## 一致性说明",
123     "",
124     "Q2、Q3 的公式、参数、参与模型范围与 `model(1).md` 一致。问题1保持用户确认后的主排名不
变; 其高相关候选复核阈值来自先前Q1任务 (0.85), 与 model §2.2 的0.9删除门槛存在表述差异, 但两
套规则都没有删除指标, 因此Q2/Q3的输入矩阵、CRITIC权重和TOPSIS得分完全一致。GLM-5.2 Phase4b 已
补充四项局部位次与独立HLE口径说明, 但仍不插补或分配综合排名。model 中的3D图属于可选增强, 本轮未
生成。实现复用现有Q1的 NumPy/pandas/Matplotlib依赖, 而非 model §10.5 偏好的纯标准库; 这不改
变数学口径。",
125 ]
126 path = docs_dir / "model_implementation_mapping.md"
127 path.write_text("\n".join(lines) + "\n", encoding="utf-8")
128 return path

```

source/src/q3/visualization.py

```

1  """Paper-ready Question 3 visualizations."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import matplotlib.pyplot as plt
8  import numpy as np
9  import pandas as pd
10
11  from src.q1.visualization import configure_style, save
12
13
14  def pareto_plot(frame: pd.DataFrame, path: Path) -> None:
15      fig, ax = plt.subplots(figsize=(9.5, 6.5))
16      dominated = frame.loc[~frame["is_pareto_frontier"]]
17      frontier = frame.loc[frame["is_pareto_frontier"]].sort_values("
standard_workload_cost_usd")
18      ax.scatter(dominated["standard_workload_cost_usd"], dominated["q1_performance_score"],
s=90, color="#94a3b8", label="被支配模型")
19      ax.plot(frontier["standard_workload_cost_usd"], frontier["q1_performance_score"], color
="#dc2626", linewidth=2.2, marker="o", markersize=10, label="Pareto 前沿")
20      for row in frame.itertuples():
21          ax.annotate(row.model, (row.standard_workload_cost_usd, row.q1_performance_score),
xytext=(6, 7), textcoords="offset points")
22      ax.set_xlim(left=0)
23      ax.set_xlabel("标准工作负载成本 (USD)")
24      ax.set_ylabel("问题1 TOPSIS 综合性能")
25      ax.set_title("大模型性能—成本 Pareto 前沿")

```

```

26     ax.grid(alpha=0.2)
27     ax.legend()
28     fig.tight_layout()
29     save(fig, path)
30
31
32 def budget_plot(frame: pd.DataFrame, path: Path) -> None:
33     fig, ax = plt.subplots(figsize=(9.5, 5.8))
34     ax.step(frame["budget_limit_usd"], frame["performance"], where="post", linewidth=2.5,
35            color="#2563eb")
36     ax.scatter(frame["budget_limit_usd"], frame["performance"], s=70, color="#dc2626",
37              zorder=3)
38     for row in frame.itertuples():
39         ax.annotate(row.recommended_model, (row.budget_limit_usd, row.performance), xytext
40                    =(4, 9), textcoords="offset points", fontsize=9)
41     ax.set_xticks(frame["budget_limit_usd"])
42     ax.set_ylim(0, 0.82)
43     ax.set_xlabel("预算上限 (USD / 标准工作负载)")
44     ax.set_ylabel("推荐模型的Q1性能得分")
45     ax.set_title("预算约束下的最优模型选择")
46     ax.grid(alpha=0.2)
47     fig.tight_layout()
48     save(fig, path)
49
50
51 def regression_plot(frame: pd.DataFrame, path: Path) -> None:
52     fig, ax = plt.subplots(figsize=(9.5, 6.2))
53     ax.scatter(frame["standard_workload_cost_usd"], frame["q1_performance_score"], s=100,
54              color="#2563eb")
55     for row in frame.itertuples():
56         ax.annotate(row.model, (row.standard_workload_cost_usd, row.q1_performance_score),
57                    xytext=(5, 7), textcoords="offset points")
58     costs = np.linspace(float(frame["standard_workload_cost_usd"].min()), float(frame["
59 standard_workload_cost_usd"].max()), 200)
60     first = frame.iloc[0]
61     fit = first["intercept"] + first["slope_log_cost"] * np.log(costs)
62     ax.plot(costs, fit, color="#dc2626", linewidth=2.2, label="对数拟合")
63     ax.text(
64         0.03, 0.95,
65         f"S = {first['intercept']:.4f} + {first['slope_log_cost']:.4f} ln(Cost)\nR^2$ = {
66 first['r_squared']:.4f}, RMSE = {first['rmse']:.4f}",
67         transform=ax.transAxes, va="top", bbox={"facecolor": "white", "alpha": 0.85, "
68 edgecolor": "#cbd5e1"},
69     )
70     ax.set_xlim(left=0)
71     ax.set_xlabel("标准工作负载成本 (USD)")
72     ax.set_ylabel("问题1 TOPSIS 综合性能")
73     ax.set_title("成本—性能对数回归 (探索性, n=5)")
74     ax.legend()
75     ax.grid(alpha=0.2)
76     fig.tight_layout()
77     save(fig, path)

```

```

72 def efficiency_plot(frame: pd.DataFrame, path: Path) -> None:
73     included = frame.loc[frame["analysis_status"] == "included_compatible_efficiency"].copy
74     ()
75     columns = ["ttft_benefit_score", "speed_benefit_score", "latency_benefit_score", "
76     deployment_score"]
77     labels = ["TTFT正向化", "输出速度正向化", "总延迟正向化", "部署综合分"]
78     x = np.arange(len(included))
79     width = 0.19
80     fig, ax = plt.subplots(figsize=(11, 6.3))
81     for index, (column, label) in enumerate(zip(columns, labels)):
82         ax.bar(x + (index - 1.5) * width, included[column], width, label=label)
83     ax.set_xticks(x, included["model"], rotation=15, ha="right")
84     ax.set_ylim(0, 1.08)
85     ax.set_ylabel("标准化得分")
86     ax.set_title("兼容配置模型的工程效率与部署综合分")
87     ax.legend(ncols=2)
88     ax.grid(axis="y", alpha=0.2)
89     fig.tight_layout()
90     save(fig, path)
91
92 def generate_figures(result: dict[str, object], output_dir: Path) -> list[Path]:
93     configure_style()
94     paths = [
95         output_dir / "performance_cost_pareto.png",
96         output_dir / "budget_selection.png",
97         output_dir / "performance_cost_fit.png",
98         output_dir / "engineering_efficiency.png",
99     ]
100     pareto_plot(result["pareto"], paths[0]) # type: ignore[arg-type]
101     budget_plot(result["budgets"], paths[1]) # type: ignore[arg-type]
102     regression_plot(result["regression"], paths[2]) # type: ignore[arg-type]
103     efficiency_plot(result["engineering"], paths[3]) # type: ignore[arg-type]
104     return paths

```